

R Course: Beginner to Expert

Sri Ram

2025-11-20

Table of contents

1	Welcome	8
2	About this Course	9
2.1	How to Use	9
3	R Basics	10
4	R as a Calculator	11
5	Objects & Assignment	12
6	object naming rules	13
7	Basic Operations in R	14
8	Comments in R	16
9	Getting Help in R	17
10	Installing and Loading Packages in R	19
10.0.1	Saving and Loading Workspaces in R	19
11	Working Directory	20
12	Vectors (Atomic)	21
13	Exercises	22
14	Data Types & Data Structures	23
15	Vectors	24
16	Lists	38
17	Matrices	50
18	Data Frames	71

19 Factors	80
20 Arrays	82
21 Summary	83
22 Manipulating Vectors, Data Frames, and Lists	84
23 Vectors: Indexing & Vectorized Ops	85
24 Data Frames with dplyr	104
25 Read sas dataset	118
26 keeping selected columns	119
27 dropping columns	120
28 Rename	122
29 Arrange (sorting)	123
30 convert var names to lower case	124
31 Creating new variables	125
32 bind rows (set operator in SAS)	126
33 recode (similar to PROC FORMAT in SAS)	127
34 factor levels	128
35 joins	129
35.1 innter join	130
35.2 left join	130
35.3 right join	131
35.4 Full Join	131
35.5 Semi Join	131
35.6 Anti join	132
36 Reshaping data with tidyr	134
37 counting with n()	135
38 dplyr distinct()	137

39	case_when()	138
40	create seq no.	139
41	Lists: lapply, purrr	140
42	Exercises	141
43	String Functions	142
44	String Functions in Base R	143
44.1	Basic String Functions	143
44.2	Pattern Matching and Replacement	144
44.3	String Splitting and Joining	145
44.4	Regular Expressions	145
44.5	Example Usage	147
45	Overall differences	148
46	Detect matches	151
46.1	str_detect(): Detect the presence or absence of a pattern in a string	151
46.2	str_which(): Find positions matching a pattern	151
46.3	str_count(): Count the number of matches in a string	152
46.4	str_locate(): Locate the position of patterns in a string	152
46.5	str_subset(): Keep strings matching a pattern, or find positions	153
46.6	str_extract(): Extract matching patterns from a string	154
46.7	str_match(): Extract matched groups from a string	155
47	Manage lengths	157
47.1	str_length(): The length of a string	157
47.2	str_pad(): Pad a string	158
47.3	str_trunc(): Truncate a character string	159
47.4	str_trim(): Trim whitespace from a string	159
47.5	str_wrap(): Wrap strings into nicely formatted paragraphs	160
48	Mutate strings	162
48.1	str_replace(): Replace matched patterns in a string	162
48.2	case: Convert case of a string	162
49	Join and split	165
49.1	str_flatten(): Flatten a string	165
49.2	str_dup(): duplicate strings within a character vector	165
49.3	str_split(): Split up a string into pieces	166
49.4	str_glue(): Interpolate strings	167

50 Order strings	169
50.1 <code>str_order()</code> : Order or sort a character vector	169
51 date and time functions	173
51.0.1 Date and Time Base Functions in R	173
51.0.2 Creating Date and Time Objects	173
51.0.3 Year Formats	173
51.0.4 Month Formats	173
51.0.5 Day Formats	174
51.0.6 Weekday Formats	174
51.0.7 Time Formats	174
51.1 1. Date & Time Basics in R	178
51.1.1 1.1 Core classes	178
51.1.2 1.2 Converting numerics to Date (origins matter)	178
51.1.3 1.3 Custom date formats	179
51.1.4 1.4 Datetime strings (ISO 8601 & time zones)	179
51.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic	179
51.2.1 2.1 Intuitive parsers	179
51.2.2 2.2 Accessors and rounding	180
51.2.3 2.3 Timespans	180
51.2.4 3 ADaM-Style Partial Date Imputation (Start & End)	180
51.2.5 Start date (AESTDTC)	180
51.2.6 End date (AEENDTC)	181
51.2.7 Censoring by death / last alive date	181
51.2.8 Imputation flags	181
51.3 4. Durations & Analysis Variables	183
51.3.1 4.1 AE duration (AEDUR) in days	183
52 Using <code>difftime</code> (base) or <code>as.numeric</code> on Date differences	184
52.0.1 6.2 Handy lubridate parsers (subset)	184
53 Reading SAS Datasets (+ Cleaning)	185
53.1 Handling Labels & Missing	187
53.2 Labelled to Factor (if needed)	187
53.3 Common Cleaning	187
54 Exercises	189
55 Base R Functions & Apply Family	190
56 Common Utilities	191
57 Apply Family	192

58 Subsetting Essentials	193
59 Exercises	194
60 Custom Functions & Validation	195
61 Function introduction	196
61.1 Function components	196
61.2 Signalling conditions	197
61.3 Ignoring conditions	198
61.4 tryCatch function	198
61.5 function scope	199
61.6 Creating Custom Functions	200
61.7 Function Arguments	200
61.8 User-defined Functions in R Programming Language	200
62 difference !,!!,!!!	205
63 Error Handling	209
64 Unit Testing with testthat	210
65 Document with roxygen2	211
66 Exercises	212
67 R Package Development	213
67.1 1. Prerequisites	213
67.2 2. Create the Package Skeleton	213
67.2.1 2.1 Create a new package directory	213
67.2.2 2.2 Initialize Git and a README	214
67.2.3 2.3 The minimal structure (what you get)	214
67.3 3. DESCRIPTION: Package Metadata	214
67.4 4. NAMESPACE: Exports and Imports (auto-generated)	215
67.5 5. Write Functions + roxygen2 Headers	215
67.6 6. Generate Documentation	216
67.7 7. Add Data, Vignettes, Tests (Optional but Recommended)	217
67.7.1 7.1 Package data	217
67.7.2 7.2 Vignettes	217
67.7.3 7.3 Tests with testthat	217
67.8 8. Manage Dependencies	218
67.9 9. Check Your Package Thoroughly	218
67.10 10. Build & Install Locally	219
67.11 11. Continuous Integration (optional)	219

67.1212. Package Website (optional)	219
67.1313. Release Checklist	220
67.13.1 Appendix A — Common Files (at a glance)	220
67.13.2 Appendix B — Troubleshooting Snippets	220
68 Git in RStudio (Setup & Auth)	222
68.1 One-Time Setup	222
68.2 Initialize Git for the Current Project	222
68.3 Connect to GitHub	222
68.4 Typical Workflow	222
68.5 Remove Git from a Project (macOS/RStudio)	223
69 Creating ADaM: ADSL from SDTM-like Inputs	224
69.1 Build ADSL	225
70 TLFs: Table, Figure, Listing	227
70.1 Create demographic table from ADSL	227
71 Capstone: End-to-End Mini Workflow	232
71.1 Parameters	232
71.2 1) Read (or Synthesize) SDTM	232
71.3 2) Derive ADSL (Minimal Demo)	233
71.4 3) TLFs	234
71.5 4) Save Outputs	235
72 Appendix: Tips, Profiles, .libPaths	236
72.1 Useful Profiles	236
72.2 Custom Library Paths	236
72.3 Format vs formatC (quick recap)	236
72.4 POSIXct vs POSIXlt	237
72.5 Recommended Packages	237
72.6 Short Glossary	237

1 Welcome

2 About this Course

This Course from **R beginner** to **expert**, with a practical focus on **clinical programming** (CDISC/ADaM and TLFs).

Each chapter includes step-by-step explanations, runnable code, and short exercises.

2.1 How to Use

1. Install: R (4.2), RStudio (or VS Code).
2. Follow along chapter-by-chapter, running code and completing exercises.
3. Use sample data or your own clinical trial datasets (SAS/CSV). ## Structure (Highlights)
 - **Basics & Data:** R syntax, data types/structures, vectors/data frames/lists.
 - **I/O:** Read SAS datasets (with **haven**), handle labels, and clean raw data.
 - **Programming:** Base functions, write your own functions, validate with tests.
 - **DevOps:** Create an R package, connect Git in RStudio/GitHub.
 - **CDISC:** Build ADaM (ADSL) from SDTM-like inputs.
 - **TLFs:** Produce a baseline Table 1, a KM plot, and a listing.

Tip: If you don't have sample SDTM/ADaM data yet, the chapters generate **small synthetic data** as a fallback so everything runs end-to-end. ## contact For questions or feedback, reach out to **r2sas2025@gmail.com**

3 R Basics

4 R as a Calculator

```
1 + 1
```

```
[1] 2
```

```
3 * (4 + 5)
```

```
[1] 27
```

5 Objects & Assignment

```
x <- 10  
y <- 3.5  
x + y
```

```
[1] 13.5
```

6 object naming rules

- R variable names can contain letters, numbers, periods, and underscores. However, they cannot start with a number or underscore. R is case-sensitive, so `age`, `Age`, and `AGE` would be considered different variables.
- R variable names should be descriptive and meaningful. Avoid using reserved words or function names as variable names.
- A variable can have a short name (like `x` and `y`) or a more descriptive name (`age`, `carname`, `total_volume`). Rules for R variables are:
- A variable name must start with a letter and can be a combination of letters, digits, period(`.`) and underscore(`_`). If it starts with period(`.`), it cannot be followed by a digit.
- A variable name cannot start with a number or underscore (`_`) Variable names are case-sensitive (`age`, `Age` and `AGE` are three different variables) Reserved words cannot be used as variables (`TRUE`, `FALSE`, `NULL`, `if...`)
- Variable names should not contain spaces. Use underscore (`_`) or period (`.`) to separate words in a variable name.
- Variable names should be meaningful and descriptive. Avoid using single-letter variable names except for temporary variables in loops or functions.

7 Basic Operations in R

R supports various basic operations, including: * Arithmetic Operations: Addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (^). Example:

```
a <- 10
b <- 5
sum <- a + b
diff <- a - b
prod <- a * b
quot <- a / b
exp <- a ^ b
sum; diff; prod; quot; exp
```

```
[1] 15
```

```
[1] 5
```

```
[1] 50
```

```
[1] 2
```

```
[1] 1e+05
```

- Comparison Operations: Equal to (==), not equal to (!=), greater than (>), less than (<), greater than or equal to (>=), and less than or equal to (<=). Example:

```
x <- 10
y <- 5
eq <- x == y
neq <- x != y
gt <- x > y
lt <- x < y
gte <- x >= y
lte <- x <= y
eq; neq; gt; lt; gte; lte
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] TRUE
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

- Logical Operations: AND (&), OR (|), and NOT (!). Example:

```
p <- TRUE  
q <- FALSE  
and <- p & q  
or <- p | q  
not <- !p  
and; or; not
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

8 Comments in R

Comments in R are created using the `#` symbol. Anything following the `#` on the same line is considered a comment and is ignored by R during execution. Example:

```
# This is a comment
x <- 10 # Assigning value to x
y <- 5  # Assigning value to y
sum <- x + y # Calculating the sum of x and y
sum # Output the sum
```

```
[1] 15
```


9 Getting Help in R

R provides several ways to get help and documentation for functions and packages: *

- `?function_name`: Displays the documentation for a specific function. Example:

```
?mean
```

- `help(function_name)`: Another way to access the documentation for a function. Example:

```
help(mean)
```

- `help.search("keyword")`: Searches for help topics related to a specific keyword. Example:

```
help.search("regression")
```

- `example(function_name)`: Shows examples of how to use a specific function. Example:

```
example(mean)
```

```
mean> x <- c(0:10, 50)
```

```
mean> xm <- mean(x)
```

```
mean> c(xm, mean(x, trim = 0.10))  
[1] 8.75 5.50
```

- `vignette("package_name")`: Opens the vignette (detailed documentation) for a specific package. Example:

```
vignette("dplyr")
```

```
starting httpd help server ... done
```

- `??keyword`: Searches for help topics related to a specific keyword (similar to `help.search`). Example:

```
??regression
```

10 Installing and Loading Packages in R

R has a vast ecosystem of packages that extend its functionality. To use a package, you need to install it first and then load it into your R session. * Installing a Package: Use the `install.packages("package_name")` function to install a package from CRAN. Example:

```
install.packages("ggplot2")
```

- Loading a Package: Use the `library(package_name)` function to load an installed package into your R session. Example:

```
library(ggplot2)
```

```
# Now you can use functions from the ggplot2 package
```

10.0.1 Saving and Loading Workspaces in R

You can save your R workspace (all objects in memory) to a file and load it later * Saving Workspace: Use the `save.image("file_name.RData")` function to save the entire workspace to a file. Example:

```
save.image("my_workspace.RData")
```

- Loading Workspace: Use the `load("file_name.RData")` function to load a saved workspace from a file. Example:

```
load("my_workspace.RData")
```

11 Working Directory

```
getwd()
```

```
[1] "/home/runner/work/r4sas/r4sas"
```

```
# setwd("/path/you/want") # avoid in reproducible code; prefer here::here() for projects
```

12 Vectors (Atomic)

```
nums <- c(1, 2, 3, 4)
chars <- c("a", "b", "c")
logical <- c(TRUE, FALSE, TRUE)
typeof(nums); typeof(chars); typeof(logical)
```

[1] "double"

[1] "character"

[1] "logical"

13 Exercises

1. Create an object `z` that stores $(5^2 + 7)/3$.
2. Use `?seq` and create a sequence from 0 to 1 by 0.1.
3. Inspect `typeof()` for a few objects you create.

14 Data Types & Data Structures

R has several built-in data structures to store and manipulate different types of data. These include vectors, lists, matrices, data frames, and factors. Below is an overview of each structure along with code examples.

15 Vectors

Vectors are the simplest data structure in R. They store elements of the same type (numeric, character, logical, etc.).

```
# Creating numeric and character vectors
numeric_vector <- c(1, 2, 3, 4)
char_vector1 <- c("apple", "banana", "cherry")
char_vector2 <- c(2, 3, 4, 5, "a")
logical_vector <- c(TRUE, FALSE, TRUE)

# Accessing elements
numeric_vector[1] # Access the first element
```

```
[1] 1
```

```
v_logical <- c(T,F,T) # logical vector
v_logical
```

```
[1] TRUE FALSE TRUE
```

```
is.vector(v_logical)
```

```
[1] TRUE
```

```
is.atomic(v_logical)
```

```
[1] TRUE
```

```
typeof(v_logical)
```

```
[1] "logical"
```



```
v_integer <- c(1L,2L,5L) # integer vector  
v_integer
```

```
[1] 1 2 5
```

```
is.vector(v_integer)
```

```
[1] TRUE
```

```
is.atomic(v_integer)
```

```
[1] TRUE
```

```
typeof(v_integer)
```

```
[1] "integer"
```

```
v_double <- c(1.3,2.1,5.0) # double vector  
v_double
```

```
[1] 1.3 2.1 5.0
```

```
is.vector(v_double)
```

```
[1] TRUE
```

```
is.atomic(v_double)
```

```
[1] TRUE
```

```
typeof(v_double)
```

```
[1] "double"
```

```
v_character <- c("a", "b", "c") # character vector
v_character
```

```
[1] "a" "b" "c"
```

```
is.vector(v_character)
```

```
[1] TRUE
```

```
is.atomic(v_character)
```

```
[1] TRUE
```

```
typeof(v_character)
```

```
[1] "character"
```

```
v_NULL <- NULL # NULL
v_NULL
```

```
NULL
```

```
typeof(v_NULL)
```

```
[1] "NULL"
```

```
# Mix type vector (type coercion or conversion)
v_mix <- c(T, 1L, 1.25, "a")
v_mix # all elements converted to characters (based on hierarchy)
```

```
[1] "TRUE" "1"    "1.25" "a"
```

```
is.vector(v_mix)
```

```
[1] TRUE
```

```
typeof(v_mix)
```

```
[1] "character"
```

```
# Vector properties
```

```
v <- c(1,2,3,4,5)
```

```
# vector length
```

```
length(v)
```

```
[1] 5
```

```
# type
```

```
typeof(v)
```

```
[1] "double"
```

```
class(v)
```

```
[1] "numeric"
```

```
# naming elements
```

```
names(v) # without names
```

```
NULL
```

```
vnames <- c("first", "second", "third", "fourth", "fifth") # element names
```

```
names(v) <- vnames # naming elements
```

```
v
```

```
first second  third fourth  fifth
      1      2      3      4      5
```

```
names(v) # new names
```

```
[1] "first" "second" "third" "fourth" "fifth"
```

```
# Create vector, access elements, modify vector
```

```
# create using c()
```

```
v <- c(1,3,5,8,0)
```

```
# create using operator :
```

```
1:100
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
[19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
[37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
[55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
[73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
[91] 91 92 93 94 95 96 97 98 99 100
```

```
10:-10
```

```
[1] 10 9 8 7 6 5 4 3 2 1 0 -1 -2 -3 -4 -5 -6 -
7 -8
[20] -9 -10
```

```
# using sequence seq()
```

```
v <- seq(from = 1, to = 100, by = 1)
```

```
v
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
[19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
[37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
[55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
[73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
[91] 91 92 93 94 95 96 97 98 99 100
```

```
v <- seq(from = 0, to = 1, by = 0.01)
```

```
v
```

```
[1] 0.00 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.10 0.11 0.12 0.13 0.14
[16] 0.15 0.16 0.17 0.18 0.19 0.20 0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29
[31] 0.30 0.31 0.32 0.33 0.34 0.35 0.36 0.37 0.38 0.39 0.40 0.41 0.42 0.43 0.44
[46] 0.45 0.46 0.47 0.48 0.49 0.50 0.51 0.52 0.53 0.54 0.55 0.56 0.57 0.58 0.59
[61] 0.60 0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.70 0.71 0.72 0.73 0.74
[76] 0.75 0.76 0.77 0.78 0.79 0.80 0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89
[91] 0.90 0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1.00
```

```
v <- seq(from = 0, to = 10, length.out = 5)
v
```

```
[1] 0.0 2.5 5.0 7.5 10.0
```

```
# let's create a vector for accessing vector elements
v <- 1:10
names(v) <- c("a", "b", "c", "d", "e", "f", "g", "h", "i", "j")
v
```

```
 a  b  c  d  e  f  g  h  i  j
1  2  3  4  5  6  7  8  9 10
```

```
# access vector elements using integer vector index
v[c(1,5,10)]
```

```
 a  e  j
1  5 10
```

```
v[1:5] # range index selection (slicing)
```

```
 a b c d e
1 2 3 4 5
```

```
v[seq(from = 1, to = 9, by = 2)]
```

```
 a c e g i
1 3 5 7 9
```

```
v[10:1] # reverse order selection
```

```
 j  i  h  g  f  e  d  c  b  a
10 9  8  7  6  5  4  3  2  1
```

```
v[c(10,1,5,3)] # mix order selection
```

```
 j  a  e  c
10 1  5  3
```

```
# access vector elements using logical vector index
v[c(T,F,F,F,F,F,F,F,F)] # access first element
```

```
a
1
```

```
v[c(F,F,F,F,F,F,F,T,T,T)] # access last three elements
```

```
h i j
8 9 10
```

```
# access elements using names
v[c("a","c","e")]
```

```
a c e
1 3 5
```

```
v[c("a", "b", "c", "d", "e", "f", "g", "h", "i", "j")]
```

```
a b c d e f g h i j
1 2 3 4 5 6 7 8 9 10
```

```
# modify vector elements
v
```

```
a b c d e f g h i j
1 2 3 4 5 6 7 8 9 10
```

```
v[2] <- 20 # alter second element
v
```

```
a b c d e f g h i j
1 20 3 4 5 6 7 8 9 10
```

```
v[c(1,5,10)] <- c(0,0,0) # alter multiple elements
v
```

```
a b c d e f g h i j
0 20 3 4 0 6 7 8 9 0
```

```
# modify elements with value 0
v
```

a	b	c	d	e	f	g	h	i	j
0	20	3	4	0	6	7	8	9	0

```
v[v==0] # filter with condition
```

a	e	j
0	0	0

```
v[v==0] <- 1000
v
```

a	b	c	d	e	f	g	h	i	j
1000	20	3	4	1000	6	7	8	9	1000

```
# truncate vector to first 3 elements
v <- v[1:3]
v
```

a	b	c
1000	20	3

```
# transpose vector change row to column vector or vice versa
v
```

a	b	c
1000	20	3

```
t(v)
```

	a	b	c
[1,]	1000	20	3

```
# delete or remove a vector
v <- NULL
v
```

```
NULL
```

```
rm(v)

# combine 2 different vectors
v1 <- 1:3
v2 <- 100:105
v1
```

```
# combine 2 different vectors
v1 <- 1:3
v2 <- 100:105
v1
```

```
v1 <- 1:3
v2 <- 100:105
v1
```

```
v2 <- 100:105
v1
```

[1] 1 2 3

v2

```
[1] 100 101 102 103 104 105
```

```
v3 <- c(v1,v2) # combine vectors
v3
```

```
[1] 1 2 3 100 101 102 103 104 105
```

```
# repet elements of a vector
rep(x = v1, times = 2)
```

```
rep(x = v1, times = 2)
```

[1] 1 2 3 1 2 3

```
rep(x = v1, times = 100)
```

[1] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
[38] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[75] 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[112] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[149] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[186] 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
[223] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
[260] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
[297] 3 1 2 3

```
rep(10,10)
```

```
[1] 10 10 10 10 10 10 10 10 10 10
```



```
# Vector arithmetics
```

```
# vector - scalar (scalar with each vector element)
```

```
v <- 1:5
```

```
a <- 10
```

```
v
```

```
[1] 1 2 3 4 5
```

```
a
```

```
[1] 10
```

```
# Addition +
```

```
v + a
```

```
[1] 11 12 13 14 15
```

```
# Subtraction -
```

```
v - a
```

```
[1] -9 -8 -7 -6 -5
```

```
# Multiplication *
```

```
v * a
```

```
[1] 10 20 30 40 50
```

```
# Division /
```

```
v / a
```

```
[1] 0.1 0.2 0.3 0.4 0.5
```

```
# Exponent ^ **
```

```
v^a
```

```
[1]      1    1024  59049 1048576 9765625
```

```
# Modulus (Remainder from division) %%  
v %% 2
```

```
[1] 1 0 1 0 1
```

```
# Integer Division %/%  
v %/% 2
```

```
[1] 0 1 1 2 2
```

```
# Other functions on vector elements  
sqrt(v)
```

```
[1] 1.000000 1.414214 1.732051 2.000000 2.236068
```

```
log(v)
```

```
[1] 0.0000000 0.6931472 1.0986123 1.3862944 1.6094379
```

```
sum(v)
```

```
[1] 15
```

```
# vector - vector (vector element to element | member-by-member)  
v1 <- seq(10,30,10)  
v2 <- rep(3,3)  
  
# Addition +  
v1 + v2
```

```
[1] 13 23 33
```

```
# Subtraction -  
v1 - v2
```

```
[1] 7 17 27
```

```
# Multiplication *  
v1 * v2
```

```
[1] 30 60 90
```

```
# Division /  
v1 / v2
```

```
[1] 3.333333 6.666667 10.000000
```

```
# Exponent ^ **  
v1^v2
```

```
[1] 1000 8000 27000
```

```
# Modulus (Remainder from division) %%  
v1 %% v2
```

```
[1] 1 2 0
```

```
# Integer Division %/%  
v1 %/% v2
```

```
[1] 3 6 10
```

```
# Vector-matrix style multiplication  
v1
```

```
[1] 10 20 30
```

```
v2
```

```
[1] 3 3 3
```

```
10*3 + 20*3 + 30*3
```

```
[1] 180
```

```
t(v1) %*% v2
```

```
      [,1]  
[1,] 180
```

```
v1 %*% v2
```

```
      [,1]  
[1,] 180
```

```
v1 %*% t(v2)
```

```
      [,1] [,2] [,3]  
[1,]    30    30    30  
[2,]    60    60    60  
[3,]    90    90    90
```

```
# Recycling rule
```

```
v1 <- c(1,1,1)
```

```
v2 <- 1:6
```

```
v1
```

```
[1] 1 1 1
```

```
v2
```

```
[1] 1 2 3 4 5 6
```

```
v1 + v2
```

```
[1] 2 3 4 5 6 7
```

```
# Set operations
```

```
v1 <- c("a", "b", "c")
```

```
v2 <- c("c", "d", "e")
```

```
v1
```

```
[1] "a" "b" "c"
```

```
v2
```

```
[1] "c" "d" "e"
```

```
union(v1,v2) # union of both sets (all unique elements)
```

```
[1] "a" "b" "c" "d" "e"
```

```
intersect(v1,v2) # intersection of both sets (elements in both sets)
```

```
[1] "c"
```

```
setdiff(v1,v2) # difference of elements (elements in v1 and not in v2)
```

```
[1] "a" "b"
```

```
identical(v1, v2) # check if vectors are identical
```

```
[1] FALSE
```

```
identical(c(1,2,3), c(1,2,3))
```

```
[1] TRUE
```

16 Lists

A list can contain elements of different types, including other lists or vectors or data structures.

```
# Creating a list
my_list <- list(name = "John", age = 25, scores = c(90, 85, 88))

# Accessing elements by name
my_list$name # Output: "John"
```

```
[1] "John"
```

```
# Create a list (and name elements)

# lets create some variables (different types)
a <- 10
b <- 2L
c <- TRUE
d <- "word"
v <- 1:10
names(v) <- paste("i", v, sep = "")
M <- matrix(data = seq(10,40,by = 10), nrow = 2, dimnames = list(c("r1", "r2"), c("c1", "c2")))
A <- array(data = 1:8, dim = c(2,2,2), dimnames = list(c("r1", "r2"), c("c1", "c2"), c("M1", "M2", "M3")))

# create list and include all variables (elements)
lst <- list(a, b, c, d, v, M, A)
lst
```

```
[[1]]
[1] 10
```

```
[[2]]
[1] 2
```

```
[[3]]
[1] TRUE
```

```
[[4]]
[1] "word"
```

```
[[5]]
  i1 i2 i3 i4 i5 i6 i7 i8 i9 i10
  1  2  3  4  5  6  7  8  9  10
```

```
[[6]]
      c1 c2
r1 10 30
r2 20 40
```

```
[[7]]
, , M1
```

```
      c1 c2
r1  1  3
r2  2  4
```

```
, , M2
```

```
      c1 c2
r1  5  7
r2  6  8
```

```
str(lst) # check list structure
```

```
List of 7
 $ : num 10
 $ : int 2
 $ : logi TRUE
 $ : chr "word"
 $ : Named int [1:10] 1 2 3 4 5 6 7 8 9 10
 ..- attr(*, "names")= chr [1:10] "i1" "i2" "i3" "i4" ...
 $ : num [1:2, 1:2] 10 20 30 40
 ..- attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:2] "r1" "r2"
 .. ..$ : chr [1:2] "c1" "c2"
 $ : int [1:2, 1:2, 1:2] 1 2 3 4 5 6 7 8
 ..- attr(*, "dimnames")=List of 3
 .. ..$ : chr [1:2] "r1" "r2"
```

```
.. ..$ : chr [1:2] "c1" "c2"  
.. ..$ : chr [1:2] "M1" "M2"
```

```
typeof(lst) # check type
```

```
[1] "list"
```

```
class(lst) # check class
```

```
[1] "list"
```

```
is.list(lst) # check if object is list
```

```
[1] TRUE
```

```
# name each list member  
names(lst) <- c("a", "b", "c", "d", "v", "M", "A")  
lst
```

```
$a  
[1] 10
```

```
$b  
[1] 2
```

```
$c  
[1] TRUE
```

```
$d  
[1] "word"
```

```
$v  
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10  
  1   2   3   4   5   6   7   8   9  10
```

```
$M  
   c1 c2  
r1 10 30  
r2 20 40
```



```
$A  
, , M1
```

```
      c1 c2  
r1  1  3  
r2  2  4
```

```
, , M2
```

```
      c1 c2  
r1  5  7  
r2  6  8
```

```
# alternative: define names as tags when list is created  
list(a=a, b=b, c=c, d=d, v=v, M=M, A=A)
```

```
$a  
[1] 10
```

```
$b  
[1] 2
```

```
$c  
[1] TRUE
```

```
$d  
[1] "word"
```

```
$v  
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10  
   1   2   3   4   5   6   7   8   9  10
```

```
$M  
      c1 c2  
r1 10 30  
r2 20 40
```

```
$A  
, , M1
```

```
      c1 c2
```

```
r1  1  3
r2  2  4
```

```
, , M2
```

```
      c1 c2
r1    5  7
r2    6  8
```

```
# Access list elements
```

```
# single square bracket [] (return a list)
```

```
lst1 <-lst[1] # access first list elements (return a list)
```

```
str(lst1)
```

```
List of 1
```

```
$ a: num 10
```

```
class(lst1)
```

```
[1] "list"
```

```
lst123 <-lst[c(1,2,3)] # access first three elements with index vector (return a list)
lst123
```

```
$a
```

```
[1] 10
```

```
$b
```

```
[1] 2
```

```
$c
```

```
[1] TRUE
```

```
class(lst123)
```

```
[1] "list"
```

```
# double square brackets [[]] (return original member)
ele <-lst[[5]] # extract 5th member-element (returns original element)
ele
```

```
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
    1   2   3   4   5   6   7   8   9  10
```

```
is.vector(ele)
```

```
[1] TRUE
```

```
# use $ operator - extract by member name (return original member)
ele <- lst$M
ele
```

```
      c1 c2
r1 10 30
r2 20 40
```

```
class(ele)
```

```
[1] "matrix" "array"
```

```
# Modify list

# remove element from a list
lst
```

```
$a
[1] 10
```

```
$b
[1] 2
```

```
$c
[1] TRUE
```

```
$d
[1] "word"
```

```
$v
  i1 i2 i3 i4 i5 i6 i7 i8 i9 i10
  1  2  3  4  5  6  7  8  9  10
```

```
$M
    c1 c2
r1 10 30
r2 20 40
```

```
$A
, , M1
```

```
    c1 c2
r1  1  3
r2  2  4
```

```
, , M2
```

```
    c1 c2
r1  5  7
r2  6  8
```

```
lst[1] <- NULL # remove first member
lst
```

```
$b
[1] 2
```

```
$c
[1] TRUE
```

```
$d
[1] "word"
```

```
$v
  i1 i2 i3 i4 i5 i6 i7 i8 i9 i10
  1  2  3  4  5  6  7  8  9  10
```

```
$M
    c1 c2
r1 10 30
```

```
r2 20 40
```

```
$A
```

```
, , M1
```

```
      c1 c2  
r1    1  3  
r2    2  4
```

```
, , M2
```

```
      c1 c2  
r1    5  7  
r2    6  8
```

```
# add element to a list (at the end)  
length(lst)
```

```
[1] 6
```

```
lst[7] <- 1000  
lst
```

```
$b
```

```
[1] 2
```

```
$c
```

```
[1] TRUE
```

```
$d
```

```
[1] "word"
```

```
$v
```

```
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10  
  1   2   3   4   5   6   7   8   9  10
```

```
$M
```

```
      c1 c2  
r1    10 30  
r2    20 40
```

```
$A  
, , M1
```

```
      c1 c2  
r1  1  3  
r2  2  4
```

```
, , M2
```

```
      c1 c2  
r1  5  7  
r2  6  8
```

```
[[7]]  
[1] 1000
```

```
# update value of a member in alist  
lst[[7]] <- 500  
lst[7]
```

```
[[1]]  
[1] 500
```

```
# update value within a vector (on a list)  
lst[[4]][5] <- 5000  
lst[[4]]
```

```
  i1  i2  i3  i4  i5  i6  i7  i8  i9  i10  
   1   2   3   4 5000   6   7   8   9  10
```

```
# convert list to a vector  
vec <- unlist(lst)  
vec
```

```
      b      c      d  v.i1  v.i2  v.i3  v.i4  v.i5  v.i6  v.i7  v.i8  
"2" "TRUE" "word"  "1"   "2"   "3"   "4" "5000"  "6"   "7"   "8"  
v.i9 v.i10      M1      M2      M3      M4      A1      A2      A3      A4      A5  
"9"  "10"  "10"  "20"  "30"  "40"  "1"   "2"   "3"   "4"   "5"  
A6    A7    A8  
"6"   "7"   "8" "500"
```

```
is.vector(vec)
```

```
[1] TRUE
```

```
# Merging lists & nested lists

# create another list
lst1 <- list(e11 = c(1,5,10), e12 = TRUE)

# merge both lists
lst_merged <- c(lst, lst1)
lst_merged
```

```
$b
```

```
[1] 2
```

```
$c
```

```
[1] TRUE
```

```
$d
```

```
[1] "word"
```

```
$v
```

i1	i2	i3	i4	i5	i6	i7	i8	i9	i10
1	2	3	4	5000	6	7	8	9	10

```
$M
```

	c1	c2
r1	10	30
r2	20	40

```
$A
```

```
, , M1
```

	c1	c2
r1	1	3
r2	2	4

```
, , M2
```

	c1	c2
--	----	----

```
r1 5 7
r2 6 8
```

```
[[7]]
[1] 500
```

```
$el1
[1] 1 5 10
```

```
$el2
[1] TRUE
```

```
str(lst_merged)
```

```
List of 9
 $ b : int 2
 $ c : logi TRUE
 $ d : chr "word"
 $ v : Named num [1:10] 1 2 3 4 5000 6 7 8 9 10
 ..- attr(*, "names")= chr [1:10] "i1" "i2" "i3" "i4" ...
 $ M : num [1:2, 1:2] 10 20 30 40
 ..- attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:2] "r1" "r2"
 .. ..$ : chr [1:2] "c1" "c2"
 $ A : int [1:2, 1:2, 1:2] 1 2 3 4 5 6 7 8
 ..- attr(*, "dimnames")=List of 3
 .. ..$ : chr [1:2] "r1" "r2"
 .. ..$ : chr [1:2] "c1" "c2"
 .. ..$ : chr [1:2] "M1" "M2"
 $ : num 500
 $ el1: num [1:3] 1 5 10
 $ el2: logi TRUE
```

```
names(lst_merged)
```

```
[1] "b" "c" "d" "v" "M" "A" "" "el1" "el2"
```

```
# nested list (recursive procedure)
list3 <- list(1, c(T,F,F)) # list sub-level 3
```



```
list2 <- list(list3) # list sub-level 2
list1 <- list(list2) # list sub-level 1

str(list1)
```

```
List of 1
 $ :List of 1
  ..$ :List of 2
   .. ..$ : num 1
   .. ..$ : logi [1:3] TRUE FALSE FALSE
```

```
# extract list level 2
list1[[1]]
```

```
[[1]]
[[1]][[1]]
[1] 1
```

```
[[1]][[2]]
[1] TRUE FALSE FALSE
```

```
# extract list level 3
list1[[1]][[1]]
```

```
[[1]]
[1] 1
```

```
[[2]]
[1] TRUE FALSE FALSE
```

```
# extract 1st member from list level 3
list1[[1]][[1]][[1]]
```

```
[1] 1
```

```
# extract 2nd member from list level 3
list1[[1]][[1]][[2]]
```

```
[1] TRUE FALSE FALSE
```

17 Matrices

A matrix is a two-dimensional structure that contains elements of the same type (numeric, character, or logical).

```
# Creating a 3x3 numeric matrix
my_matrix <- matrix(1:9, nrow = 3, ncol = 3)

# Accessing elements
my_matrix[1, 2] # Access the element in row 1, column 2
```

```
[1] 4
```

```
# using matrix()
M <- matrix(data = 1:9, nrow = 3, ncol = 3)
M
```

```
      [,1] [,2] [,3]
[1,]     1     4     7
[2,]     2     5     8
[3,]     3     6     9
```

```
M <- matrix(data = 1:9, nrow = 3, ncol = 3, byrow = T)
M
```

```
      [,1] [,2] [,3]
[1,]     1     2     3
[2,]     4     5     6
[3,]     7     8     9
```

```
matrix(data = 1:6, nrow = 2, ncol = 3)
```

```
      [,1] [,2] [,3]
[1,]     1     3     5
[2,]     2     4     6
```

```
# by merging multiple vectors
v1 <- c(1,2,3)
v2 <- c(4,5,6)
v3 <- c(7,8,9)

rbind(v1,v2,v3)
```

```
      [,1] [,2] [,3]
v1      1    2    3
v2      4    5    6
v3      7    8    9
```

```
cbind(v1,v2,v3)
```

```
      v1 v2 v3
[1,]  1  4  7
[2,]  2  5  8
[3,]  3  6  9
```

```
# by altering vector dimension
v <- 1:9
v
```

```
[1] 1 2 3 4 5 6 7 8 9
```

```
dim(v) <- c(3,3)
v
```

```
      [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
# Matrix properties
```

```
# rownames & colnames
```

```
M <- matrix(1:12, nrow = 4, dimnames = list(c("r1","r2","r3", "r4"), c("c1","c2","c3")))
M
```

```

      c1 c2 c3
r1  1  5  9
r2  2  6 10
r3  3  7 11
r4  4  8 12

```

```
rownames(M)
```

```
[1] "r1" "r2" "r3" "r4"
```

```
colnames(M)
```

```
[1] "c1" "c2" "c3"
```

```
# matrix dimension
dim(M)
```

```
[1] 4 3
```

```
# get all attributes
attributes(M)
```

```
$dim
[1] 4 3
```

```
$dimnames
$dimnames[[1]]
[1] "r1" "r2" "r3" "r4"
```

```
$dimnames[[2]]
[1] "c1" "c2" "c3"
```

```
# change rownames & colnames
rownames(M) <- paste("row ", 1:4, sep = "")
colnames(M) <- paste("col ", 1:3, sep = "")
attributes(M)
```

```

$dim
[1] 4 3

$dimnames
$dimnames[[1]]
[1] "row 1" "row 2" "row 3" "row 4"

$dimnames[[2]]
[1] "col 1" "col 2" "col 3"

```

```
M
```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```

# class and type
class(M)

```

```
[1] "matrix" "array"
```

```
typeof(M)
```

```
[1] "integer"
```

```

# check for matrix
is.matrix(M)

```

```
[1] TRUE
```

```

# Access matrix elements

# integer vector as index
M

```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```
M[2,3]
```

```
[1] 10
```

```
M[c(1,2),3]
```

row 1	row 2
9	10

```
M[c(2,3),] # selected rows and all columns
```

	col 1	col 2	col 3
row 2	2	6	10
row 3	3	7	11

```
M[,c(2,3)] # selected columns and all rows
```

	col 2	col 3
row 1	5	9
row 2	6	10
row 3	7	11
row 4	8	12

```
# logical vector as index
M[c(T,T,F,F), c(T,T,T)]
```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10

```
# character vector as index
M[c("row 2", "row 3"), c("col 1", "col 2")]
```

	col 1	col 2
row 2	2	6
row 3	3	7

```
# range of indexes (slicing rows and columns)
M[1:3,2:3]
```

	col 2	col 3
row 1	5	9
row 2	6	10
row 3	7	11

```
# Access matrix elements

# modify 1 element
M
```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```
M[1,1] <- 10
M
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```
# modify more than one element
M[2:3,3] <- 20
M
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	20
row 3	3	7	20
row 4	4	8	12

```
# modify elements based on condition
M[M>10] <- 0
M
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0

```
# transpose a matrix
t(M)
```

	row 1	row 2	row 3	row 4
col 1	10	2	3	4
col 2	5	6	7	8
col 3	9	0	0	0

```
# add row to matrix
M
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0

```
rbind(M, c(0,0,0))
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0
	0	0	0


```
# add column to matrix
cbind(M, c(0,0,0, 0))
```

```
      col 1 col 2 col 3
row 1    10     5     9 0
row 2     2     6     0 0
row 3     3     7     0 0
row 4     4     8     0 0
```

```
# alter matrix dimensions
dim(M)
```

```
[1] 4 3
```

```
dim(M) <- c(3,4) # names are dropped
M
```

```
      [,1] [,2] [,3] [,4]
[1,]    10     4     7     0
[2,]     2     5     8     0
[3,]     3     6     9     0
```

```
# merge 2 matrices
M1 <- matrix(data = rep(0,4), nrow = 2, ncol = 2)
M2 <- matrix(data = rep(1,4), nrow = 2, ncol = 2)
M1
```

```
      [,1] [,2]
[1,]     0     0
[2,]     0     0
```

```
M2
```

```
      [,1] [,2]
[1,]     1     1
[2,]     1     1
```

```
rbind(M1,M2)
```

```
      [,1] [,2]  
[1,]    0    0  
[2,]    0    0  
[3,]    1    1  
[4,]    1    1
```

```
cbind(M1,M2)
```

```
      [,1] [,2] [,3] [,4]  
[1,]    0    0    1    1  
[2,]    0    0    1    1
```

```
# Matrix arithmetics
```

```
# matrix - scalar (scalar with each vector element)  
M
```

```
      [,1] [,2] [,3] [,4]  
[1,]   10    4    7    0  
[2,]    2    5    8    0  
[3,]    3    6    9    0
```

```
a <- 10
```

```
# Addition +  
M + a
```

```
      [,1] [,2] [,3] [,4]  
[1,]   20   14   17   10  
[2,]   12   15   18   10  
[3,]   13   16   19   10
```

```
# Subtraction -  
M - a
```

```
      [,1] [,2] [,3] [,4]  
[1,]    0   -6   -3  -10  
[2,]   -8   -5   -2  -10  
[3,]   -7   -4   -1  -10
```

```
# Multiplication *
```

```
M * a
```

```
      [,1] [,2] [,3] [,4]
[1,]  100   40   70    0
[2,]   20   50   80    0
[3,]   30   60   90    0
```

```
# Division /
```

```
M / a
```

```
      [,1] [,2] [,3] [,4]
[1,]   1.0  0.4  0.7    0
[2,]   0.2  0.5  0.8    0
[3,]   0.3  0.6  0.9    0
```

```
# Exponent ^ **
```

```
M^a
```

```
      [,1]      [,2]      [,3] [,4]
[1,] 1.0000e+10  1048576  282475249    0
[2,] 1.0240e+03   9765625 1073741824    0
[3,] 5.9049e+04  60466176 3486784401    0
```

```
# Modulus (Remainder from division) %%
```

```
M %% 2
```

```
      [,1] [,2] [,3] [,4]
[1,]    0    0    1    0
[2,]    0    1    0    0
[3,]    1    0    1    0
```

```
# Integer Division %/%
```

```
M %/% 2
```

```
      [,1] [,2] [,3] [,4]
[1,]     5     2     3     0
[2,]     1     2     4     0
[3,]     1     3     4     0
```

```
# Other functions on matrix elements
sqrt(M)
```

```
      [,1]      [,2]      [,3] [,4]
[1,] 3.162278 2.000000 2.645751    0
[2,] 1.414214 2.236068 2.828427    0
[3,] 1.732051 2.449490 3.000000    0
```

```
log(M)
```

```
      [,1]      [,2]      [,3] [,4]
[1,] 2.3025851 1.386294 1.945910 -Inf
[2,] 0.6931472 1.609438 2.079442 -Inf
[3,] 1.0986123 1.791759 2.197225 -Inf
```

```
sum(M)
```

```
[1] 54
```

```
# matrix - vector (matrix element to element | member-by-member)
M1 <- matrix(data = 1:9, nrow = 3, byrow = T)
M2 <- matrix(data = rep(3,9), nrow = 3)

# Addition +
M1 + M2
```

```
      [,1] [,2] [,3]
[1,]    4    5    6
[2,]    7    8    9
[3,]   10   11   12
```

```
# Subtraction -
M1 - M2
```

```
      [,1] [,2] [,3]
[1,]   -2   -1    0
[2,]    1    2    3
[3,]    4    5    6
```

```
# Multiplication *
M1 * M2
```

```
      [,1] [,2] [,3]
[1,]    3    6    9
[2,]   12   15   18
[3,]   21   24   27
```

```
# Division /
M1 / M2
```

```
      [,1]      [,2] [,3]
[1,] 0.3333333 0.6666667 1
[2,] 1.3333333 1.6666667 2
[3,] 2.3333333 2.6666667 3
```

```
# Exponent ^ **
M1^M2
```

```
      [,1] [,2] [,3]
[1,]    1    8   27
[2,]   64  125  216
[3,]  343  512  729
```

```
# Modulus (Remainder from division) %%
M1 %% M2
```

```
      [,1] [,2] [,3]
[1,]    1    2    0
[2,]    1    2    0
[3,]    1    2    0
```

```
# Integer Division %/%
M1 %/% M2
```

```
      [,1] [,2] [,3]
[1,]    0    0    1
[2,]    1    1    2
[3,]    2    2    3
```

```
# matrix-matrix style multiplication
M1
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
M2
```

```
      [,1] [,2] [,3]
[1,]    3    3    3
[2,]    3    3    3
[3,]    3    3    3
```

```
t(M1) %*% M2
```

```
      [,1] [,2] [,3]
[1,]   36   36   36
[2,]   45   45   45
[3,]   54   54   54
```

```
M1 %*% M2
```

```
      [,1] [,2] [,3]
[1,]   18   18   18
[2,]   45   45   45
[3,]   72   72   72
```

```
# matrix algebra (matrix based functions)
M <- matrix(data = c(1,5,3,2,4,7,4,6,2), nrow = 3, byrow = T)

# get diagonal elements
diag(M)
```

```
[1] 1 4 2
```

```
# get matrix determinant  
det(M)
```

```
[1] 74
```

```
# get inverse of a matrix M(-1)  
solve(M)
```

```
      [,1]      [,2]      [,3]  
[1,] -0.45945946  0.1081081  0.31081081  
[2,]  0.32432432 -0.1351351 -0.01351351  
[3,] -0.05405405  0.1891892 -0.08108108
```

```
# get eigen values  
eigen(M)
```

```
eigen() decomposition
```

```
$values
```

```
[1] 11.778446+0.0000000i -2.389223+0.7578106i -2.389223-0.7578106i
```

```
$vectors
```

```
      [,1]      [,2]      [,3]  
[1,] 0.4687233+0i  0.5211486+0.2411697i  0.5211486-0.2411697i  
[2,] 0.6544420+0i -0.6642393+0.0000000i -0.6642393+0.0000000i  
[3,] 0.5932993+0i  0.4573822-0.1408153i  0.4573822+0.1408153i
```

```
# calculate sum over rows or columns  
M
```

```
      [,1] [,2] [,3]  
[1,]    1    5    3  
[2,]    2    4    7  
[3,]    4    6    2
```

```
rowSums(M)
```

```
[1]  9 13 12
```

```
colSums(M)
```

```
[1] 7 15 12
```

```
# Lets solve simple matrix equation
# A * X = B
A <- matrix(data = c(1,2,4,5), nrow = 2, byrow = T)
B <- matrix(data = c(5,24,17,66), nrow = 2, byrow = T)
# X = A(-1) * B
X <- solve(A) %*% B
X
```

```
      [,1] [,2]
[1,]     3     4
[2,]     1    10
```

```
# test
A %*% X # should get B
```

```
      [,1] [,2]
[1,]     5    24
[2,]    17    66
```

```
# summarizing a matrix (apply)
M
```

```
      [,1] [,2] [,3]
[1,]     1     5     3
[2,]     2     4     7
[3,]     4     6     2
```

```
# sum of rows
apply(X = M, MARGIN = 1, FUN = sum)
```

```
[1] 9 13 12
```

```
# sum of columns
apply(X = M, MARGIN = 2, FUN = sum)
```


[1] 7 15 12

```
# create matrix of random numbers  
rnorm(n = 1000, mean = 0, sd = 2)
```

```
[1] -0.6174908869 -1.9245729845 -1.7953024035 -1.0834748890 2.5427921200  
[6] -1.1606479726 -1.4290499314 1.6053907155 -1.8720614346 2.0336997468  
[11] -0.0044806265 -2.3477397903 -0.9264351636 -1.4321100751 -3.8610593189  
[16] -2.1034295891 0.0522743464 3.3603824608 -0.3039148946 1.1516899507  
[21] 0.8522042644 -0.1164075883 2.9313165271 -1.3155855818 -0.4828045728  
[26] 1.3492463250 -0.0236065959 -1.2451639059 0.9498498965 -3.6143154372  
[31] 0.8371597088 1.5216540085 -2.3741256477 -1.8398008843 1.6083626206  
[36] -0.4180629952 5.1280217436 -1.7378187167 -2.3598288852 -1.2454257256  
[41] -0.9568453068 1.3685418398 -3.2147918520 0.2180282728 0.1792053054  
[46] -1.1271994250 -0.1223126269 -0.9657842877 0.3216588735 0.4156337878  
[51] -0.6083702908 0.6873022447 5.5773086899 -1.8549440494 -1.3160179043  
[56] 2.7581879565 -1.6735543642 -2.6417882413 0.8952257029 0.8858918060  
[61] -2.2161957499 0.0085767601 -1.9391183333 -4.8296753116 -3.4292606181  
[66] -0.1873194797 -0.4706321486 -1.1040792050 -1.6041261008 -0.0742574440  
[71] -0.1580086097 0.0713682758 0.3142231347 -0.6133971031 1.4302837539  
[76] 1.4541515431 -4.3750801618 4.4828480168 -0.5336607368 -0.6461368836  
[81] -0.8950245793 -1.4288027216 1.5131914940 1.7513122659 0.8489445896  
[86] 1.6629921384 1.6538421823 2.4451206624 1.0109452471 1.7215817543  
[91] 2.3028904758 -0.3746555136 -4.6978854262 -1.7291111967 2.4545090120  
[96] 7.0865341428 1.4058605432 -1.1332244130 -0.0271864796 3.6067930297  
[101] 1.4493118305 -0.3804340188 3.9355473194 0.3360348352 -1.9118498603  
[106] 0.3069602730 0.5101206458 0.0030980004 2.8562948301 -1.0912755253  
[111] 0.2044975706 1.6001715029 0.4348643104 -2.0574284845 -1.6708441955  
[116] 2.4766877196 -0.2540272658 -0.2953978586 -0.5046052631 -0.6491227863  
[121] -0.9585219219 2.0769893589 -1.2324543276 3.3939227089 -0.5395286340  
[126] 1.0333879344 -1.5817050188 -1.6867900996 0.0799808985 -0.9046137697  
[131] 2.0849187801 0.6976396265 -2.3072398699 -2.7463826122 1.0210663841  
[136] -3.2093806466 3.0271792204 -1.1269221838 -2.6758075799 -1.3686108128  
[141] -2.7934872838 3.4830587881 3.1593909582 -3.2441908285 3.0509704011  
[146] 0.1360753422 1.5925341181 2.6739338806 2.7688051287 -2.7669275389  
[151] -1.5635240099 -0.6991363102 0.1160880048 -0.7185285298 -1.2030420723  
[156] -1.5984300034 1.5773107877 2.4369967340 0.3869644738 2.3330759337  
[161] 1.6207437584 -0.8559991722 -1.0266677177 3.4818354585 1.8042407437  
[166] -0.3292044538 -1.9679246147 1.7392041757 1.8544388997 1.5054879259  
[171] 1.9986740572 -3.3227887711 2.3183943331 -0.8633494268 0.9530744988  
[176] 0.2967506652 1.1121641799 -1.6150629136 2.1298639248 -4.0585894662  
[181] 0.1081904099 -1.2547985383 0.8705070088 -1.1599467933 1.1398756020
```

[186]	2.1066150560	-0.6025358526	0.6557839275	1.0838579365	-0.6145014650
[191]	-0.5192667365	1.4020654089	-1.3188992913	1.1391159413	-3.0584106960
[196]	-3.5629442535	-0.7054631573	1.4982057390	0.6756961939	4.3091686069
[201]	-0.6895301609	0.1933679843	3.1382761881	4.1363758212	0.3093578360
[206]	-3.2069739414	0.1190598925	-1.8371366908	-4.8166502721	-1.6219458246
[211]	0.9993420633	1.5158352996	-2.5533937069	3.7723623716	-4.2906067784
[216]	-1.1355072831	-0.2148654800	0.4464658914	1.0053097508	1.7840427407
[221]	0.1351012543	-0.8764326999	-2.2030969728	-3.3772497222	-2.6891540682
[226]	2.5183115272	-0.8560853866	0.8159022052	-3.0509240704	1.6733631309
[231]	-0.2719862906	1.4128955796	0.9115935134	-1.3969702020	-5.5421888214
[236]	2.8095016551	0.2244721761	-3.3074895754	-0.1044854987	1.4547910714
[241]	-1.7562908556	-0.0404880986	3.1469962118	-0.7183927449	-1.7503125602
[246]	1.4026889791	-0.7271000481	-1.2480054749	-1.3298397486	-1.0045516096
[251]	2.5688419379	0.6656390518	-1.1614393295	-3.0536892345	-1.0454090167
[256]	-2.8233627362	0.0267549858	3.1192217748	-0.0350548594	-0.4358090942
[261]	-0.2556218554	-2.6572230586	-0.0853818629	-1.8342058135	0.5068588889
[266]	0.2156229891	-0.0697209305	-0.5656406519	-0.2444712370	-4.1934410892
[271]	-3.3634764500	-0.3269055780	1.8380131005	4.0394387327	1.1805367591
[276]	-3.1963035536	-1.2662748078	-4.0431522997	-2.1011068919	0.8467087026
[281]	-1.4583559805	-0.5442136785	-3.2757928707	1.4931506238	-0.4517436884
[286]	2.2326682138	-2.3807348119	3.2598698350	2.4094667669	4.2505241546
[291]	-0.9756938119	1.4841598091	-2.4369161351	-0.0305152537	2.4708656975
[296]	4.4748114144	1.2407205039	0.2964195284	-1.5703540846	-2.5810005598
[301]	-2.7320790164	1.6460785310	1.8228556424	-3.8647694248	0.6358003756
[306]	-2.8443687252	-0.5889388901	-0.2747155621	-0.1683919141	0.7033376330
[311]	-0.1975833680	1.9269308618	5.9971008213	-2.6564019984	-0.6465086152
[316]	2.8891102813	3.2784302696	-1.0133202590	-3.3390473447	1.2373405509
[321]	1.6431232968	-0.6758729773	-2.1099317179	1.9729728922	-1.9329571672
[326]	2.3295327585	0.3131526278	1.5403132591	0.3038647806	0.1877602294
[331]	-1.8965187389	-1.0521044103	0.9696728770	-1.4540404132	0.8951866928
[336]	0.9424071901	0.3387561305	-3.4259307146	3.7853458496	-0.1627561239
[341]	0.0176366945	-1.1320636158	1.2621140537	0.1494396016	0.4427482129
[346]	0.8157163713	1.3958937262	-0.3395691428	0.8535962526	1.9926291607
[351]	5.1715221439	1.5900245794	-0.0592492612	-3.4664869713	-0.4764450771
[356]	-1.9557788216	0.3138474101	2.2261326215	-2.6060263295	0.6960641059
[361]	-0.5934976975	1.0198518470	-0.4465981468	1.0370676880	2.3458203743
[366]	1.4065297940	4.0379579855	0.3079739108	4.9322906177	-0.5210716081
[371]	-1.3242036584	0.4679767378	-1.7699802092	-0.8701540321	1.2184656834
[376]	-3.5779833778	0.2329739961	-1.8839066322	0.0393630676	-1.0505125552
[381]	-1.2549763001	-1.9372554076	0.0247927530	2.2007164762	-1.3216586294
[386]	0.2639109828	-4.2069144542	1.8090184290	0.6983318936	0.7250995314
[391]	-2.4264539797	-0.6623230532	-1.3628385252	-1.1784541827	0.4615671583
[396]	1.4550459723	-1.8404461393	-4.5372525512	1.9581737752	-2.2632652016

[401]	-1.3210686891	-0.7933995771	0.9006813639	-3.5885765722	-3.2483977268
[406]	0.5539694309	-3.9981693659	-0.9843512313	-1.1251800538	2.0514395175
[411]	-0.4354572574	3.3246476192	0.9994913504	-0.9464742546	1.6039478750
[416]	-1.5872257379	-1.5138179303	1.7160996894	1.6080099466	1.9646619348
[421]	0.6595166058	-0.2022627612	-0.0586076106	0.7903564650	0.6706802389
[426]	1.8498340714	-1.7390441426	-2.9311148573	0.2571173439	2.3432904325
[431]	-1.5698699894	-1.8969819488	1.0252628212	0.4572694846	1.6425958147
[436]	-0.4438669436	1.8363984186	0.7453070142	-1.9993172677	0.3484063374
[441]	0.2464928385	1.0844123181	0.5533643608	-0.0054467983	1.7424178115
[446]	1.0863403103	2.2001334296	1.3362176761	2.5700598629	-4.3148803450
[451]	-2.3855795531	3.6234752494	0.9306634535	0.3183036361	1.1164598268
[456]	1.4488221395	0.4528822388	0.1072187238	-1.7721783884	0.3067672700
[461]	2.8166443820	-4.3422507871	1.2369775227	0.2263173205	-0.0249615538
[466]	1.0297936992	-1.1123505732	1.3460287864	0.7311487277	-3.4884292109
[471]	2.1270460130	1.0277594005	-0.7908834595	1.5099788505	0.9710749441
[476]	1.3435386267	-1.5946214763	-0.0748698452	-1.8254573149	1.7468655069
[481]	1.0482414895	-0.3662130864	-0.4216688612	1.9264034418	1.6828553217
[486]	2.6613249765	-1.4696790652	2.0357545785	-2.1979638822	-1.7818097227
[491]	-0.1371003554	0.0008407917	2.6567178630	2.6794034435	0.8009146988
[496]	1.2786031638	-2.1691108559	-0.7100418725	0.2874487892	-0.6891411732
[501]	-1.2678667312	-0.6116855022	-3.0383315120	-1.4832649332	-1.7019381489
[506]	-0.3488299487	2.2537785840	-0.1598954584	-3.4370023350	3.5312653397
[511]	-0.7007797605	-2.1637177252	1.0259014116	-2.8200100507	0.8739739905
[516]	-0.6195180848	1.3098165277	2.5934038441	1.8434649839	-0.9838419237
[521]	3.5044976494	-0.2418319278	1.7268796118	-2.5955010784	-0.9596440520
[526]	1.3318570542	-1.8635793462	3.0151016979	4.6822900453	-2.2472397707
[531]	-3.9129922850	-6.0634606173	2.8881265801	1.0900998561	-4.7503526736
[536]	-4.5788665233	-1.4672445123	0.7960307000	-1.1463099735	-0.6897865523
[541]	2.1046027440	1.2557863197	-3.7680904696	-1.5999781579	-3.3849742241
[546]	3.8754832854	1.0702399765	1.0927544769	0.5588258791	0.3165526671
[551]	0.8161303088	2.5196347262	-1.8053102284	1.5016443951	2.7663894227
[556]	-2.6473489440	3.2120404303	-0.9382646350	5.8087294328	1.9813186807
[561]	3.7185724319	-1.5246130832	-0.3517723753	1.2225842306	1.0611744284
[566]	-2.4820422307	-1.1985068290	-2.6219457106	-1.0003875042	0.4136665644
[571]	0.1075792003	-2.4289489200	-1.9786983378	1.1002187642	1.5010695025
[576]	-0.2282757445	1.0180136699	-1.8562778342	-0.7191274886	-1.1437338570
[581]	-1.1182416835	0.8934692548	-3.1329980804	-2.4660718035	2.4982217132
[586]	1.3106282660	1.6842865444	-1.7540406421	-0.9511221148	2.4187658453
[591]	1.5410733684	-1.1375743220	-0.5633180131	1.5922154077	-0.1543266220
[596]	-3.3941152470	0.4620155227	0.1903793300	-0.2501084228	-4.4420026203
[601]	0.4262449029	-1.1843858660	-2.5676865993	0.6629990484	-4.4080066402
[606]	-0.5080424002	5.8025300827	-2.3889312768	1.4021044686	1.3336885312
[611]	2.2791879054	-1.2569885565	-1.9466288149	0.0060290376	-1.4061513888

[616]	-1.8953859222	1.1119853366	0.2358804127	2.9888513881	1.8100342055
[621]	-1.6338586207	-0.9098208639	-0.1916733572	-0.1925494972	4.3063434384
[626]	2.7677246701	-2.5667738841	0.5995981627	0.3531470547	-0.0527249844
[631]	0.2102088554	1.0071960366	4.8050971545	1.0196927649	1.7381837442
[636]	0.5386814485	0.2826370834	-1.5444918250	1.7248817134	-2.7349425801
[641]	-2.2146312791	-2.4607519833	-2.6889987727	-2.4285155414	0.8035439203
[646]	1.2630315155	-1.6669746594	1.4275321646	0.1002070103	3.1177547469
[651]	-2.5716341395	1.0784442950	0.8245680943	1.2580121517	-1.8225476090
[656]	-2.7500650408	-0.9009915354	1.0692702050	3.6234406907	0.5868006223
[661]	0.2855778224	0.2625301852	1.0526266395	-2.4430320476	-0.5199035159
[666]	2.1465085064	-1.5530468308	-4.3177573051	-3.3126992848	-0.2651213591
[671]	-0.3898749750	-4.0196924454	-0.8964824935	0.3775520675	-1.7604548341
[676]	1.7572550759	0.1835729481	2.3445277102	0.3635353031	-2.2734017626
[681]	-0.5288799701	1.0882440589	0.3139932992	1.2358111914	0.1415968230
[686]	-1.7374290705	-0.6923446257	-0.6302271623	-1.1507549856	0.1963143790
[691]	1.0830410271	3.8640087299	-0.3150699626	3.1554598687	-0.4590480897
[696]	2.6646789980	-0.8148422454	-2.1457255159	3.2111021201	1.3948061472
[701]	0.0247377849	-2.1876419647	-2.9695588526	0.5269760829	0.4669337300
[706]	0.9486775249	-0.1192341239	0.5285439800	-0.6142596136	4.3188766434
[711]	1.4085253697	-0.0566566679	1.9672358330	-1.1466559916	2.0966202231
[716]	0.6365955591	-1.3135760814	-0.8656543046	0.8564850267	-2.1367701345
[721]	-0.8697928615	2.8539469236	-3.2728000468	1.3135044193	3.8645011786
[726]	0.3456300934	0.4908789347	4.6944903009	-1.5295050471	2.0544727384
[731]	-0.1313837501	1.2886529710	-1.3318138441	-0.7341149108	3.0400388660
[736]	-3.4458276203	-5.7256551610	-0.8926177321	-3.1393995498	-0.6594990880
[741]	0.6250388253	-4.0692088179	-2.5278975436	0.9475569251	1.5493262934
[746]	0.9399071623	0.9532974835	-0.9201926374	0.6021782159	-0.4716460438
[751]	-2.8894120732	-0.8043159817	1.3699457273	-2.3407199997	-1.7082933853
[756]	1.6348579574	3.2573875533	0.0645365945	-2.8884079920	-0.5868774348
[761]	1.4751093292	-0.8831728807	-0.8271595689	1.4583709405	-2.9008447220
[766]	-2.2108259850	2.2905169824	1.4527297469	-0.1596311644	0.4057160993
[771]	-2.0005835253	2.0548933283	0.9506088625	-3.7229651608	-0.5961509816
[776]	-2.9069780243	1.1827472332	1.2865406804	-0.1291540342	-1.0810631242
[781]	-1.7436410782	-0.1567022311	1.5668380110	-0.1689132217	-1.0081857635
[786]	-3.1750528718	0.2043271410	0.5788397038	2.7844814879	0.5749351732
[791]	0.9285569342	-0.9883183016	-1.0615878823	-1.9535188357	-2.5715885395
[796]	-0.3434870037	-2.8786128954	0.5064840450	-0.2841184963	0.5957341998
[801]	-3.0288186316	1.5041063816	-1.8820292443	0.6245138994	4.5826552756
[806]	0.7058643013	2.5017745227	-0.8015437128	-2.2740853440	-2.4974602885
[811]	5.0456153505	0.2244348574	4.7889846898	-0.8939733181	-2.1471071798
[816]	0.4212250865	-1.1730617754	0.4046824418	-0.0680835205	-2.5832331637
[821]	0.2715743377	-1.3213616252	2.0582330477	-1.2274770768	0.8461914033
[826]	-2.1880878496	1.0678163324	2.6281958585	-3.9856524440	2.9107206519

```

[831] -0.0446762716 -5.1659799880 -0.1855975681  2.7759257328 -1.4587195421
[836]  3.1763910673 -1.6388728229 -1.5834656228  2.2378988920  0.8256230964
[841] -1.0980610622 -2.6740388436 -1.5276637338 -2.3148014035 -1.6412250304
[846]  0.2687352546  1.3009145746 -1.9047493159 -3.7644281911  1.7647924715
[851] -0.9928808773  0.8917331750 -1.2628032991 -0.5348302921  3.6428450536
[856] -1.5988083080  1.3595317810 -2.1072078404 -1.6402782580 -0.1044240172
[861] -2.7687588867  3.2006464378 -0.6448370645  2.5474245937  1.1262028152
[866] -1.6575148547  3.3373733278 -1.6948063393 -2.4663101092 -1.1335443815
[871] -1.9077316537  3.0832423389  5.0782156331  1.8921025477  0.5228502690
[876] -1.9790392031 -3.5135364761 -3.7073378304 -0.8674240792  0.0032763735
[881]  3.7750091435  2.2420436576  2.2235548703  2.8170542624 -1.7173533518
[886]  1.8064807189  3.2937922266  0.6748320975 -0.1457207589  1.8908067551
[891]  0.8146783750 -0.2376304882 -0.5360171480 -1.8104500910  2.4405203746
[896]  4.9282213472  2.0075867351  1.0796345090  3.3180646479 -1.1287753090
[901]  1.3243454435 -0.1158675484 -1.3274034639  1.3826043672 -1.1435820384
[906]  2.2920150017  3.8742887554 -0.4712391823  0.7829352404 -2.6710219554
[911] -0.5392903295 -5.6108119403  1.0720200332 -0.6662891176 -1.6450521002
[916]  1.5789891379 -1.7858457813  1.1811441364 -0.5494285986 -2.4699838645
[921] -1.0325197218 -0.6374596039  1.0646267246  0.2551477893  2.1718872760
[926] -1.9452613807 -1.7456859728  0.0786430111  2.6373429829 -0.5653627302
[931]  0.1649840513  3.7416301152 -0.4499538932 -2.6306660546  0.7017316898
[936] -3.2883670918 -0.4431954984 -0.7124711906  2.7290713772  0.7161620130
[941]  0.3031920307 -0.8394520378  1.5637194622  0.7637758359 -2.2946718844
[946]  0.3695490402 -1.0641513050  0.4153374659 -0.8830831007 -2.5728789865
[951] -3.0088806680  1.8502425357  5.0341318639 -0.9885966722 -1.0008400191
[956] -1.2540440364 -1.9363101748  0.9718334202 -0.1503460837  2.4641606668
[961] -0.2104200102  1.1948329179  1.9469912074  1.9527528922 -0.5240899554
[966]  0.0271220888 -1.7436611705 -0.6333551111  2.2080394732 -1.3066452369
[971] -0.6317517011 -0.6269698114  0.9589281042  2.9227327693 -0.8635260613
[976] -0.2816123168  1.5640809234  1.9355331661 -2.4450868577  2.0627510316
[981] -2.8608150670  0.6661350115  1.2810001117 -0.9566509588 -1.1431768369
[986] -3.4192406870 -1.3056529295 -0.1442154099  0.6170935491 -0.7857593181
[991] -2.9413612397 -0.0069846903  3.0069675765 -1.7653594668 -0.6612672349
[996]  0.2819830336 -0.7745580218  3.0992815194 -3.8604783575 -1.6301274848

```

```

A <- matrix(data = rnorm(n = 1000, mean = 0, sd = 2), nrow = 100, ncol = 10)

# get mean over columns
apply(A, 2, mean)

```

```

[1] -0.15013126 -0.17426022  0.19004727 -0.15134885 -0.28319166 -0.06439175
[7]  0.05447213 -0.16074462  0.19816753 -0.02842086

```

```
# get mean over rows
apply(A, 1, mean)
```

```
[1] -0.21259465 -0.55117100  0.74781070 -0.06784853  0.96409951 -0.12874498
[7]  0.55581001 -0.64399683  0.08372248  0.50326844  0.53087421 -0.84769501
[13] -0.81763103 -0.65916853  0.08142897 -0.28454965 -0.05728346  0.72081852
[19]  0.08164943 -0.24481556 -0.34895207 -1.30518764 -0.85788570 -1.22334372
[25] -0.60205016  0.10276842  1.08344259  0.22063671 -1.69389169  0.06161454
[31] -0.33528763 -0.96928964  2.61097417  0.52511189 -0.42405825 -0.10585952
[37] -0.15670161  0.59900158 -0.73108147 -0.34945502  0.19514232 -0.67345874
[43] -0.06135065  0.77036131  1.34202848  0.98909786 -1.16130097  0.54735893
[49]  0.19256833 -0.76702206  0.10688194 -0.60714214 -0.36705148  1.14975911
[55] -0.13498664 -0.98034920  0.09213744  0.09471995  0.16547843 -0.11829200
[61] -0.03516347 -0.51682804 -0.35044228  0.11003419 -0.71012677  0.72467809
[67] -0.87835509  1.14046314 -0.10191947 -0.10682503  0.30029995 -1.71862503
[73]  0.57216369  0.25034112  1.38840546  0.37859258 -0.84483401 -0.18157807
[79]  0.35173092 -0.25228634  0.41207175  1.03863439  0.98518553 -1.08609459
[85]  0.68576491  0.18165096  0.91665122  0.38244162  0.35569112 -0.79194125
[91] -0.21403424 -0.18222460  0.24449374  0.31340934 -0.89699495 -0.59198897
[97] -1.20146557 -0.73370667 -0.80440211 -0.85995918
```

```
# calculate standard deviation for each column
apply(A, 2, sd)
```

```
[1] 1.939281 2.165912 2.012185 2.024657 2.013188 2.031942 2.179331 2.011602
[9] 1.824828 1.991854
```

18 Data Frames

A data frame is a table where each column can contain elements of different types (e.g., numbers, strings). It's the most common structure used for data sets.

```
# Creating a data frame
my_data <- data.frame(
  Name = c("Alice", "Bob", "Charlie"),
  Age = c(23, 30, 25),
  Gender = c("F", "M", "M")
)
```

```
my_data
```

	Name	Age	Gender
1	Alice	23	F
2	Bob	30	M
3	Charlie	25	M

```
# Accessing columns
my_data$Name # Output: "Alice", "Bob", "Charlie"
```

```
[1] "Alice" "Bob" "Charlie"
```

```
# create data frame
df1 <- data.frame(col1 = 1:3,
                  col2 = c("a", "b", "c"),
                  col3 = c(T, F, T),
                  col4 = c(as.Date("2020-01-01"), as.Date("2020-01-03"), as.Date("2020-01-03"))

# create data frame - vectors
col1 <- seq(10,100,10)
col2 <- seq(as.Date("2020-01-01"), length = 10, by = "weeks")
col3 <- rep("word", 10)
```

```
df2 <- data.frame(num = col1,
                  date = col2,
                  string = col3)
```

```
# check DF structure
str(df2)
```

```
'data.frame':  10 obs. of  3 variables:
 $ num      : num  10 20 30 40 50 60 70 80 90 100
 $ date     : Date, format: "2020-01-01" "2020-01-08" ...
 $ string   : chr  "word" "word" "word" "word" ...
```

```
# create data frame - matrix
M <- matrix(data = 1:100, nrow = 10, ncol = 10, byrow = T)
rownames(M) <- paste("row", 1:10, sep = "")
colnames(M) <- paste("col", 1:10, sep = "")
M
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20
row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
df3 <- as.data.frame(M)
df3
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20
row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60

row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
# check DF dimensions
dim(df3)
```

```
[1] 10 10
```

```
nrow(df3)
```

```
[1] 10
```

```
ncol(df3)
```

```
[1] 10
```

```
# check DF type / class
class(df3)
```

```
[1] "data.frame"
```

```
typeof(df3)
```

```
[1] "list"
```

```
# Accessing DF
```

```
# let's create DF - employees
```

```
df_emp <- data.frame(id = 1:6,
  name = c("Max", "Jane", "John", "Tony", "Janis", "Helen"),
  surname = c("Gordon", "Smith", "Don", "Price", "Jett", "Dust"),
  age = c(55, 35, 46, 22, 60, 27),
  date_start_work = c(as.Date("1985-09-01"), as.Date("2010-10-01"), as.Date("2015-01-01"), as.Date("2018-05-01"), as.Date("2019-03-01"), as.Date("2020-07-01")),
  gender = c("M", "F", "M", "M", "F", "M"),
  manager_position = c(T, F, F, F, T, F)
)
```

```
# extract data as data frame (one column) - []  
df_extr <- df_emp["name"]  
df_extr
```

```
      name  
1      Max  
2      Jane  
3      John  
4      Tony  
5     Janis  
6     Helen
```

```
class(df_extr)
```

```
[1] "data.frame"
```

```
# extract data as vector (one column) [[]] $  
df_extr <- df_emp[["age"]]  
df_extr
```

```
[1] 55 35 46 22 60 27
```

```
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```
df_extr <- df_emp$age  
df_extr
```

```
[1] 55 35 46 22 60 27
```

```
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```
# extract multiple columns
df_extr <- df_emp[c("name", "age")]
df_extr
```

```
   name age
1   Max  55
2  Jane  35
3  John  46
4  Tony  22
5 Janis  60
6 Helen  27
```

```
# data frame slicing
df_emp
```

```
   id name surname age date_start_work gender manager_position
1  1   Max  Gordon  55    1985-09-01      M             TRUE
2  2  Jane   Smith  35    2010-10-01      F             FALSE
3  3  John    Don  46    1999-06-01      M             FALSE
4  4  Tony   Price  22    2019-03-01      M             FALSE
5  5 Janis    Jett  60    1980-04-15      F             TRUE
6  6 Helen   Dust  27    2015-02-20      M             FALSE
```

```
#extract second row in name column (1 cell)
df_emp[2,2]
```

```
[1] "Jane"
```

```
df_emp[2,"name"]
```

```
[1] "Jane"
```

```
# extract first 4 rows of last 2 columns
df_emp[1:4, 6:7]
```

```
   gender manager_position
1      M             TRUE
2      F             FALSE
3      M             FALSE
4      M             FALSE
```

```
df_emp[1:4, c("gender", "manager_position")]
```

	gender	manager_position
1	M	TRUE
2	F	FALSE
3	M	FALSE
4	M	FALSE

```
# extract first column (all rows)  
df_emp[,1]
```

```
[1] 1 2 3 4 5 6
```

```
df_emp[, "id"]
```

```
[1] 1 2 3 4 5 6
```

```
df_emp$id
```

```
[1] 1 2 3 4 5 6
```

```
# extract last 2 rows (all columns)  
df_emp[5:6,]
```

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```
cols <- colnames(df_emp)  
df_emp[5:6, cols]
```

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```

# Modifying data frame

# append column
df_emp <- cbind(df_emp, role = c("director", "secretary", "analyst", "researcher", "CEO", "an
df_emp$new_col <- 1

# append rows
df_emp <- rbind(df_emp, list(7, "Mark", "Jax", 32, as.Date("2020-01-01"), "M", F, "researcher

# problem with factor variables (new values not in factor levels)
# easy solution - append new row as data frame (rbind 2 data frames)!!!
# will show few rows later

# remove column
df_emp$new_col <- NULL

# remove row
df_emp <- df_emp[-7,]

# merge two data frames (row wise)
df_new_emp <- data.frame(id = 7,
                        name = "Mark",
                        surname = "Jax",
                        age = 32,
                        date_start_work = as.Date("2020-01-01"),
                        gender = "M",
                        manager_position = F,
                        role = "researcher")

df_emp <- rbind(df_emp, df_new_emp)

# merge two data frames (column wise)
df_attr <- data.frame(eye_color = c("blue", "green", "brown", "hazel", "blue", "brown", "bro
                        hair_color = c("blonde", "light brown", "black", "brown", "blonde", "d
df_emp <- cbind(df_emp, df_attr)

# Tips

# Df summary
summary(df_emp)

```

id		name		surname		age	
Min.	:1.0	Length:7		Length:7		Min.	:22.00
1st Qu.	:2.5	Class :character		Class :character		1st Qu.	:29.50
Median	:4.0	Mode :character		Mode :character		Median	:35.00
Mean	:4.0					Mean	:39.57
3rd Qu.	:5.5					3rd Qu.	:50.50
Max.	:7.0					Max.	:60.00

date_start_work		gender		manager_position		role	
Min.	:1980-04-15	Length:7		Mode :logical		Length:7	
1st Qu.	:1992-07-16	Class :character		FALSE:5		Class :character	
Median	:2010-10-01	Mode :character		TRUE :2		Mode :character	
Mean	:2004-05-06						
3rd Qu.	:2017-02-24						
Max.	:2020-01-01						

eye_color		hair_color	
Length:7		Length:7	
Class :character		Class :character	
Mode :character		Mode :character	

```
# rows subsetting
subset(x = df_emp, gender == "M")
```

	id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE	director
3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

```
subset(x = df_emp, gender == "F" & manager_position == T)
```

	id	name	surname	age	date_start_work	gender	manager_position	role	eye_color
--	----	------	---------	-----	-----------------	--------	------------------	------	-----------

```

5 5 Janis Jett 60 1980-04-15 F TRUE CEO blue
   hair_color
5 blonde

```

```

rows <- which(df_emp[, "gender"] == "M")
df_emp[rows,]

```

```

   id name surname age date_start_work gender manager_position role
1  1  Max  Gordon  55 1985-09-01      M      TRUE  director
3  3  John   Don  46 1999-06-01      M     FALSE  analyst
4  4  Tony  Price  22 2019-03-01      M     FALSE researcher
6  6 Helen  Dust  27 2015-02-20      M     FALSE  analyst
7  7  Mark   Jax  32 2020-01-01      M     FALSE researcher
   eye_color hair_color
1    blue    blonde
3   brown    black
4   hazel    brown
6   brown dark brown
7   brown    brown

```

```

rows <- which(df_emp[, "gender"] == "F" & df_emp[, "manager_position"] == T)
df_emp[rows,]

```

```

   id name surname age date_start_work gender manager_position role eye_color
5  5 Janis Jett 60 1980-04-15      F      TRUE  CEO    blue
   hair_color
5 blonde

```

```

# some calculations regarding data frames
nr_managers <- sum(df_emp$manager_position)
mean_age <- mean(df_emp$age)
df_emp$name_surname <- paste(df_emp$name, df_emp$surname, sep = " ") # merge name and surname

# use apply to sum over columns (age, manager_position)
apply(df_emp[, c("age", "manager_position")], 2, sum)

```

```

age manager_position
277                2

```

19 Factors

Factors are used to represent categorical data. They store both the data values and the corresponding levels.

```
gender_factor <- factor(c("Male", "Female", "Male"))

# Display the factor and its levels
print(gender_factor)
levels(gender_factor)

# create factor variable (gender)
gender <- factor(x = c("male", "female", "female"))

# check new variable
gender
str(gender)
class(gender)
typeof(gender)

# create with ordering
gender <- factor(x = c("male", "female", "female"), ordered = T)
is.ordered(gender)

# check levels
levels(gender) # order of levels based on variable (string alphabetic order)

# we can define our own levels (custom levels order)
gender <- factor(x = c("male", "female", "female"), levels = c("male", "female"), ordered = T)
gender
levels(gender)

# factor properties
levels(gender)
is.factor(gender)
is.ordered(gender)
```



```
# create other object to factor
strings <- c("a", "b", "a", "c")
f_strings <- factor(strings)

#f_string
```

20 Arrays

Arrays are similar to matrices but can have more than two dimensions.

```
# Creating a 3-dimensional array
my_array <- array(1:24, dim = c(3, 4, 2))

# Accessing elements
my_array[1, 2, 1] # Access the element in the first dimension, second row, and first slice
```

```
[1] 4
```

21 Summary

- **Vector:** One-dimensional, homogeneous.
- **List:** One-dimensional, heterogeneous.
- **Matrix:** Two-dimensional, homogeneous.
- **Data Frame:** Two-dimensional, heterogeneous (columns can be different types).
- **Factor:** Categorical data representation.
- **Array:** Multi-dimensional, homogeneous.

22 Manipulating Vectors, Data Frames, and Lists

```
library(haven)  
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(tidyr)
```

23 Vectors: Indexing & Vectorized Ops

```
#Creating vector
x=1:10
x=10:1
x=-5:10
x=c(1:10)
x=c(-5:10)
x=c(1,2,3,4,5,6,7,8,9,10)

# Naming a vector
a=c(1:3)
names(a) #Returns null
```

NULL

```
names(a)=c("one","two","three")
names(a)
```

```
[1] "one"  "two"  "three"
```

```
a
```

```
  one  two three
    1    2    3
```

```
#2)Accessing vector element
x=c(1,3,5,7)
x[2]
```

```
[1] 3
```

```
x[c(1,3)]
```

```
[1] 1 5
```

```
x[-2]
```

```
[1] 1 5 7
```

```
x[-c(1,3)]
```

```
[1] 3 7
```

```
x[0]
```

```
numeric(0)
```

```
y=x[10]  
class(y)
```

```
[1] "numeric"
```

```
#Note: X[0],x[10],output is numeric class only  
#*****  
#3)Modification of Vector elements  
x=c(9,3,5,7)  
x[2]=13  
x[-c(2,3)]=c(11,17)  
x[-1]=c(110,170,70)  
  
#*****  
x=c(11,3,5,7)  
x[9]=x[7]  
#*****  
x=c(11,3,5,7)  
x[9]=x[2]  
  
x=c(1,3,5,7)  
x[2]=x[11]
```

```
#####
x=c(1,3,5,7)
x[c(2,5)]=x[c(4,4)] # It assign 4 element of to 2nd element and 4th element to 5th element
##### from x=1,3,5,7
x=c(11,3,5,7)
x[c(2,7)]=x[c(1,3)]

x=c(1,3,5,7)
x[c(2,3)]=x[c(1,10)]

#4)Airthematic Operations on Vector
x=c(1,3,5,7)
x+10
```

```
[1] 11 13 15 17
```

```
x-5
```

```
[1] -4 -2 0 2
```

```
x*10
```

```
[1] 10 30 50 70
```

```
x/10
```

```
[1] 0.1 0.3 0.5 0.7
```

```
x=c(1,3,5,7)
x%%2
```

```
[1] 0 1 2 3
```

```
x=c(1,3,5,7)
x%%2
```

```
[1] 1 1 1 1
```

```
min(x)
```

```
[1] 1
```

```
max(x)
```

```
[1] 7
```

```
median(x)
```

```
[1] 4
```

```
mean(x)
```

```
[1] 4
```

```
range(x)
```

```
[1] 1 7
```

```
var(x)
```

```
[1] 6.666667
```

```
sd(x)
```

```
[1] 2.581989
```

```
quantile(x)
```

```
 0%  25%  50%  75% 100%  
1.0  2.5  4.0  5.5  7.0
```

```
quantile(1:20,probs=c(.25,0.9))
```

```
 25%   90%  
5.75 18.10
```



```
IQR(x)
```

```
[1] 3
```

```
#5)"WHICH" function  
x=c(2,3,4,5,11,112,133,33)  
x>5
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE
```

```
x=c(2,3,4,5,11,112,133,33)  
y=which(x>5) #Returns the position,not values  
y=x[which(x>5)]  
y=x[x>5]  
  
x=c(2,3,4,5,11,112,133,33)  
min(x)
```

```
[1] 2
```

```
which(x==min(x))
```

```
[1] 1
```

```
x[which(x==min(x))]  
#Returns the value
```

```
[1] 2
```

```
which.min(x)  
#Returns the position,not values
```

```
[1] 1
```

```
x=c(2,3,4,5,11,112,133,33)  
max(x)
```

```
[1] 133
```

```
which(x==max(x)) #Returns the position,not values
```

```
[1] 7
```

```
x[which(x==max(x))]  
#Returns the value
```

```
[1] 133
```

```
which.max(x)#Returns the position,not values
```

```
[1] 7
```

```
x=c(8,7,4,5,11,112,133,33)  
which(x>2 & x<5)
```

```
[1] 3
```

```
x[which(x>2 & x<5)]
```

```
[1] 4
```

```
x=c(8,7,4,5,11,112,133,33)  
which(x>7 | x<12)
```

```
[1] 1 2 3 4 5 6 7 8
```

```
x[which(x>7 | x<12)]
```

```
[1] 8 7 4 5 11 112 133 33
```

```
#6) "REP" function  
x=rep(1:5,times=10)  
x=rep(100,times=10)  
x=rep(c(3,6),times=4)  
x=rep("Kummam",times=5)  
x=rep(c("Ramesh","Kummam"),times=3)
```

```

x=rep(1:4,5:8)
#x=rep(1:4,1:2) #output as invalid argument
x=rep(1:4,c(2,3,5,7))
x=rep(1:4,each=3)
x=rep(1:4,each=2,times=3)

#7) "SEQ" function
x=seq(from=1,to=10,by=3)
#x=seq(from=1,to=10,by=-3) # wrong arguments
x=seq(from=10,to=1,by=-3)
x=seq(from=1,to=10,length=100)
x=seq(from=1,by=2,length=100)
y=seq(from=1,by=3,length=50)
z=c(x,y)

#8) seq_len() & seq_along() functions
x=c(8,7,4,5,11,112,133,33)
length(x)

```

```
[1] 8
```

```
seq_len(length(x))
```

```
[1] 1 2 3 4 5 6 7 8
```

```
seq_along(x) #Returns length of the object
```

```
[1] 1 2 3 4 5 6 7 8
```

```

#9) Dealing with missing values
x=c(11,3,5,7)
x[2]=NA
x[c(2,3)]=NA
is.na(x) #Output is a logical vector

```

```
[1] FALSE TRUE TRUE FALSE
```

```
x[!is.na(x)]
```

```
[1] 11 7
```

```
na.omit(x)
```

```
[1] 11 7  
attr(,"na.action")  
[1] 2 3  
attr(,"class")  
[1] "omit"
```

```
#Note: Observe the below operations carefully
```

```
x=c(1,NA,5,NA)  
#x==NA # not followed it so far  
#x[x==NA]# not followed it so far
```

```
#10) Naming a vector
```

```
x=c(1:3)  
names(x)=c("a","b","c")  
x[c("a","b")]
```

```
a b  
1 2
```

```
y=c("Ramesh","Kumman")  
names(y)=c("First","Last")  
y[c("Last","First")]
```

```
      Last      First  
"Kumman" "Ramesh"
```

```
#9) Checking the availability of elements in a vector
```

```
a=c(1:10)  
b=c(5:15)  
1 %in% a
```

```
[1] TRUE
```

```
1 %in% b
```

```
[1] FALSE
```

```
a %in% b
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
b %in% a
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

```
is.element(a,b)
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
is.element(b,a)
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

```
# print strings  
print("string")
```

```
[1] "string"
```

```
# concatenate strings "a" + "b" = "ab"  
paste("a", "b", sep = "")
```

```
[1] "ab"
```

```
# paste objects of different length  
paste("i", 1:10, sep = ".")
```

```
[1] "i.1" "i.2" "i.3" "i.4" "i.5" "i.6" "i.7" "i.8" "i.9" "i.10"
```

```
paste(c("i","j", "k"), 1:10, sep=".")
```

```
[1] "i.1" "j.2" "k.3" "i.4" "j.5" "k.6" "i.7" "j.8" "k.9" "i.10"
```

```
# paste with collapsing
```

```
paste(c("i","j", "k"), 1:3, sep = "", collapse = "")
```

```
[1] "i1j2k3"
```

```
# paste without collapsing
```

```
paste(c("i","j", "k"), 1:3, sep = "")
```

```
[1] "i1" "j2" "k3"
```

```
# paste0() shorter version of paste(..., sep="")
```

```
paste0("Hello", "world", ",", "I", "use", "R")
```

```
[1] "Helloworld,IuseR"
```

```
paste("Hello", "world", ",", "I", "use", "R", sep=" ")
```

```
[1] "Hello world , I use R"
```

```
# concatenate strings with cat()
```

```
cat("Hello", "world", "!")
```

```
Hello world !
```

```
# it prints without quotes !!!
```

```
cat("Hello", "world", "!", sep = "/")
```

```
Hello/world/!
```

```
# counting number of characters nchar()
```

```
nchar("Hello world")
```

```
[1] 11
```

```
# load US states names from data frame regarding crime rate
df <- USArrests
head(df)
```

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
Alaska	10.0	263	48	44.5
Arizona	8.1	294	80	31.0
Arkansas	8.8	190	50	19.5
California	9.0	276	91	40.6
Colorado	7.9	204	78	38.7

```
rownames(df)
```

[1] "Alabama"	"Alaska"	"Arizona"	"Arkansas"
[5] "California"	"Colorado"	"Connecticut"	"Delaware"
[9] "Florida"	"Georgia"	"Hawaii"	"Idaho"
[13] "Illinois"	"Indiana"	"Iowa"	"Kansas"
[17] "Kentucky"	"Louisiana"	"Maine"	"Maryland"
[21] "Massachusetts"	"Michigan"	"Minnesota"	"Mississippi"
[25] "Missouri"	"Montana"	"Nebraska"	"Nevada"
[29] "New Hampshire"	"New Jersey"	"New Mexico"	"New York"
[33] "North Carolina"	"North Dakota"	"Ohio"	"Oklahoma"
[37] "Oregon"	"Pennsylvania"	"Rhode Island"	"South Carolina"
[41] "South Dakota"	"Tennessee"	"Texas"	"Utah"
[45] "Vermont"	"Virginia"	"Washington"	"West Virginia"
[49] "Wisconsin"	"Wyoming"		

```
states <- rownames(df)

# convert all states names to upper case
states_upper <- toupper(states)

# convert all names to lower
states_lower <- tolower(states)

# or select which to apply with function casefold
casefold(x = states, upper = T)
```

[1] "ALABAMA"	"ALASKA"	"ARIZONA"	"ARKANSAS"
---------------	----------	-----------	------------

[5]	"CALIFORNIA"	"COLORADO"	"CONNECTICUT"	"DELAWARE"
[9]	"FLORIDA"	"GEORGIA"	"HAWAII"	"IDAHO"
[13]	"ILLINOIS"	"INDIANA"	"IOWA"	"KANSAS"
[17]	"KENTUCKY"	"LOUISIANA"	"MAINE"	"MARYLAND"
[21]	"MASSACHUSETTS"	"MICHIGAN"	"MINNESOTA"	"MISSISSIPPI"
[25]	"MISSOURI"	"MONTANA"	"NEBRASKA"	"NEVADA"
[29]	"NEW HAMPSHIRE"	"NEW JERSEY"	"NEW MEXICO"	"NEW YORK"
[33]	"NORTH CAROLINA"	"NORTH DAKOTA"	"OHIO"	"OKLAHOMA"
[37]	"OREGON"	"PENNSYLVANIA"	"RHODE ISLAND"	"SOUTH CAROLINA"
[41]	"SOUTH DAKOTA"	"TENNESSEE"	"TEXAS"	"UTAH"
[45]	"VERMONT"	"VIRGINIA"	"WASHINGTON"	"WEST VIRGINIA"
[49]	"WISCONSIN"	"WYOMING"		

```
casefold(x = states, upper = F)
```

[1]	"alabama"	"alaska"	"arizona"	"arkansas"
[5]	"california"	"colorado"	"connecticut"	"delaware"
[9]	"florida"	"georgia"	"hawaii"	"idaho"
[13]	"illinois"	"indiana"	"iowa"	"kansas"
[17]	"kentucky"	"louisiana"	"maine"	"maryland"
[21]	"massachusetts"	"michigan"	"minnesota"	"mississippi"
[25]	"missouri"	"montana"	"nebraska"	"nevada"
[29]	"new hampshire"	"new jersey"	"new mexico"	"new york"
[33]	"north carolina"	"north dakota"	"ohio"	"oklahoma"
[37]	"oregon"	"pennsylvania"	"rhode island"	"south carolina"
[41]	"south dakota"	"tennessee"	"texas"	"utah"
[45]	"vermont"	"virginia"	"washington"	"west virginia"
[49]	"wisconsin"	"wyoming"		

```
# character translation
chartr(old = "o", new = "0", x = "Hello World")
```

```
[1] "Hello W0rld"
```

```
# sorting strings
sort(states, decreasing = F) #ascending order
```

[1]	"Alabama"	"Alaska"	"Arizona"	"Arkansas"
[5]	"California"	"Colorado"	"Connecticut"	"Delaware"
[9]	"Florida"	"Georgia"	"Hawaii"	"Idaho"

[13]	"Illinois"	"Indiana"	"Iowa"	"Kansas"
[17]	"Kentucky"	"Louisiana"	"Maine"	"Maryland"
[21]	"Massachusetts"	"Michigan"	"Minnesota"	"Mississippi"
[25]	"Missouri"	"Montana"	"Nebraska"	"Nevada"
[29]	"New Hampshire"	"New Jersey"	"New Mexico"	"New York"
[33]	"North Carolina"	"North Dakota"	"Ohio"	"Oklahoma"
[37]	"Oregon"	"Pennsylvania"	"Rhode Island"	"South Carolina"
[41]	"South Dakota"	"Tennessee"	"Texas"	"Utah"
[45]	"Vermont"	"Virginia"	"Washington"	"West Virginia"
[49]	"Wisconsin"	"Wyoming"		

```
sort(states, decreasing = T) #descending order
```

[1]	"Wyoming"	"Wisconsin"	"West Virginia"	"Washington"
[5]	"Virginia"	"Vermont"	"Utah"	"Texas"
[9]	"Tennessee"	"South Dakota"	"South Carolina"	"Rhode Island"
[13]	"Pennsylvania"	"Oregon"	"Oklahoma"	"Ohio"
[17]	"North Dakota"	"North Carolina"	"New York"	"New Mexico"
[21]	"New Jersey"	"New Hampshire"	"Nevada"	"Nebraska"
[25]	"Montana"	"Missouri"	"Mississippi"	"Minnesota"
[29]	"Michigan"	"Massachusetts"	"Maryland"	"Maine"
[33]	"Louisiana"	"Kentucky"	"Kansas"	"Iowa"
[37]	"Indiana"	"Illinois"	"Idaho"	"Hawaii"
[41]	"Georgia"	"Florida"	"Delaware"	"Connecticut"
[45]	"Colorado"	"California"	"Arkansas"	"Arizona"
[49]	"Alaska"	"Alabama"		

```
# extracting parts of string
# sub string first 3 letters from state name Alabama
substr(x = "Alabama", start = 1, stop = 3)
```

```
[1] "Ala"
```

```
# String matching - back to toy example
help(regex)

# get all country names
#install.packages("countrycode")
require(countrycode)
```

```
Loading required package: countrycode
```

```
countries <- as.vector(countrycode::codelist$country.name.en)
```

```
# countries beginning with letter "A"
grep(pattern = "^A", x = countries)
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
```

```
countries[grep(pattern = "^A", x = countries)]
```

```
[1] "Afghanistan"      "Albania"          "Algeria"
[4] "American Samoa"   "Andorra"          "Angola"
[7] "Anguilla"         "Antarctica"       "Antigua & Barbuda"
[10] "Argentina"        "Armenia"          "Aruba"
[13] "Australia"        "Austria"          "Austria-Hungary"
[16] "Azerbaijan"
```

```
countries[grep(pattern = "^A", x = countries)]
```

```
[1] "Afghanistan"      "Albania"          "Algeria"
[4] "American Samoa"   "Andorra"          "Angola"
[7] "Anguilla"         "Antarctica"       "Antigua & Barbuda"
[10] "Argentina"        "Armenia"          "Aruba"
[13] "Australia"        "Austria"          "Austria-Hungary"
[16] "Azerbaijan"
```

```
# all country names that end with letter "y"
rez <- grep(pattern = "*y$", x = countries)
countries[rez]
```

```
[1] "Austria-Hungary"      "British Indian Ocean Territory"
[3] "Germany"             "Guernsey"
[5] "Hungary"             "Italy"
[7] "Jersey"              "Norway"
[9] "Paraguay"            "Saxony"
[11] "St. Barthélemy"      "Turkey"
[13] "Tuscany"             "Uruguay"
[15] "Vatican City"
```

```
# all country with 2 words or more for a country name
rez <- grep(pattern = "\\w\\s\\w", x = countries)
countries[rez]
```

```
[1] "American Samoa"
[2] "Bouvet Island"
[3] "British Indian Ocean Territory"
[4] "British Virgin Islands"
[5] "Burkina Faso"
[6] "Cape Verde"
[7] "Caribbean Netherlands"
[8] "Cayman Islands"
[9] "Central African Republic"
[10] "Channel Islands"
[11] "Christmas Island"
[12] "Cook Islands"
[13] "Costa Rica"
[14] "Côte d'Ivoire"
[15] "Dominican Republic"
[16] "El Salvador"
[17] "Equatorial Guinea"
[18] "Falkland Islands"
[19] "Faroe Islands"
[20] "French Guiana"
[21] "French Polynesia"
[22] "French Southern Territories"
[23] "German Democratic Republic"
[24] "Heard & McDonald Islands"
[25] "Hesse Electoral"
[26] "Hesse Grand Ducal"
[27] "Hong Kong SAR China"
[28] "Isle of Man"
[29] "Macao SAR China"
[30] "Marshall Islands"
[31] "Mecklenburg Schwerin"
[32] "Micronesia (Federated States of)"
[33] "Netherlands Antilles"
[34] "New Caledonia"
[35] "New Zealand"
[36] "Norfolk Island"
[37] "North Korea"
[38] "North Macedonia"
```

```

[39] "Northern Mariana Islands"
[40] "Orange Free State"
[41] "Palestinian Territories"
[42] "Papua New Guinea"
[43] "Pitcairn Islands"
[44] "Puerto Rico"
[45] "Republic of Vietnam"
[46] "Saint Martin (French part)"
[47] "San Marino"
[48] "Saudi Arabia"
[49] "Serbia and Montenegro"
[50] "Sierra Leone"
[51] "Sint Maarten"
[52] "Solomon Islands"
[53] "South Africa"
[54] "South Georgia & South Sandwich Islands"
[55] "South Korea"
[56] "South Sudan"
[57] "Sri Lanka"
[58] "Svalbard & Jan Mayen"
[59] "São Tomé & Príncipe"
[60] "Turks & Caicos Islands"
[61] "Two Sicilies"
[62] "U.S. Virgin Islands"
[63] "United Arab Emirates"
[64] "United Arab Republic"
[65] "United Kingdom"
[66] "United Province CA"
[67] "United States"
[68] "United States Minor Outlying Islands (the)"
[69] "Vatican City"
[70] "Western Sahara"
[71] "Yemen Arab Republic"
[72] "Yemen People's Republic"
[73] "Åland Islands"

```

```

# all country names that end with letter "e" or "i"
rez <- grep(pattern = "*e$|*i$", x = countries)
countries[rez]

```

```

[1] "Belize"           "Brunei"           "Burundi"
[4] "Cape Verde"       "Chile"             "Congo - Brazzaville"

```

```

[7] "Côte d'Ivoire"      "Djibouti"          "Eswatini"
[10] "Fiji"               "France"            "Greece"
[13] "Guadeloupe"        "Haiti"             "Kiribati"
[16] "Malawi"             "Mali"              "Martinique"
[19] "Mayotte"           "Mozambique"        "Niue"
[22] "Orange Free State" "Sierra Leone"     "Singapore"
[25] "Suriname"          "São Tomé & Príncipe" "Timor-Leste"
[28] "Ukraine"           "Zimbabwe"

```

```

# all country names which contain combination of letters "gin"
rez <- grep(pattern = "*(gin)", x = countries)
countries[rez]

```

```

[1] "British Virgin Islands" "U.S. Virgin Islands"

```

```

# metacharacters and double backslash sign
strings <- c("dollar $", "dollar", "US dollar")

# how to escape metacharacters in R
# we would like to match a word with "$" dollar sign
strings

```

```

[1] "dollar $" "dollar"   "US dollar"

```

```

rez1 <- grep(pattern = "$", x = strings) # wrong way all are find $ indicating end of string
strings[rez1]

```

```

[1] "dollar $" "dollar"   "US dollar"

```

```

# we need to escape $ with \ backslash
#rez2 <- grep(pattern = "\$", x = strings) # wrong way error
#strings[rez2]

# right way $ escape with \\ double backslash
#rez3 <- grep(pattern = "\\$", x = strings) # wrong way error
#strings[rez3]

# Escape dot .
strings <-c("word.word", "word word")
rez <- grep(pattern = "\\.", x = strings)
strings[rez]

```

```
[1] "word.word"
```

```
# the hat sign beginning of the string and dollar sign end of string
strings <- c("Word", "worD")
strings[grepl("^W", strings)]
```

```
[1] "Word"
```

```
strings[grepl("D$", strings)]
```

```
[1] "worD"
```

```
# example of an anchor sequences in R
strings <- c("123", "onetwothree", "1twothree")
strings[grepl("\\d", strings)] # digit character
```

```
[1] "123"          "1twothree"
```

```
strings[grepl("\\D", strings)] # non-digit character
```

```
[1] "onetwothree" "1twothree"
```

```
# example of a character classes
strings <- c("123", "dollar", "shkjl")
strings[grepl("[aeiou]", strings)] # match word with any vowel
```

```
[1] "dollar"
```

```
strings[grepl("[0-9]", strings)] # match word with digits
```

```
[1] "123"
```

```
# find and replace sub() & gsub()
string <- "I have started to learn R, and I already love R."

# replace "R" with "X"
sub(pattern = "R", replacement = "X", x = string) #only first occurrence replaced
```

```
[1] "I have started to learn X, and I already love R."
```

```
gsub(pattern = "R", replacement = "X", x = string) #all occurrences replaced
```

```
[1] "I have started to learn X, and I already love X."
```

```
#replace white space with blank space  
gsub(pattern = "\\s", replacement = "", x = string)
```

```
[1] "IhavestartedtolearnR,andIalreadyloveR."
```

```
# string split, split given sentence by comma ","  
strsplit(x = string, split = ",")
```

```
[[1]]  
[1] "I have started to learn R" " and I already love R."
```

```
# split phone numbers by digits  
numbers <- c("310-555-123", "311-444-456")  
strsplit(x = numbers, split = "-")
```

```
[[1]]  
[1] "310" "555" "123"
```

```
[[2]]  
[1] "311" "444" "456"
```

24 Data Frames with dplyr

```
# create data frame
df1 <- data.frame(col1 = 1:3,
                  col2 = c("a", "b", "c"),
                  col3 = c(T, F, T),
                  col4 = c(as.Date("2020-01-01"), as.Date("2020-01-03"), as.Date("2020-01-03")))

# create data frame - vectors
col1 <- seq(10,100,10)
col2 <- seq(as.Date("2020-01-01"), length = 10, by = "weeks")
col3 <- rep("word", 10)

df2 <- data.frame(num = col1,
                  date = col2,
                  string = col3)

# check DF structure
str(df2)
```

```
'data.frame':  10 obs. of  3 variables:
 $ num      : num  10 20 30 40 50 60 70 80 90 100
 $ date     : Date, format: "2020-01-01" "2020-01-08" ...
 $ string   : chr  "word" "word" "word" "word" ...
```

```
# create data frame - matrix
M <- matrix(data = 1:100, nrow = 10, ncol = 10, byrow = T)
rownames(M) <- paste("row", 1:10, sep = "")
colnames(M) <- paste("col", 1:10, sep = "")
M
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20

row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
df3 <- as.data.frame(M)
df3
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20
row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
# check DF dimensions
dim(df3)
```

```
[1] 10 10
```

```
nrow(df3)
```

```
[1] 10
```

```
ncol(df3)
```

```
[1] 10
```

```
# check DF type / class
class(df3)
```

```
[1] "data.frame"
```

```
typeof(df3)
```

```
[1] "list"
```

```
# 4.3.2 Accessing DF
```

```
# let's create DF - employees
```

```
df_emp <- data.frame(id = 1:6,  
                     name = c("Max", "Jane", "John", "Tony", "Janis", "Helen"),  
                     surname = c("Gordon", "Smith", "Don", "Price", "Jett", "Dust"),  
                     age = c(55, 35, 46, 22, 60, 27),  
                     date_start_work = c(as.Date("1985-09-01"), as.Date("2010-10-01"), as.Date("2015-01-01"),  
                                          as.Date("2012-03-01"), as.Date("2018-05-01"), as.Date("2019-08-01")),  
                     gender = c("M", "F", "M", "M", "F", "M"),  
                     manager_position = c(T, F, F, F, T, F)  
                     )
```

```
# extract data as data frame (one column) - []
```

```
df_extr <- df_emp["name"]
```

```
df_extr
```

```
   name  
1   Max  
2  Jane  
3  John  
4  Tony  
5 Janis  
6 Helen
```

```
class(df_extr)
```

```
[1] "data.frame"
```

```
# extract data as vector (one column) [[]] $
```

```
df_extr <- df_emp[["age"]]
```

```
df_extr
```

```
[1] 55 35 46 22 60 27
```

```
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```
df_extr <- df_emp$age  
df_extr
```

```
[1] 55 35 46 22 60 27
```

```
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```
# extract multiple columns  
df_extr <- df_emp[c("name", "age")]  
df_extr
```

```
      name age  
1    Max  55  
2   Jane  35  
3   John  46  
4   Tony  22  
5 Janis  60  
6 Helen  27
```

```
# data frame slicing  
df_emp
```

```
   id  name surname age date_start_work gender manager_position  
1  1   Max  Gordon  55    1985-09-01      M             TRUE  
2  2  Jane   Smith  35    2010-10-01      F             FALSE  
3  3  John    Don   46    1999-06-01      M             FALSE  
4  4  Tony   Price  22    2019-03-01      M             FALSE  
5  5 Janis    Jett  60    1980-04-15      F             TRUE  
6  6 Helen   Dust  27    2015-02-20      M             FALSE
```

```
#extract second row in name column (1 cell)
df_emp[2,2]
```

```
[1] "Jane"
```

```
df_emp[2,"name"]
```

```
[1] "Jane"
```

```
# extract first 4 rows of last 2 columns
df_emp[1:4, 6:7]
```

	gender	manager_position
1	M	TRUE
2	F	FALSE
3	M	FALSE
4	M	FALSE

```
df_emp[1:4, c("gender", "manager_position")]
```

	gender	manager_position
1	M	TRUE
2	F	FALSE
3	M	FALSE
4	M	FALSE

```
# extract first column (all rows)
df_emp[,1]
```

```
[1] 1 2 3 4 5 6
```

```
df_emp[,"id"]
```

```
[1] 1 2 3 4 5 6
```

```
df_emp$id
```

```
[1] 1 2 3 4 5 6
```

```
# extract last 2 rows (all columns)
df_emp[5:6,]
```

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```
cols <- colnames(df_emp)
df_emp[5:6, cols]
```

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```
# 4.3.3 Modifying data frame
```

```
# append column
```

```
df_emp <- cbind(df_emp, role = c("director", "secretary", "analyst", "researcher", "CEO", "analyst"))
df_emp$new_col <- 1
```

```
# append rows
```

```
df_emp <- rbind(df_emp, list(7, "Mark", "Jax", 32, as.Date("2020-01-01"), "M", F, "researcher"))
```

```
# problem with factor variables (new values not in factor levels)
```

```
# easy solution - append new row as data frame (rbind 2 data frames)!!!
```

```
# will show few rows later
```

```
# remove column
```

```
df_emp$new_col <- NULL
```

```
# remove row
```

```
df_emp <- df_emp[-7,]
```

```
# merge two data frames (row wise)
```

```
df_new_emp <- data.frame(id = 7,
```

[illegible]

id		name		surname		age	
Min.	:1.0	Length:7		Length:7		Min.	:22.00
1st Qu.:	2.5	Class :character		Class :character		1st Qu.:	29.50
Median	:4.0	Mode :character		Mode :character		Median	:35.00
Mean	:4.0					Mean	:39.57
3rd Qu.:	5.5					3rd Qu.:	50.50
Max.	:7.0					Max.	:60.00
date_start_work		gender		manager_position		role	
Min.	:1980-04-15	Length:7		Mode :logical		Length:7	
1st Qu.:	1992-07-16	Class :character		FALSE:5		Class :character	
Median	:2010-10-01	Mode :character		TRUE :2		Mode :character	
Mean	:2004-05-06						
3rd Qu.:	2017-02-24						
Max.	:2020-01-01						
eye_color		hair_color					
Length:7		Length:7					
Class :character		Class :character					
Mode :character		Mode :character					

```
# rows subsetting
subset(x = df_emp, gender == "M")
```

	id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE	director
3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

```
subset(x = df_emp, gender == "F" & manager_position == T)
```

	id	name	surname	age	date_start_work	gender	manager_position	role	eye_color
5	5	Janis	Jett	60	1980-04-15	F	TRUE	CEO	blue

	hair_color
5	blonde

```
rows <- which(df_emp[, "gender"] == "M")
df_emp[rows,]
```

	id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE	director
3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

```
rows <- which(df_emp[, "gender"] == "F" & df_emp[, "manager_position"] == T)
df_emp[rows,]
```

```
   id name surname age date_start_work gender manager_position role eye_color
5  5 Janis      Jett  60      1980-04-15      F              TRUE   CEO      blue
   hair_color
5      blonde
```

```
# some calculations regarding data frames
nr_managers <- sum(df_emp$manager_position)
mean_age <- mean(df_emp$age)
df_emp$name_surname <- paste(df_emp$name, df_emp$surname, sep = " ") # merge name and surname

# use apply to sum over columns (age, manager_position)
apply(df_emp[, c("age", "manager_position")], 2, sum)
```

```
      age manager_position
      277                2
```

```
# if statement

x <- 3
y <- 14

if(x < y){
  print("x is less than y")
}
```

```
[1] "x is less than y"
```

```
# if-else statement

x <- 3
y <- 14

if(x > y){
  print("x is greater than y")
} else{
  print("x is less or equal to y")
}
```



```
[1] "x is less or equal to y"
```

```
# if-else if-else statement

x <- 14
y <- 14

if(x > y){
  print("x is greater than y")
} else if (x < y){
  print("x is less than y")
} else{
  print("x is equal to y")
}
```

```
[1] "x is equal to y"
```

```
# 5.1.2 relational operators

# scalar
x <- 3

x == 4
```

```
[1] FALSE
```

```
x > 0
```

```
[1] TRUE
```

```
x < 5
```

```
[1] TRUE
```

```
x <= 3
```

```
[1] TRUE
```

```
x >= 10
```

```
[1] FALSE
```

```
x %in% c(1,2,3,4,5)
```

```
[1] TRUE
```

```
# vectors
X <- c(F,T,0,10)
Y <- c(T,F,F,T)

X == Y # element-wise comparison
```

```
[1] FALSE FALSE TRUE FALSE
```

```
X > Y
```

```
[1] FALSE TRUE FALSE TRUE
```

```
X < Y
```

```
[1] TRUE FALSE FALSE FALSE
```

```
X >= Y
```

```
[1] FALSE TRUE TRUE TRUE
```

```
X <= Y
```

```
[1] TRUE FALSE TRUE FALSE
```

```
X != Y
```

```
[1] TRUE TRUE FALSE TRUE
```

```
X %in% Y
```

```
[1] TRUE TRUE TRUE FALSE
```

```
# 5.1.3 Logical operators
```

```
X | Y
```

```
[1] TRUE TRUE FALSE TRUE
```

```
#X || Y
```

```
X & Y
```

```
[1] FALSE FALSE FALSE TRUE
```

```
#X && Y
```

```
!X
```

```
[1] TRUE FALSE TRUE FALSE
```

```
# logical operators in if statement (flip coin twice)
```

```
c1 <- "H"
```

```
c2 <- "H"
```

```
if(c1 == "H" & c2 == "H"){  
  paste("You win 10$")  
} else if((c1 == "H" & c2 == "T") | (c1 == "T" & c2 == "H")){  
  paste("You win 2$")  
} else{  
  paste("You lose all the money")  
}
```

```
[1] "You win 10$"
```

```
# loop over iterations and print number of iteration
```

```
for(it in 1:10){  
  print(paste("iteration ", it, sep = ""))  
}
```

```
[1] "iteration 1"
[1] "iteration 2"
[1] "iteration 3"
[1] "iteration 4"
[1] "iteration 5"
[1] "iteration 6"
[1] "iteration 7"
[1] "iteration 8"
[1] "iteration 9"
[1] "iteration 10"
```

```
# sum of numbers in a sequence
sequence <- c(1,2,3,4,5)
s <- 0 # initial sum

for(val in sequence){
  s <- s + val
  print(paste("value = ", val, sep = ""))
  print(paste("s = ", s, sep = ""))
  print("-----")
}
```

```
[1] "value = 1"
[1] "s = 1"
[1] "-----"
[1] "value = 2"
[1] "s = 3"
[1] "-----"
[1] "value = 3"
[1] "s = 6"
[1] "-----"
[1] "value = 4"
[1] "s = 10"
[1] "-----"
[1] "value = 5"
[1] "s = 15"
[1] "-----"
```

```
# for loop with conditional statement
# (count number of even numbers in a vector)
x <- c(1,3,2,4,5,10,22,21,100) # given numbers
count <- 0 # counter for even numbers
```

```
for(val in x){  
  if(val %% 2 == 0){ # if number is divisible with 2  
    count <- count + 1  
  }  
}  
  
print(count)
```

```
[1] 5
```

25 Read sas dataset

```
dm <- haven::read_sas("data/sdtm/dm.sas7bdat")  
ae <- haven::read_sas("data/sdtm/ae.sas7bdat")
```

26 keeping selected columns

```
dm_1 <- dm |>
  dplyr::select(USUBJID,ARM) |>
  dplyr::filter(ARM == "Placebo")

dm_2 <- dm |>
  dplyr::select(USUBJID:ARM)

dm_2 <- dm |>
  dplyr::select(1,2,3)

dm_2 <- dm |>
  dplyr::select(1:3)

dm_2 <- dm |>
  dplyr::select(starts_with("AR"))

dm_2 <- dm |>
  dplyr::select(contains("DT"))

dm2 <- dplyr::select(dm,USUBJID,ARM)
```

27 dropping columns

To select all columns except certain ones, put a “-” in front of the variable to exclude it. For multiple variables, you can use the function `c()` to combine values into a vector or list. This will select all the variables in surveys except ARM, SITEID

```
adsl <- haven::read_sas("data/adam/adsl.sas7bdat")
adsl2 <- dplyr::select(adsl, -c(AGE, SEX, TRT01A))
adsl3 <- dplyr::select(adsl, -c(STUDYID:ARM))
adsl4 <- dplyr::select(adsl, -c(1:6))
adsl4 <- dplyr::select(adsl, -c(1:6, 43))
adsl5 <- dplyr::select(adsl, -starts_with("A"))
adsl6 <- dplyr::select(adsl, -ends_with("DTC"))
adsl7 <- dplyr::select(adsl, -contains("TRT"))
```

There are many helper functions available with `select()` like: `starts_with()`, `ends_with()`, `contains()` among others. These were imported from the `tidyselect` package. You can put a “-” in front of the helper function to negate it. Here is an example using `contains()`: # Filtering rows To choose rows based on a specific criteria, use `filter()`: To select rows where the planned treatment code equals “Placebo”:

```
adsl1 <- adsl |>
  dplyr::select(USUBJID, TRT01A) |>
  dplyr::filter(TRT01A == "Placebo" )

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A == "Placebo" & SEX == "M" & AGE == 70)

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A == "Placebo" & (SEX == "M" | AGE == 70))

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A == "Placebo" & SEX == "M" & AGE %in% c(70, 80))
```



```

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A=="Placebo" & SEX == "M" & !AGE %in% c(70,80))

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A=="Placebo" & !SEX == "M" & !AGE %in% c(70,80))

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A %in% c("Placebo", "Xanomeline High Dose") & !SEX == "M" & !AGE %in% c(70,80))

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A %in% c("Placebo", "Xanomeline High Dose") & !SEX == "M" & AGE >= 70)

```

28 Rename

```
ads11 <- ads1 |>
  dplyr::rename(usubjid=USUBJID)

ads11 <- ads1 |>
  dplyr::rename(usubjid=USUBJID,
               trt=TRT01P)
```

29 Arrange (sorting)

```
adsl1 <- adsl |>
  dplyr::select(USUBJID,SEX,AGE) |>
  dplyr::arrange(AGE,SEX)

adsl1 <- adsl |>
  dplyr::select(USUBJID,SEX,AGE) |>
  dplyr::arrange(AGE,desc(SEX))
```

30 convert var names to lower case

```
tolower(names(ads1))
```

```
[1] "studyid" "usubjid" "subjid" "siteid" "sitegr1" "arm"  
[7] "trt01p" "trt01pn" "trt01a" "trt01an" "trtsdt" "trtedt"  
[13] "trtdur" "avgdd" "cumdose" "age" "agegr1" "agegr1n"  
[19] "ageu" "race" "racen" "sex" "ethnic" "saffl"  
[25] "ittfl" "efffl" "comp8fl" "comp16fl" "comp24fl" "disconfl"  
[31] "dsraefl" "dthfl" "bmibl" "bmiblgr1" "heightbl" "weightbl"  
[37] "educlvl" "disonsdt" "durdis" "durdsgr1" "visit1dt" "rfstdtc"  
[43] "rfendtc" "visnumen" "rfendt" "dcdecod" "dcreascd" "mmsetot"
```

```
ads11 <- ads1
```

```
names(ads11) <- tolower(names(ads11))
```

31 Creating new variables

```
adsl1 <- adsl |>
  dplyr::mutate(name="Sri Ram")

adsl_test <- adsl |>
  dplyr::mutate(
    AGEGR1_t = dplyr::if_else(AGE < 65, "<65", ">=65"),           # character
    SAFFL_t  = dplyr::if_else(!is.na(TRTSDT), "Y", "N")         # character flag
  ) |>
  dplyr::select(USUBJID,AGE,AGEGR1_t,SAFFL_t,TRTSDT)
```

32 bind rows (set operator in SAS)

```
# bind rows
df_a <- data.frame(
  id = 1:3,
  value = c("A", "B", "C"),
  score = c(10, 20, 30)
)
df_b <- data.frame(
  id = 4:6,
  value = c("D", "E", "F"),
  score = c(40, 50, 60)
)

com <- dplyr::bind_rows(df_a,
                        df_b)

df_combined <- dplyr::bind_rows(df_a, df_b, df_a, .id = "source")

df_combined <- dplyr::bind_rows(df_a, df_b, df_a, .id = "source")

df_combined <- dplyr::bind_rows("dset1"=df_a,
                                "dset2"=df_b,
                                "test"=df_a, .id = "Source_var_name")

df_combined <- dplyr::bind_rows(select(df_a, id, value),
                                df_b)

adsl_tot <- dplyr::bind_rows(adsl |>
                             dplyr::select(USUBJID, TRT01P, TRT01PN),

                             adsl |>
                             dplyr::mutate(TRT01P="Total", TRT01PN=99) |>
                             dplyr::select(USUBJID, TRT01P, TRT01PN)
)
```

33 recode (similar to PROC FORMAT in SAS)

```
sex_fm <- c("M"="Male",
           "F"="Female"
)
adsl_test1 <- adsl |>
  dplyr::mutate(gender=dplyr::recode(SEX,!!!sex_fm))

adsl_test2 <- adsl |>
  dplyr::mutate(SEX=dplyr::recode(SEX,"M"="Male",
                                "F"="Female"))

adsl_test <- adsl |>
  dplyr::mutate(
    gn = dplyr::if_else(SEX=="M","Male","Female")           # character
  ) |>
  dplyr::select(USUBJID,SEX,gn) |>
  head(5)
```

34 factor levels

```
adsl_test <- adsl |>
  dplyr::mutate(
    AGEGR1_f = factor(dplyr::if_else(AGE < 65, "<65", ">=65"), levels = c("<65", ">=65")),
    SAFFL_f = factor(dplyr::if_else(!is.na(TRTSDT), "Y", "N"), levels = c("Y", "N"))
  ) |>
  dplyr::select(USUBJID, AGE, AGEGR1_f, SAFFL_f, TRTSDT)
```


35 joins

The dplyr package provides functions to join tables based on shared keys, such as patient IDs. The main types of joins include:

- `inner_join()`
- `left_join()`
- `right_join()`
- `full_join()`
- `semi_join()`
- `anti_join()`

```
# Patient demographics data
patients <- data.frame(
  patient_id = c(101, 102, 103, 104),
  age = c(34, 45, 29, 56),
  gender = c("F", "M", "M", "F")
)

# Patient lab results data
lab_results <- data.frame(
  patient_id = c(103, 104, 105, 106),
  test = c("CBC", "Lipid Panel", "Blood Glucose", "CBC"),
  result = c("Normal", "High Cholesterol", "Elevated", "Normal")
)

# Display the data frames
patients
```

	patient_id	age	gender
1	101	34	F
2	102	45	M
3	103	29	M
4	104	56	F

```
lab_results
```

	patient_id	test	result
1	103	CBC	Normal
2	104	Lipid Panel	High Cholesterol
3	105	Blood Glucose	Elevated
4	106	CBC	Normal

35.1 inner join

`inner_join()`: returns only rows with matching patient IDs in both data frames.

```
# Inner join on patient_id
inner_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender	test	result
1	103	29	M	CBC	Normal
2	104	56	F	Lipid Panel	High Cholesterol

In this example, only patients with `patient_ids` 103 and 104 are in both tables, so only their information appears in the result.

35.2 left join

`left_join()` returns all rows from the patients data frame and matches rows from `lab_results` where possible. Unmatched rows in `lab_results` are filled with NA.

```
# Left join on patient_id
left_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender	test	result
1	101	34	F	<NA>	<NA>
2	102	45	M	<NA>	<NA>
3	103	29	M	CBC	Normal
4	104	56	F	Lipid Panel	High Cholesterol

Here, all patients from `patients` are included, with NA for lab results where no match is found.

35.3 right join

`right_join()` returns all rows from `lab_results`, matching rows from `patients` where possible. Unmatched rows in `patients` are filled with `NA`.

```
# Right join on patient_id
right_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender	test	result
1	103	29	M	CBC	Normal
2	104	56	F	Lipid Panel	High Cholesterol
3	105	NA	<NA>	Blood Glucose	Elevated
4	106	NA	<NA>	CBC	Normal

In this example, all patients with lab results are included, with demographic data from `patients` where available.

35.4 Full Join

`full_join()` returns all rows from both tables, with `NA` for missing matches in either table.

```
# Full join on patient_id
full_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender	test	result
1	101	34	F	<NA>	<NA>
2	102	45	M	<NA>	<NA>
3	103	29	M	CBC	Normal
4	104	56	F	Lipid Panel	High Cholesterol
5	105	NA	<NA>	Blood Glucose	Elevated
6	106	NA	<NA>	CBC	Normal

35.5 Semi Join

`semi_join()` returns only the rows in `patients` that have a match in `lab_results`, without bringing in columns from `lab_results`.

```
# Semi join on patient_id
semi_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender
1	103	29	M
2	104	56	F

Only patients with lab results are included here (patients 103 and 104).

35.6 Anti join

`anti_join()` returns only the rows in `patients` that do not have a match in `lab_results`.

```
# Anti join on patient_id
anti_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender
1	101	34	F
2	102	45	M

```
#example
tab_a <- data.frame(
  id=c(001,002,003,004),
  age=c(25,50,30,40)
)

tab_b <- data.frame(
  id=c(001,002,004,005),
  sex=c("M","F","M","F")
)

#inner join:
injointab <- dplyr::inner_join(tab_a,tab_b,by="id")
#full join:
fulljointab <- full_join(tab_a,tab_b,by="id")
#left join
leftjointab <- left_join(x=tab_a,y=tab_b,by="id")
#right join
rightjointab <- right_join(x=tab_a,y=tab_b,by="id")
```

```
#anti_join
antijointab <- anti_join(x=tab_a,y=tab_b,by="id")
antijointab1 <- anti_join(x=tab_b,y=tab_a,by="id")
```

36 Reshaping data with tidyr

```
test <- data.frame(
  visit=c("wk0","wk2","wk4","wk6","wk12"),
  drug_a=c(10,20,30,40,50),
  drug_b=c(50,40,30,20,10),
  drug_c=c(15,25,35,45,55))
#tidyr::pivot_longer
library(tidyr)
test1 <- pivot_longer(data=test,
                      cols=c(drug_a,drug_b,drug_c),
                      names_to="drug",
                      values_to = "dose")

test2 <- pivot_wider(data=test1,
                    id_cols =visit,
                    names_prefix = "d_", # it is optional
                    values_from = dose,
                    names_from = drug)
```

37 counting with n()

When working with data, we often want to know the number of observations found for each factor or combination of factors. For this task, dplyr provides `count()`. For example, if we wanted to count the number of rows of data for each sex, we would do:

```
cnt <- adsl |>
  dplyr::count(SEX)

# race count by treatment
cnt2 <- adsl |>
  dplyr::count(TRT01P, RACE)

#treatment count
cnt3 <- adsl |>
  dplyr::filter(!is.na(TRT01P)) |>
  dplyr::count(TRT01P)

adae <- haven::read_sas("data/adam/adae.sas7bdat") |>
  dplyr::mutate(dplyr::across(where(is.character),~ dplyr::na_if(.,"")))

adsl <- haven::read_sas("data/adam/adsl.sas7bdat") |>
  dplyr::mutate(dplyr::across(where(is.character),~ dplyr::na_if(.,"")))

bign <- adsl |>
  dplyr::filter(SAFFL == "Y") |>
  dplyr::count(TRT01AN, TRT01A) |>
  dplyr::rename(TRTA = TRT01A, TRTAN = TRT01AN, bign = n)

bign1 <- dplyr::count(adsl, TRT01AN, TRT01A) |>
  dplyr::rename(TRTA = TRT01A, TRTAN = TRT01AN, bign = n)

adae1 <- adae |>
  dplyr::distinct(USUBJID, AEBODSYS, AEDECOD, TRTA, TRTAN, SAFFL)
```

```

ae_t <- adae1 |>
  dplyr::filter(SAFFL == "Y") |>
  dplyr::group_by(TRTA) |>
  dplyr::count(AEBODSYS, AEDECOD)

ae_t1 <- ae_t |>
  dplyr::left_join(bign, by = "TRTA") |>
  dplyr::mutate(
    pct = paste0("(", round(n / bign * 100, 2), "%"),
    val = paste0(n, " ", pct)
  )

ae2 <- pivot_wider(
  data = ae_t1,
  id_cols = c(AEBODSYS, AEDECOD),
  names_from = TRTA,
  values_from = val,
  values_fill = "0 (0.0)"
)

```


38 dplyr distinct()

The `distinct()` function in `dplyr` is used to return unique/distinct rows from a data frame or tibble. It removes duplicate rows based on the values in specified columns or all columns if none are specified.

```
# distinct
distinct_trt <- adsl |>
  dplyr::distinct(TRT01P, TRT01PN)
print(distinct_trt)
```

```
# A tibble: 3 x 2
  TRT01P          TRT01PN
  <chr>          <dbl>
1 Placebo          0
2 Xanomeline High Dose 81
3 Xanomeline Low Dose  54
```

39 case_when()

The `case_when()` function in `dplyr` is a powerful tool for creating new variables based on multiple conditions. It allows you to specify a series of conditions and corresponding values to assign when those conditions are met.

```
adsl1 <- adsl |>
dplyr::mutate(
  agegrp1 = dplyr::case_when(
    AGE < 18 ~ "Child",
    AGE >= 18 & AGE < 65 ~ "Adult",
    AGE >= 65 ~ "Senior",
    TRUE ~ "Unknown" # Default case
  )
) |>
dplyr::select(USUBJID, AGE, agegrp1) |>
head(10)
```

40 create seq no.

```
class <- haven::read_sas("data/class.sas7bdat")

t <- dplyr::arrange(class, Age)
# Create t1 data set with first and last records for each unique value of 'age'

t1 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
  dplyr::mutate(seq = dplyr::row_number())

t2 <- class |>
  dplyr::mutate(seq = dplyr::row_number(), .by = Age) |>
  dplyr::arrange(Age)

t3 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
  dplyr::filter(dplyr::row_number() == 1 )

t4 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
  dplyr::filter(dplyr::row_number() == n())

t5 <- class |>
  dplyr::group_by(Age) |>
  dplyr::filter(row_number() == 1 & row_number() == n())

t6 <- class |>
  dplyr::group_by(Age) |>
  dplyr::filter(row_number() == 1|row_number() == n())
```

41 Lists: lapply, purrr

```
lst <- list(a=1:3, b=10:12)  
lapply(lst, mean)
```

```
$a  
[1] 2
```

```
$b  
[1] 11
```

42 Exercises

1. Using `across()`, standardize $(x - \text{mean}) / \text{sd}$ numeric columns.
2. Create a row-wise mean of `age` and `wt`.
3. Split `df` by `grp` and compute group means with `lapply` or `purrr::map`.

43 String Functions

44 String Functions in Base R

Strings (or character vectors) are a fundamental data type in R, and base R provides a variety of functions to manipulate and analyze strings. Here are some commonly used string functions in base R:

```
# Create a character vector
text <- c("Hello, World!", "R is great", "Data Science")
```

44.1 Basic String Functions

```
# nchar(): Returns the number of characters in a string
nchar(text)
```

```
[1] 13 10 12
```

```
# tolower(): Converts a string to lowercase
tolower(text)
```

```
[1] "hello, world!" "r is great"    "data science"
```

```
# toupper(): Converts a string to uppercase
toupper(text)
```

```
[1] "HELLO, WORLD!" "R IS GREAT"    "DATA SCIENCE"
```

```
# substr(): Extracts a substring from a string
substr(text, start = 1, stop = 5)
```

```
[1] "Hello" "R is " "Data "
```

```
# paste(): Concatenates strings together
paste("Hello", "R", sep = " ")
```

```
[1] "Hello R"
```

```
# paste0(): Concatenates strings without any separator
paste0("Hello", "R")
```

```
[1] "HelloR"
```

```
# trimws(): Trims leading and trailing whitespace from a string
trimws("  Hello, World!  ")
```

```
[1] "Hello, World!"
```

44.2 Pattern Matching and Replacement

```
# grep(): Searches for patterns in a character vector and returns the indices of matches
grep("R", text)
```

```
[1] 2
```

```
# grepl(): Searches for patterns and returns a logical vector indicating matches
grepl("R", text)
```

```
[1] FALSE TRUE FALSE
```

```
# sub(): Replaces the first occurrence of a pattern in a string
sub("great", "awesome", text)
```

```
[1] "Hello, World!" "R is awesome" "Data Science"
```

```
# gsub(): Replaces all occurrences of a pattern in a string
gsub(" ", "_", text)
```

```
[1] "Hello,_World!" "R_is_great" "Data_Science"
```


44.3 String Splitting and Joining

```
# strsplit(): Splits a string into substrings based on a specified delimiter
strsplit(text, split = " ")
```

```
[[1]]
[1] "Hello," "World!"

[[2]]
[1] "R"      "is"      "great"

[[3]]
[1] "Data"    "Science"
```

```
# unlist(): Converts a list to a vector (useful after strsplit)
unlist(strsplit(text, split = " "))
```

```
[1] "Hello," "World!" "R"      "is"      "great" "Data"    "Science"
```

44.4 Regular Expressions

```
# regexpr(): Finds the position of the first match of a pattern in a string
regexpr("R", text)
```

```
[1] -1  1 -1
attr("match.length")
[1] -1  1 -1
attr("index.type")
[1] "chars"
attr("useBytes")
[1] TRUE
```

```
# gregexpr(): Finds the positions of all matches of a pattern in a string
gregexpr(" ", text)
```

```
[[1]]
[1] 7
attr(,"match.length")
[1] 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
[[2]]
[1] 2 5
attr(,"match.length")
[1] 1 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
[[3]]
[1] 5
attr(,"match.length")
[1] 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
# regmatches(): Extracts the matched substrings based on the positions returned by regexpr or
regmatches(text, gregexpr(" ", text))
```

```
[[1]]
[1] " "
```

```
[[2]]
[1] " " " " "
```

```
[[3]]
[1] " "
```

44.5 Example Usage

```
# Example: Convert text to lowercase and replace spaces with underscores
cleaned_text <- tolower(gsub(" ", "_", text))
cleaned_text
```

```
[1] "hello,_world!" "r_is_great"    "data_science"
```

These functions provide a solid foundation for string manipulation in R. For more advanced string operations, you # using the **stringr** package, which offers a more consistent and user-friendly interface for string handling.

45 Overall differences

We'll begin with a lookup table between the most important stringr functions and their base R equivalents.

```
library(stringr)
data_stringr_base_diff <- tibble::tribble(
  ~stringr,
  "str_detect(string, pattern)",
  "str_dup(string, times)",
  "str_extract(string, pattern)",
  "str_extract_all(string, pattern)",
  "str_length(string)",
  "str_locate(string, pattern)",
  "str_locate_all(string, pattern)",
  "str_match(string, pattern)",
  "str_order(string)",
  "str_replace(string, pattern, replacement)",
  "str_replace_all(string, pattern, replacement)",
  "str_sort(string)",
  "str_split(string, pattern)",
  "str_sub(string, start, end)",
  "str_subset(string, pattern)",
  "str_to_lower(string)",
  "str_to_title(string)",
  "str_to_upper(string)",
  "str_trim(string)",
  "str_which(string, pattern)",
  "str_wrap(string)",
  ~base_r,
  "grepl(pattern, x)",
  "strrep(x, times)",
  "regmatches(x, m = regexpr(pattern, text))",
  "regmatches(x, m = gregexpr(pattern, text))",
  "nchar(x)",
  "regexpr(pattern, text)",
  "gregexpr(pattern, text)",
  "regmatches(x, m = regexec(pattern, text))",
  "order(...)",
  "sub(pattern, replacement, x)",
  "gsub(pattern, replacement, x)",
  "sort(x)",
  "strsplit(x, split)",
  "substr(x, start, stop)",
  "grep(pattern, x, value = TRUE)",
  "tolower(x)",
  "tools::toTitleCase(text)",
  "toupper(x)",
  "trimws(x)",
  "grep(pattern, x)",
  "strwrap(x)"
)

# create MD table, arranged alphabetically by stringr fn name
data_stringr_base_diff |>
  dplyr::mutate(dplyr::across(
    .cols = everything(),
    .fns = ~ paste0("`", .x, "`")
  )) |>
```

stringr	base R
<code>str_detect(string, pattern)</code>	<code>grepl(pattern, x)</code>
<code>str_dup(string, times)</code>	<code>strrep(x, times)</code>
<code>str_extract(string, pattern)</code>	<code>regmatches(x, m = regexpr(pattern, text))</code>
<code>str_extract_all(string, pattern)</code>	<code>regmatches(x, m = gregexpr(pattern, text))</code>
<code>str_length(string)</code>	<code>nchar(x)</code>
<code>str_locate(string, pattern)</code>	<code>regexpr(pattern, text)</code>
<code>str_locate_all(string, pattern)</code>	<code>gregexpr(pattern, text)</code>
<code>str_match(string, pattern)</code>	<code>regmatches(x, m = regexec(pattern, text))</code>
<code>str_order(string)</code>	<code>order(...)</code>
<code>str_replace(string, pattern, replacement)</code>	<code>sub(pattern, replacement, x)</code>
<code>str_replace_all(string, pattern, replacement)</code>	<code>gsub(pattern, replacement, x)</code>
<code>str_sort(string)</code>	<code>sort(x)</code>
<code>str_split(string, pattern)</code>	<code>strsplit(x, split)</code>
<code>str_sub(string, start, end)</code>	<code>substr(x, start, stop)</code>
<code>str_subset(string, pattern)</code>	<code>grep(pattern, x, value = TRUE)</code>
<code>str_to_lower(string)</code>	<code>tolower(x)</code>
<code>str_to_title(string)</code>	<code>tools::toTitleCase(text)</code>
<code>str_to_upper(string)</code>	<code>toupper(x)</code>
<code>str_trim(string)</code>	<code>trimws(x)</code>
<code>str_which(string, pattern)</code>	<code>grep(pattern, x)</code>
<code>str_wrap(string)</code>	<code>strwrap(x)</code>

```

dplyr::arrange(stringr) |>
dplyr::rename(`base R` = base_r) |>
gt::gt() |>
gt::fmt_markdown(columns = everything()) |>
gt::tab_options(column_labels.font.weight = "bold")

```

Overall the main differences between base R and stringr are:

1. stringr functions start with `str_` prefix; base R string functions have no consistent naming scheme.
2. The order of inputs is usually different between base R and stringr. In base R, the `pattern` to match usually comes first; in stringr, the `string` to manipulate always comes first. This makes stringr easier to use in pipes, and with `lapply()` or `purrr::map()`.
3. Functions in stringr tend to do less, where many of the string processing functions in base R have multiple purposes.

4. The output and input of stringr functions has been carefully designed. For example, the output of `str_locate()` can be fed directly into `str_sub()`; the same is not true of `regexpr()` and `substr()`.
5. Base functions use arguments (like `perl`, `fixed`, and `ignore.case`) to control how the pattern is interpreted. To avoid dependence between arguments, stringr instead uses helper functions (like `fixed()`, `regex()`, and `coll()`).

Next we'll walk through each of the functions, noting the similarities and important differences. These examples are adapted from the stringr documentation and here they are contrasted with the analogous base R operations.

46 Detect matches

46.1 `str_detect()`: Detect the presence or absence of a pattern in a string

Suppose you want to know whether each word in a vector of fruit names contains an “a”.

```
fruit <- c("apple", "banana", "pear", "pineapple")  
  
# base  
grepl(pattern = "a", x = fruit)
```

```
[1] TRUE TRUE TRUE TRUE
```

```
#stringr  
stringr::str_detect(fruit, pattern = "a")
```

```
[1] TRUE TRUE TRUE TRUE
```

In base you would use `grepl()` (see the “l” and think logical) while in `stringr` you use `str_detect()` (see the verb “detect” and think of a yes/no action).

46.2 `str_which()`: Find positions matching a pattern

Now you want to identify the positions of the words in a vector of fruit names that contain an “a”.

```
# base  
grep(pattern = "a", x = fruit)
```

```
[1] 1 2 3 4
```

```
# stringr
str_which(fruit, pattern = "a")
```

```
[1] 1 2 3 4
```

In base you would use `grep()` while in stringr you use `str_which()` (by analogy to `which()`).

46.3 `str_count()`: Count the number of matches in a string

How many “a”s are in each fruit?

```
# base
loc <- gregexpr(pattern = "a", text = fruit, fixed = TRUE)
sapply(loc, function(x) length(attr(x, "match.length")))
```

```
[1] 1 3 1 1
```

```
# stringr
str_count(fruit, pattern = "a")
```

```
[1] 1 3 1 1
```

This information can be gleaned from `gregexpr()` in base, but you need to look at the `match.length` attribute as the vector uses a length-1 integer vector (-1) to indicate no match.

46.4 `str_locate()`: Locate the position of patterns in a string

Within each fruit, where does the first “p” occur? Where are all of the “p”s?

```
fruit3 <- c("papaya", "lime", "apple")

# base
str(gregexpr(pattern = "p", text = fruit3))
```


List of 3

```
$ : int [1:2] 1 3
..- attr(*, "match.length")= int [1:2] 1 1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
$ : int -1
..- attr(*, "match.length")= int -1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
$ : int [1:2] 2 3
..- attr(*, "match.length")= int [1:2] 1 1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
```

```
# stringr
str_locate(fruit3, pattern = "p")
```

```
      start end
[1,]      1   1
[2,]     NA  NA
[3,]      2   2
```

```
str_locate_all(fruit3, pattern = "p")
```

```
[[1]]
      start end
[1,]      1   1
[2,]      3   3

[[2]]
      start end

[[3]]
      start end
[1,]      2   2
[2,]      3   3
```

46.5 str_subset(): Keep strings matching a pattern, or find positions

We may want to retrieve strings that contain a pattern of interest:

```
# base
grep(pattern = "g", x = fruit, value = TRUE)
```

```
character(0)
```

```
# stringr
str_subset(fruit, pattern = "g")
```

```
character(0)
```

46.6 str_extract(): Extract matching patterns from a string

We may want to pick out certain patterns from a string, for example, the digits in a shopping list:

```
shopping_list <- c("apples x4", "bag of flour", "10", "milk x2")

# base
matches <- regexpr(pattern = "\\d+", text = shopping_list) # digits
regmatches(shopping_list, m = matches)
```

```
[1] "4" "10" "2"
```

```
matches <- gregexpr(pattern = "[a-z]+", text = shopping_list) # words
regmatches(shopping_list, m = matches)
```

```
[[1]]
[1] "apples" "x"
```

```
[[2]]
[1] "bag" "of" "flour"
```

```
[[3]]
character(0)
```

```
[[4]]
[1] "milk" "x"
```

```
# stringr
str_extract(shopping_list, pattern = "\\d+")
```

```
[1] "4" NA "10" "2"
```

```
str_extract_all(shopping_list, "[a-z]+")
```

```
[[1]]
```

```
[1] "apples" "x"
```

```
[[2]]
```

```
[1] "bag" "of" "flour"
```

```
[[3]]
```

```
character(0)
```

```
[[4]]
```

```
[1] "milk" "x"
```

Base R requires the combination of `regexpr()` with `regmatches()`; but note that the strings without matches are dropped from the output. `stringr` provides `str_extract()` and `str_extract_all()`, and the output is always the same length as the input.

46.7 `str_match()`: Extract matched groups from a string

We may also want to extract groups from a string. Here I'm going to use the scenario from Section 14.4.3 in [R for Data Science](#).

```
head(sentences)
```

```
[1] "The birch canoe slid on the smooth planks."
```

```
[2] "Glue the sheet to the dark blue background."
```

```
[3] "It's easy to tell the depth of a well."
```

```
[4] "These days a chicken leg is a rare dish."
```

```
[5] "Rice is often served in round bowls."
```

```
[6] "The juice of lemons makes fine punch."
```

```
noun <- "([A]a|[Tt]he) ([^ ]+)"

# base
matches <- regexec(pattern = noun, text = head(sentences))
do.call("rbind", regmatches(x = head(sentences), m = matches))
```

```
      [,1]      [,2] [,3]
[1,] "The birch" "The" "birch"
[2,] "the sheet" "the" "sheet"
[3,] "the depth" "the" "depth"
[4,] "The juice" "The" "juice"
```

```
# stringr
str_match(head(sentences), pattern = noun)
```

```
      [,1]      [,2] [,3]
[1,] "The birch" "The" "birch"
[2,] "the sheet" "the" "sheet"
[3,] "the depth" "the" "depth"
[4,] NA         NA    NA
[5,] NA         NA    NA
[6,] "The juice" "The" "juice"
```

As for extracting the full match base R requires the combination of two functions, and inputs with no matches are dropped from the output.

47 Manage lengths

47.1 `str_length()`: The length of a string

To determine the length of a string, base R uses `nchar()` (not to be confused with `length()` which gives the length of vectors, etc.) while `stringr` uses `str_length()`.

```
# base  
nchar(letters)
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
# stringr  
str_length(letters)
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

There are some subtle differences between base and `stringr` here. `nchar()` requires a character vector, so it will return an error if used on a factor. `str_length()` can handle a factor input.

```
# base  
nchar(factor("abc"))
```

```
Error in nchar(factor("abc")): 'nchar()' requires a character vector
```

```
# stringr  
str_length(factor("abc"))
```

```
[1] 3
```

Note that “characters” is a poorly defined concept, and technically both `nchar()` and `str_length()` returns the number of code points. This is usually the same as what you’d consider to be a character, but not always:

```
x <- c("\u00fc", "u\u0308")
x
```

```
[1] "ü" "ü"
```

```
nchar(x)
```

```
[1] 1 2
```

```
str_length(x)
```

```
[1] 1 2
```

47.2 str_pad(): Pad a string

To pad a string to a certain width, use stringr's `str_pad()`. In base R you could use `sprintf()`, but unlike `str_pad()`, `sprintf()` has many other functionalities.

```
# base
sprintf("%30s", "Sriram")
```

```
[1] "                               Sriram"
```

```
sprintf("%-30s", "Sriram")
```

```
[1] "Sriram                               "
```

```
# "both" is not as straightforward
```

```
# stringr
rbind(
  str_pad("Sriram", 30, "left"),
  str_pad("Sriram", 30, "right"),
  str_pad("Sriram", 30, "both")
)
```

```
      [,1]
[1,] "                               Sriram"
[2,] "Sriram                               "
[3,] "          Sriram                               "
```

47.3 str_trunc(): Truncate a character string

The stringr package provides an easy way to truncate a character string: `str_trunc()`. Base R has no function to do this directly.

```
x <- "This string is moderately long"

# stringr
rbind(
  str_trunc(x, 20, "right"),
  str_trunc(x, 20, "left"),
  str_trunc(x, 20, "center")
)
```

```
      [,1]
[1,] "This string is mo..."
[2,] "...s moderately long"
[3,] "This stri...ely long"
```

47.4 str_trim(): Trim whitespace from a string

Similarly, stringr provides `str_trim()` to trim whitespace from a string. This is analogous to base R's `trimws()` added in R 3.3.0.

```
# base
trimws(" String with trailing and leading white space\t")
```

```
[1] "String with trailing and leading white space"
```

```
trimws("\n\nString with trailing and leading white space\n\n")
```

```
[1] "String with trailing and leading white space"
```

```
# stringr
str_trim(" String with trailing and leading white space\t")
```

```
[1] "String with trailing and leading white space"
```

```
str_trim("\n\nString with trailing and leading white space\n\n")
```

```
[1] "String with trailing and leading white space"
```

The stringr function `str_squish()` allows for extra whitespace within a string to be trimmed (in contrast to `str_trim()` which removes whitespace at the beginning and/or end of string). In base R, one might take advantage of `gsub()` to accomplish the same effect.

```
# stringr
str_squish(" String with trailing, middle,   and leading white space\t")
```

```
[1] "String with trailing, middle, and leading white space"
```

```
str_squish("\n\nString with excess, trailing and leading white space\n\n")
```

```
[1] "String with excess, trailing and leading white space"
```

47.5 str_wrap(): Wrap strings into nicely formatted paragraphs

`strwrap()` and `str_wrap()` use different algorithms. `str_wrap()` uses the famous [Knuth-Plass algorithm](#).

```
gettysburg <- "Four score and seven years ago our fathers brought forth on this continent, a  
# base  
cat(strwrap(gettysburg, width = 60), sep = "\n")
```

```
Four score and seven years ago our fathers brought forth on  
this continent, a new nation, conceived in Liberty, and  
dedicated to the proposition that all men are created  
equal.
```

```
# stringr  
cat(str_wrap(gettysburg, width = 60), "\n")
```

```
Four score and seven years ago our fathers brought forth  
on this continent, a new nation, conceived in Liberty, and  
dedicated to the proposition that all men are created equal.
```


Note that `strwrap()` returns a character vector with one element for each line; `str_wrap()` returns a single string containing line breaks.

48 Mutate strings

48.1 `str_replace()`: Replace matched patterns in a string

To replace certain patterns within a string, `stringr` provides the functions `str_replace()` and `str_replace_all()`. The base R equivalents are `sub()` and `gsub()`. Note the difference in default input order again.

```
fruits <- c("apple", "banana", "pear", "pineapple")  
  
# base  
sub("[aeiou]", "-", fruits)
```

```
[1] "-pple"      "b-nana"     "p-ar"       "p-neapple"
```

```
gsub("[aeiou]", "-", fruits)
```

```
[1] "-ppl-"      "b-n-n-"     "p--r"       "p-n--ppl-"
```

```
# stringr  
str_replace(fruits, "[aeiou]", "-")
```

```
[1] "-pple"      "b-nana"     "p-ar"       "p-neapple"
```

```
str_replace_all(fruits, "[aeiou]", "-")
```

```
[1] "-ppl-"      "b-n-n-"     "p--r"       "p-n--ppl-"
```

48.2 case: Convert case of a string

Both `stringr` and base R have functions to convert to upper and lower case. Title case is also provided in `stringr`.

```
dog <- "The quick brown dog"
```

```
# base  
toupper(dog)
```

```
[1] "THE QUICK BROWN DOG"
```

```
tolower(dog)
```

```
[1] "the quick brown dog"
```

```
tools::toTitleCase(dog)
```

```
[1] "The Quick Brown Dog"
```

```
# stringr  
str_to_upper(dog)
```

```
[1] "THE QUICK BROWN DOG"
```

```
str_to_lower(dog)
```

```
[1] "the quick brown dog"
```

```
str_to_title(dog)
```

```
[1] "The Quick Brown Dog"
```

In stringr we can control the locale, while in base R locale distinctions are controlled with global variables. Therefore, the output of your base R code may vary across different computers with different global settings.

```
# stringr  
str_to_upper("i") # English
```

```
[1] "I"
```

```
str_to_upper("i", locale = "tr") # Turkish
```

```
[1] "i"
```

49 Join and split

49.1 `str_flatten()`: Flatten a string

If we want to take elements of a string vector and collapse them to a single string we can use the `collapse` argument in `paste()` or use `stringr`'s `str_flatten()`.

```
# base
paste0(letters, collapse = "-")
```

```
[1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

```
# stringr
str_flatten(letters, collapse = "-")
```

```
[1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

The advantage of `str_flatten()` is that it always returns a vector the same length as its input; to predict the return length of `paste()` you must carefully read all arguments.

49.2 `str_dup()`: duplicate strings within a character vector

To duplicate strings within a character vector use `strrep()` (in R 3.3.0 or greater) or `str_dup()`:

```
fruit <- c("apple", "pear", "banana")

# base
strrep(fruit, 2)
```

```
[1] "appleapple"  "pearpear"    "bananabanana"
```

```
strrep(fruit, 1:3)
```

```
[1] "apple"          "pearpear"       "bananabanabanana"
```

```
# stringr  
str_dup(fruit, 2)
```

```
[1] "appleapple"    "pearpear"      "bananabanana"
```

```
str_dup(fruit, 1:3)
```

```
[1] "apple"          "pearpear"       "bananabanabanana"
```

49.3 str_split(): Split up a string into pieces

To split a string into pieces with breaks based on a particular pattern match stringr uses `str_split()` and base R uses `strsplit()`. Unlike most other functions, `strsplit()` starts with the character vector to modify.

```
fruits <- c(  
  "apples and oranges and pears and bananas",  
  "pineapples and mangos and guavas"  
)  
# base  
strsplit(fruits, " and ")
```

```
[[1]]  
[1] "apples" "oranges" "pears"  "bananas"
```

```
[[2]]  
[1] "pineapples" "mangos"    "guavas"
```

```
# stringr  
str_split(fruits, " and ")
```

```
[[1]]
[1] "apples" "oranges" "pears"  "bananas"
```

```
[[2]]
[1] "pineapples" "mangos"      "guavas"
```

The `stringr` package's `str_split()` allows for more control over the split, including restricting the number of possible matches.

```
# stringr
str_split(fruits, " and ", n = 3)
```

```
[[1]]
[1] "apples"          "oranges"          "pears and bananas"
```

```
[[2]]
[1] "pineapples" "mangos"      "guavas"
```

```
str_split(fruits, " and ", n = 2)
```

```
[[1]]
[1] "apples"          "oranges and pears and bananas"
```

```
[[2]]
[1] "pineapples"      "mangos and guavas"
```

49.4 `str_glue()`: Interpolate strings

It's often useful to interpolate varying values into a fixed string. In base R, you can use `sprintf()` for this purpose; `stringr` provides a wrapper for the more general purpose [glue](#) package.

```
name <- "Fred"
age <- 50
anniversary <- as.Date("1991-10-12")

# base
sprintf(
  "My name is %s my age next year is %s and my anniversary is %s.",
```

```
name,  
age + 1,  
format(anniversary, "%A, %B %d, %Y")  
)
```

```
[1] "My name is Fred my age next year is 51 and my anniversary is Saturday, October 12, 1991"
```

```
# stringr  
str_glue(  
  "My name is {name}, ",  
  "my age next year is {age + 1}, ",  
  "and my anniversary is {format(anniversary, '%A, %B %d, %Y')}".  
)
```

```
My name is Fred, my age next year is 51, and my anniversary is Saturday, October 12, 1991.
```


50 Order strings

50.1 `str_order()`: Order or sort a character vector

Both base R and stringr have separate functions to order and sort strings.

```
# base  
order(letters)
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  
[26] 26
```

```
sort(letters)
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"  
[20] "t" "u" "v" "w" "x" "y" "z"
```

```
# stringr  
str_order(letters)
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  
[26] 26
```

```
str_sort(letters)
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"  
[20] "t" "u" "v" "w" "x" "y" "z"
```

Some options in `str_order()` and `str_sort()` don't have analogous base R options. For example, the stringr functions have a `locale` argument to control how to order or sort. In base R the locale is a global setting, so the outputs of `sort()` and `order()` may differ across different computers. For example, in the Norwegian alphabet, å comes after z:

```
x <- c("â", "a", "z")
str_sort(x)
```

```
[1] "a" "â" "z"
```

```
str_sort(x, locale = "no")
```

```
[1] "a" "z" "â"
```

The stringr functions also have a **numeric** argument to sort digits numerically instead of treating them as strings.

```
# stringr
x <- c("100a10", "100a5", "2b", "2a")
str_sort(x)
```

```
[1] "100a10" "100a5" "2a" "2b"
```

```
str_sort(x, numeric = TRUE)
```

```
[1] "2a" "2b" "100a5" "100a10"
```

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```

ae <- haven::read_sas("./data/sdtm/ae.sas7bdat")
ae1 <- ae |>
  select (USUBJID, AEDECOD, AEBODSYS)

#find function in sas, str_detect

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "HER"))

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "^HIATUS ")) # start of the string

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "HERNIA$")) # end of the string

#substr in sas

ae2 <- ae1 |>
  mutate(newvar1=str_sub(AEDECOD, 1, 6),
         newvar2=str_sub(AEDECOD, 2, 6),
         newvar3=str_sub(AEDECOD, -3),
         newvar4=str_length(AEDECOD))

#cat function in sas , Str_c in r

ae2 <- ae1 |>
  mutate(newvar1=str_c(AEDECOD, AEBODSYS, sep="/"),
         newvar2=paste(AEDECOD, AEBODSYS, sep="/"))

# scan function in sas, word in r

ae2 <- ae1 |>
  mutate(newvar1=word(AEDECOD, 1),
         newvar2=word(AEDECOD, 2))

# upper & lower case

ae2 <- ae1 |>
  mutate(newvar1=str_to_lower(AEDECOD),
         newvar2=str_to_upper(AEDECOD),
         newvar3=str_to_title(AEDECOD),
         newvar4=str_to_sentence(AEDECOD))

```

```
#str_trim  
  
a <- "  this is    my String  "  
b <- str_trim(a)  
c <- str_replace_all(a," "," ")  
d <- str_squish(a)
```

51 date and time functions

51.0.1 Date and Time Base Functions in R

Date and time manipulation is a common task in data analysis. R provides several base functions to work with dates and times. Here are some of the most commonly used base R functions for handling date and time data:

```
# Get the current date
current_date <- Sys.Date()
print(current_date)
```

```
[1] "2025-11-20"
```

```
# Get the current date and time
current_datetime <- Sys.time()
print(current_datetime)
```

```
[1] "2025-11-20 20:24:40 UTC"
```

51.0.2 Creating Date and Time Objects

51.0.3 Year Formats

Code	Meaning	Example (<code>format(dt, ...)</code>)
%Y	4-digit year	"2025"
%y	2-digit year (00–99)	"25"
%C	Century (year / 100, 00–99)	"20" (for 2025; OS-dep.)

51.0.4 Month Formats

Code	Meaning	Example
%m	Month number (01–12)	"11"
%b	Abbrev. month name (locale)	"Nov"
%B	Full month name (locale)	"November"
%h	Same as %b (abbrev. month name)	"Nov"

51.0.5 Day Formats

Code	Meaning	Example
%d	Day of month (01–31)	"14"
%e	Day of month (1–31, padded with space)	"14" (or " 7" for 7th; OS-dep.)
%j	Day of year (001–366)	"318"

51.0.6 Weekday Formats

Code	Meaning	Example
%a	Abbrev. weekday name (locale)	"Fri"
%A	Full weekday name (locale)	"Friday"
%w	Weekday number (0–6, 0 = Sunday)	"5"
%u	ISO weekday number (1–7, 1 = Monday) (OS-dep.)	"5" (Friday)

51.0.7 Time Formats

Code	Meaning	Example
%H	Hour (00–23)	"14"
%I	Hour (01–12)	"02"
%M	Minute (00–59)	"30"
%S	Second (00–61, allows for leap)	"05"
%p	AM/PM indicator (locale)	"PM"

###Locale-dependent “full date/time” codes

###These depend on your system locale (language + region):

Code	Meaning	Example
%c	Date and time representation	"Fri Nov 14 16:05:30 2025"
%x	Date representation	"11/14/25" (varies by locale)
%X	Time representation	"16:05:30"

```
# Create a Date object
dt <- as.Date("2025-11-14")
# Create a POSIXct object (date-time)
dt <- as.POSIXct("2025-11-14 16:05:30")
# Format examples
format(dt, "%Y-%m-%d") # e.g. "2025-11-14"
```

```
[1] "2025-11-14"
```

```
format(dt, "%m/%d/%y") # e.g. "11/14/25"
```

```
[1] "11/14/25"
```

```
format(dt, "%B %d, %Y") # e.g. "November 14, 2025"
```

```
[1] "November 14, 2025"
```

```
format(dt, "%c") # e.g. "Fri Nov 14 16:05:30 2025"
```

```
[1] "Fri Nov 14 16:05:30 2025"
```

```
format(dt, "%x") # e.g. "11/14/25" (depends on locale)
```

```
[1] "11/14/25"
```

```
format(dt, "%X") # e.g. "16:05:30"
```

```
[1] "16:05:30"
```

SAS informat	R parsing
ddmmyy10.	as.Date(x, "%d/%m/%Y")
ddmmyyd10.	as.Date(x, "%d-%m-%Y")
ddmmyyp10.	as.Date(x, "%d.%m.%Y")
mmddyy10.	as.Date(x, "%m/%d/%Y")
date9.	as.Date(x, "%d%b%Y")
julian7.	as.Date(strptime(x, "%Y%j"))
time8.	as.POSIXct(x, "%H:%M:%S")
time10.	as.POSIXct(x, "%I:%M:%S%p") (if am/pm)
datetime18.	as.POSIXct(x, "%d%b%Y:%H:%M:%S")
datetime20.	as.POSIXct(x, "%d%b%Y:%I:%M:%S%p")

SAS format	Meaning	R equivalent
worddate18.	Month date, year (e.g., February 23, 2003)	format(d, "%B %d, %Y")
weekdate24.	Weekday, month date, year	format(d, "%A, %B %d, %Y")
year.	numeric year	as.integer(format(d, "%Y"))
month.	month number (1–12)	as.integer(format(d, "%m"))
day.	day of month	as.integer(format(d, "%d"))
weekday.	day of week (1–7)	as.integer(format(d, "%u")) (Mon = 1)

```
# Convert a character string to a Date object
date_string <- "2023-10-01"
date_object <- as.Date(date_string)
print(date_object)
```

```
[1] "2023-10-01"
```

```
# Convert a character string to a POSIXct object (date-time)
datetime_string <- "2023-10-01 12:34:56"
datetime_object <- as.POSIXct(datetime_string)
print(datetime_object)
```

```
[1] "2023-10-01 12:34:56 UTC"
```



```
# Extract components from a Date object
year <- format(date_object, "%Y")
month <- format(date_object, "%m")
day <- format(date_object, "%d")
print(paste("Year:", year, "Month:", month, "Day:", day))
```

```
[1] "Year: 2023 Month: 10 Day: 01"
```

```
# Extract components from a POSIXct object
hour <- format(datetime_object, "%H")
minute <- format(datetime_object, "%M")
second <- format(datetime_object, "%S")
print(paste("Hour:", hour, "Minute:", minute, "Second:", second))
```

```
[1] "Hour: 12 Minute: 34 Second: 56"
```

```
# Perform date arithmetic
tomorrow <- current_date + 1
yesterday <- current_date - 1
print(paste("Tomorrow:", tomorrow, "Yesterday:", yesterday))
```

```
[1] "Tomorrow: 2025-11-21 Yesterday: 2025-11-19"
```

```
# Calculate the difference between two dates
date_diff <- as.numeric(difftime(current_date, date_object, units = "days"))
print(paste("Difference in days:", date_diff))
```

```
[1] "Difference in days: 781"
```

```
# Format a Date object to a specific string format
formatted_date <- format(current_date, "%B %d, %Y")
print(formatted_date)
```

```
[1] "November 20, 2025"
```

```
# Format a POSIXct object to a specific string format
formatted_datetime <- format(current_datetime, "%Y-%m-%d %H:%M:%S")
print(formatted_datetime)
```

```
[1] "2025-11-20 20:24:40"
```

These functions allow you to create, manipulate, and format date and time data in R. For more advanced date and time handling, you might consider using the **lubridate** package, which provides a more user-friendly interface for working with dates and times.

A practical, CDISC-friendly guide to working with **dates/times** and implementing **partial date imputation** in R using **base R** and **lubridate**.

51.1 1. Date & Time Basics in R

51.1.1 1.1 Core classes

- **Date**: calendar dates without time.
- **POSIXct**: seconds since 1970-01-01 (compact, good for storage/arithmetic).
- **POSIXlt**: list of components (year, mon, mday, hour, min, sec) — convenient to extract parts.

51.1.2 1.2 Converting numerics to Date (origins matter)

Different systems use different **origins** for numeric dates: - Excel: 1900-01-01 (beware 1900 leap-year bug in older Excels) - SAS: 1960-01-01 - R: 1970-01-01

```
num_dates <- c(0, 159, 464, 15674)

as.Date(num_dates, origin = "1960-01-01") # SAS origin
```

```
[1] "1960-01-01" "1960-06-08" "1961-04-09" "2002-11-30"
```

```
as.Date(num_dates, origin = "1970-01-01") # R origin
```

```
[1] "1970-01-01" "1970-06-09" "1971-04-10" "2012-11-30"
```

```
as.Date(num_dates, origin = "1900-01-01") # Excel origin
```

```
[1] "1900-01-01" "1900-06-09" "1901-04-10" "1942-12-01"
```

51.1.3 1.3 Custom date formats

```
x1 <- c("01/05/1965", "08/16/1975")
as.Date(x1, "%m/%d/%Y")

x2 <- c("05-01-1965", "16-08-1975")
as.Date(x2, "%d-%m-%Y")

x3 <- c("01MAY1965", "16AUG1975")
as.Date(x3, "%d%b%Y")

# Unknown format? Try multiple patterns
x4 <- c("01MAY1965MORNING", "16AUGUST1975")
as.Date(x4, tryFormats = c("%d%b%Y", "%d%B%Y"))
```

51.1.4 1.4 Datetime strings (ISO 8601 & time zones)

```
x <- c("2015-10-19T10:15", "2018-12-19T23:05")
# Keep the literal 'T' in the format string
as.POSIXct(x, format = "%Y-%m-%dT%H:%M", tz = "UTC")

# Extract parts
xt <- as.POSIXlt(x, format = "%Y-%m-%dT%H:%M")
format(xt, "%Y-%m-%d") # date part
format(xt, "%H:%M")   # time part
```

51.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic

Install & load: `install.packages("lubridate"); library(lubridate)`

51.2.1 2.1 Intuitive parsers

- `ymd()`, `mdy()`, `dmy()` for dates
- `ymd_hms()`, `ymd_hm()`, `ymd_h()` for date-times

```
ymd("20210529")
dmy("29/05/2021")
ymd_hm("2019/01/19T17:15") # delimiter agnostic
```

51.2.2 2.2 Accessors and rounding

```
x <- ymd_hms("2009-08-03 12:01:59")
year(x); month(x); mday(x); wday(x); hour(x); minute(x); second(x)

floor_date(x, "month") # 2009-08-01
ceiling_date(x, "month") # 2009-09-01
round_date(x, "month") # 2009-08-01
rollback(x) # last day of previous month
```

51.2.3 2.3 Timespans

- **Durations:** exact seconds, e.g., `ddays(2)`
- **Periods:** human units, e.g., `months(1)`
- **Intervals:** start-end pair, e.g., `interval(start, end)`

```
start <- ymd("2015-10-19")
end <- ymd("2018-12-19")
int <- interval(start, end)
as.duration(int) # exact seconds
as.period(int) # years/months/days
```

51.2.4 3 ADaM-Style Partial Date Imputation (Start & End)

Imputation rules vary by study; always follow the SAP. A common pattern:

51.2.5 Start date (AESTDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **day** = **01** (first of month)
- YYYY → impute **month** = **01**, **day** = **01** (01-Jan)

51.2.6 End date (AEENDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **last day of that month**
- YYYY → impute **31-Dec**

51.2.7 Censoring by death / last alive date

Additionally, bound AEENDT by death / last alive date (when present):

- CENSOR = coalesce(DTHDT, LASTALVDT)
- AEENDT = min(imputed_end, CENSOR)

51.2.8 Imputation flags

Add flags:

- AESTDTF, AEENDTF:
 - "D" → day imputed
 - "M,D" → month & day imputed
- Add "C" if we had to **censor the end date** at DTHDT / LASTALVDT.

```
library(dplyr, warn.conflicts = FALSE)
library(stringr, warn.conflicts = FALSE)
library(lubridate, warn.conflicts = FALSE)

df <- data.frame(
  USUBJID   = sprintf("01-001-%03d", 1:6),
  AESTDTC   = c("2015-10-19", "2015-10", "2015", "2019-02", "2020-02", "2018-12-31"),
  AEENDTC   = c("2018-12-19", "2018-10", "2017-05-29", "2019-02", "2020-02", "2020"),
  RANDDT    = ymd(c("2015-10-01", "2015-10-01", "2015-01-10", "2019-01-15", "2020-02-10", "2020-02-10")),
  DTHDT     = ymd(c(NA, NA, NA, NA, "2020-08-03", NA)),
  LASTALVDT = ymd(c(NA, NA, NA, NA, NA, "2020-12-15"))
)
```

```

df_imp <- df |>
mutate(
  # Length of partial date strings
  AESTDTC_LEN = str_length(AESTDTC),
  AEENDTC_LEN = str_length(AEENDTC),
  ## --- AE start date imputation (minimum: first possible date) ---
  AESTDT = case_when(
    is.na(AESTDTC) ~ as.Date(NA),
    AESTDTC_LEN == 10 ~ ymd(AESTDTC, quiet = TRUE),
    AESTDTC_LEN == 7 ~ ymd(str_c(AESTDTC, "-01"), quiet = TRUE),
    AESTDTC_LEN == 4 ~ ymd(str_c(AESTDTC, "-01-01"), quiet = TRUE),
    TRUE ~ as.Date(NA)
  ),
  AESTDTF = case_when(
    is.na(AESTDTC) ~ NA_character_,
    AESTDTC_LEN == 10 ~ "",
    AESTDTC_LEN == 7 ~ "D",
    AESTDTC_LEN == 4 ~ "M,D",
    TRUE ~ "UNK"
  ),
  ## --- AE end date imputation (maximum: latest possible date) ---
  AEENDT_RAW = case_when(
    is.na(AEENDTC) ~ as.Date(NA),
    AEENDTC_LEN == 10 ~ ymd(AEENDTC, quiet = TRUE),
    AEENDTC_LEN == 7 ~ ymd(str_c(AEENDTC, "-01"), quiet = TRUE),
    AEENDTC_LEN == 4 ~ ymd(str_c(AEENDTC, "-12-31"), quiet = TRUE),
    TRUE ~ as.Date(NA)
  ),
  AEENDT_RAW = case_when(
    is.na(AEENDTC) ~ NA_character_,
    AEENDTC_LEN == 10 ~ "",
    AEENDTC_LEN == 7 ~ "D",
    AEENDTC_LEN == 4 ~ "M,D",
    TRUE ~ "UNK"
  )
) %>%
mutate(
  AEENDT_RAW = if_else(
    AEENDTC_LEN == 7 & !is.na(AEENDT_RAW),
    ceiling_date(AEENDT_RAW, "month") - days(1),
    AEENDT_RAW
  ),

```

```

CENSOR = coalesce(DTHDT, LASTALVDT),
AEENDT = case_when(
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR ~ CENSOR,
  TRUE ~ AEENDT_RAW
),
AEENDTF = case_when(
  is.na(AEENDT) ~ AEENDTF_RAW,
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR & AEENDTF_RAW == "" ~ "C",
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR & AEENDTF_RAW != "" ~ str_c(
  TRUE ~ AEENDTF_RAW
)
)

```

51.3 4. Durations & Analysis Variables

51.3.1 4.1 AE duration (AEDUR) in days

52 Using difftime (base) or as.numeric on Date differences

```
AEDUR <- as.integer(out$AEENDT - out$ASTDT)
```

52.0.1 6.2 Handy lubridate parsers (subset)

- Dates: `ymd()`, `mdy()`, `dmy()`, `yq()`
 - Datetimes: `ymd_hms()`, `ymd_hm()`, `ymd_h()`
 - Times: `hms()`, `hm()`, `ms()`
 - Accessors: `year()`, `month()`, `mday()`, `wday()`, `hour()`, `minute()`, `second()`, `date()`
 - Rounding: `floor_date()`, `ceiling_date()`, `round_date()`, `rollback()`
-

53 Reading SAS Datasets (+ Cleaning)

```
library(haven)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(labelled)
```

We try to read a SAS dataset (e.g., SDTM DM). If not present, we **synthesize** an example.

```
dm_path <- "data/sdtm/dm.sas7bdat"

if (file.exists(dm_path)) {
  dm <- read_sas(dm_path)
} else {
  dm <- tibble::tibble(
    STUDYID = "XYZ123",
    USUBJID = sprintf("XYZ-%03d", 1:10),
    ARM = rep(c("Placebo", "Active"), length.out=10),
    AGE = c(55, 62, 47, 50, 71, 66, 45, 59, 53, 68),
    SEX = rep(c("M", "F"), length.out=10)
  )
  message("Synthesized `dm` since data/sdtm/dm.sas7bdat was not found.")
}
str(dm)
```

```

tibble [306 x 25] (S3: tbl_df/tbl/data.frame)
 $ STUDYID : chr [1:306] "CDISCPIL0T01" "CDISCPIL0T01" "CDISCPIL0T01" "CDISCPIL0T01" ...
  ..- attr(*, "label")= chr "Study Identifier"
 $ DOMAIN  : chr [1:306] "DM" "DM" "DM" "DM" ...
  ..- attr(*, "label")= chr "Domain Abbreviation"
 $ USUBJID : chr [1:306] "01-701-1015" "01-701-1023" "01-701-1028" "01-701-
1033" ...
  ..- attr(*, "label")= chr "Unique Subject Identifier"
 $ SUBJID  : chr [1:306] "1015" "1023" "1028" "1033" ...
  ..- attr(*, "label")= chr "Subject Identifier for the Study"
 $ RFSTDTC : chr [1:306] "2014-01-02" "2012-08-05" "2013-07-19" "2014-03-
18" ...
  ..- attr(*, "label")= chr "Subject Reference Start Date/Time"
 $ RFENDTC : chr [1:306] "2014-07-02" "2012-09-02" "2014-01-14" "2014-04-
14" ...
  ..- attr(*, "label")= chr "Subject Reference End Date/Time"
 $ RFXSTDTC: chr [1:306] "2014-01-02" "2012-08-05" "2013-07-19" "2014-03-
18" ...
  ..- attr(*, "label")= chr "Date/Time of First Study Treatment"
 $ RFXENDTC: chr [1:306] "2014-07-02" "2012-09-01" "2014-01-14" "2014-03-
31" ...
  ..- attr(*, "label")= chr "Date/Time of Last Study Treatment"
 $ RFICDTC : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Date/Time of Informed Consent"
 $ RFPENDTC: chr [1:306] "2014-07-02T11:45" "2013-02-18" "2014-01-14T11:10" "2014-
09-15" ...
  ..- attr(*, "label")= chr "Date/Time of End of Participation"
 $ DTHDTC  : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Date/Time of Death"
 $ DTHFL   : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Subject Death Flag"
 $ SITEID  : chr [1:306] "701" "701" "701" "701" ...
  ..- attr(*, "label")= chr "Study Site Identifier"
 $ AGE      : num [1:306] 63 64 71 74 77 85 59 68 81 84 ...
  ..- attr(*, "label")= chr "Age"
 $ AGEU     : chr [1:306] "YEARS" "YEARS" "YEARS" "YEARS" ...
  ..- attr(*, "label")= chr "Age Units"
 $ SEX      : chr [1:306] "F" "M" "M" "M" ...
  ..- attr(*, "label")= chr "Sex"
 $ RACE     : chr [1:306] "WHITE" "WHITE" "WHITE" "WHITE" ...
  ..- attr(*, "label")= chr "Race"
 $ ETHNIC   : chr [1:306] "HISPANIC OR LATINO" "HISPANIC OR LATINO" "NOT HISPANIC OR LATINO" ...
  ..- attr(*, "label")= chr "Ethnicity"

```

```

$ ARMCD      : chr [1:306] "Pbo" "Pbo" "Xan_Hi" "Xan_Lo" ...
  ..- attr(*, "label")= chr "Planned Arm Code"
$ ARM        : chr [1:306] "Placebo" "Placebo" "Xanomeline High Dose" "Xanomeline Low Dose" ..
  ..- attr(*, "label")= chr "Description of Planned Arm"
$ ACTARMCD   : chr [1:306] "Pbo" "Pbo" "Xan_Hi" "Xan_Lo" ...
  ..- attr(*, "label")= chr "Actual Arm Code"
$ ACTARM      : chr [1:306] "Placebo" "Placebo" "Xanomeline High Dose" "Xanomeline Low Dose" ..
  ..- attr(*, "label")= chr "Description of Actual Arm"
$ COUNTRY    : chr [1:306] "USA" "USA" "USA" "USA" ...
  ..- attr(*, "label")= chr "Country"
$ DMDTC      : chr [1:306] "2013-12-26" "2012-07-22" "2013-07-11" "2014-03-
10" ...
  ..- attr(*, "label")= chr "Date/Time of Collection"
$ DMDY       : num [1:306] -7 -14 -8 -8 -7 -21 NA -9 -13 -7 ...
  ..- attr(*, "label")= chr "Study Day of Collection"

```

53.1 Handling Labels & Missing

```

# Example: Convert blank strings "" to NA for character columns
convert_blanks_to_na <- function(x) {
  if (is.character(x)) x[x == ""] <- NA_character_
  x
}
dm <- dm |> mutate(across(where(is.character), convert_blanks_to_na))

```

53.2 Labelled to Factor (if needed)

```

if (inherits(dm$SEX, "labelled")) {
  dm <- dm |> mutate(SEX = to_factor(SEX))
}

```

53.3 Common Cleaning

```
dm <- dm |>
  mutate(
    AGEGR1 = cut(AGE, breaks=c(-Inf, 50, 65, Inf),
      labels=c("<50", "50-65", "65+"))
  )
```

54 Exercises

1. Read another SAS dataset (e.g., `sv.sas7bdat`) if available. If not, create a synthetic tibble.
2. Write a function to trim character whitespace for all character columns.
3. Make a clean factor for `ARM` with levels `Placebo < Active`.

55 Base R Functions & Apply Family

56 Common Utilities

```
x <- 1:10  
sum(x); mean(x); sd(x); var(x); quantile(x)
```

```
[1] 55
```

```
[1] 5.5
```

```
[1] 3.02765
```

```
[1] 9.166667
```

0%	25%	50%	75%	100%
1.00	3.25	5.50	7.75	10.00

```
seq(0, 1, by=0.1); rep(5, times=3)
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
[1] 5 5 5
```

57 Apply Family

```
m <- matrix(1:9, nrow=3)
apply(m, 1, mean) # row means
```

```
[1] 4 5 6
```

```
apply(m, 2, mean) # col means
```

```
[1] 2 5 8
```

```
lst <- list(a=1:3, b=10:12)
sapply(lst, mean) # simplifies result
```

```
  a  b
2 11
```

```
mapply(sum, 1:3, 10:12)
```

```
[1] 11 13 15
```


58 Subsetting Essentials

```
df <- data.frame(id=1:3, val=c(10,20,30))  
df[1, "val"]
```

```
[1] 10
```

```
df[df$val > 10, ]
```

```
  id val  
2  2  20  
3  3  30
```

59 Exercises

1. Use `apply` to get the max per column of a numeric matrix.
2. Write a base R snippet to compute IQR for each column of `mtcars`.
3. Compare `lapply` vs `sapply` in behavior on a list with mixed types.

60 Custom Functions & Validation

61 Function introduction

In R, a **function** is a reusable block of code that performs a specific task. Functions help you:

- Avoid repeating the same code again and again
- Organize code into logical pieces
- Make your code easier to read, test, and maintain

R has many **built-in functions** (e.g., `sum()`, `mean()`, `lm()`), and you can also define your **own functions**.

```
# Example: using a built-in function
x <- c(1, 2, 3, 4, 5)
mean(x)
```

```
[1] 3
```

61.1 Function components

A function has three parts:

- The `formals()`, the list of arguments that control how you call the function.
- The `body()`, the code inside the function.
- The `environment()`, the data structure that determines how the function finds the values associated with the names.

While the `formals` and `body` are specified explicitly when you create a function, the `environment` is specified implicitly, based on *where* you defined the function. The function environment always exists, but it is only printed when the function isn't defined in the global environment.

```
f02 <- function(x, y) {  
  # A comment  
  x + y  
}
```

```
formals(f02)
```

```
$x
```

```
$y
```

```
body(f02)
```

```
{  
  x + y  
}
```

```
environment(f02)
```

```
<environment: R_GlobalEnv>
```

61.2 Signalling conditions

There are three conditions that you can signal in code: errors, warnings, and messages.

- Errors are the most severe; they indicate that there is no way for a function to continue and execution must stop.
- Warnings fall somewhat in between errors and message, and typically indicate that something has gone wrong but the function has been able to at least partially recover.
- Messages are the mildest; they are way of informing users that some action has been performed on their behalf.

There is a final condition that can only be generated interactively: an interrupt, which indicates that the user has interrupted execution by pressing Escape, Ctrl + Break, or Ctrl + C (depending on the platform).

Conditions are usually displayed prominently, in a bold font or coloured red, depending on the R interface. You can tell them apart because errors always start with “Error”, warnings with “Warning” or “Warning message”, and messages with nothing.

```
stop("This is what an error looks like")

warning("This is what a warning looks like")

message("This is what a message looks like")
```

61.3 Ignoring conditions

You can choose to ignore conditions using `try()`, `tryCatch()` or `suppressMessages()`, `suppressWarnings()`.

```
# Example: ignoring a warning
suppressWarnings(log(-1))

suppressWarnings({
  warning("Uhoh!")
  warning("Another warning")
  1
})

suppressMessages({
  message("Hello there")
  2
})

suppressWarnings({
  message("You can still see me")
  3
})

suppressWarnings(library(dplyr))
suppressWarnings(library(ggplot2))
```

61.4 tryCatch function

The `tryCatch()` function allows you to handle conditions (errors, warnings, messages) in a controlled way. You can specify custom behavior for each type of condition.

```

result <- tryCatch({
  # Code that may produce an error
  log(-1)
},
warning = function(w) {
  cat("Caught a warning:", conditionMessage(w), "\n")
  NA # Return NA on warning
},
error = function(e) {
  cat("Caught an error:", conditionMessage(e), "\n")
  NA # Return NA on error
},
finally = {
  cat("Execution completed.\n")
})

```

Caught a warning: NaNs produced
Execution completed.

```
result
```

```
[1] NA
```

61.5 function scope

Functions in R have their own scope, meaning that variables defined inside a function are not accessible outside of it, and vice versa. This helps prevent naming conflicts and keeps functions self-contained.

```

my_function <- function() {
  x <- 10 # x is local to the function
  return(x)
}
my_function()

```

```
[1] 10
```

61.6 Creating Custom Functions

```
# Define a simple function to add two numbers
add_numbers <- function(a, b) {
  return(a + b)
}
# Call the function
add_numbers(3, 5)
```

```
[1] 8
```

61.7 Function Arguments

```
# Function with default argument
greet <- function(name = "Guest") {
  paste("Hello,", name)
}
greet()           # Uses default
```

```
[1] "Hello, Guest"
```

```
greet("Alice")   # Overrides default
```

```
[1] "Hello, Alice"
```

61.8 User-defined Functions in R Programming Language

User-defined functions are the functions that are created by the user. User defines the working, parameters, default parameter, etc. of that user-defined function. They can be only used in that specific code.

1. Single Input Single Output : Now create a function in R that will take a single input and return a single output.
2. Multiple Input Multiple Output : Now create a function in R Language that will take multiple inputs and return multiple outputs.


```

# Single Input Single Output
calculate_bmi <- function(data) {
  # BMI formula: weight (kg) / height (m)^2
  bmi <- data$weight / (data$height ^ 2)
  return(bmi)
}

# Example data
patient_data <- data.frame(
  weight = c(70, 80, 65), # in kg
  height = c(1.75, 1.8, 1.6) # in meters
)

# Calculate BMI
bmi_values <- calculate_bmi(patient_data)
print(bmi_values) # Outputs BMI for each patient

```

```
[1] 22.85714 24.69136 25.39062
```

```

# Multiple Input Multiple Output
calculate_summary_stats <- function(age, weight, height) {
  age_stats <- list(
    mean_age = mean(age, na.rm = TRUE),
    median_age = median(age, na.rm = TRUE),
    sd_age = sd(age, na.rm = TRUE)
  )

  weight_stats <- list(
    mean_weight = mean(weight, na.rm = TRUE),
    median_weight = median(weight, na.rm = TRUE),
    sd_weight = sd(weight, na.rm = TRUE)
  )

  height_stats <- list(
    mean_height = mean(height, na.rm = TRUE),
    median_height = median(height, na.rm = TRUE),
    sd_height = sd(height, na.rm = TRUE)
  )

  # Return a list containing all summary stats
  return(list(age = age_stats, weight = weight_stats, height = height_stats))
}

```

```

}

# Example data
age <- c(40, 45, 50, 55, 60)
weight <- c(70, 75, 65, 80, 68)
height <- c(1.75, 1.8, 1.6, 1.65, 1.7)

summary_stats <- calculate_summary_stats(age, weight, height)
print(summary_stats) # Outputs summary stats for age, weight, and height

```

```

$age
$age$mean_age
[1] 50

```

```

$age$median_age
[1] 50

```

```

$age$sd_age
[1] 7.905694

```

```

$weight
$weight$mean_weight
[1] 71.6

```

```

$weight$median_weight
[1] 70

```

```

$weight$sd_weight
[1] 5.94138

```

```

$height
$height$mean_height
[1] 1.7

```

```

$height$median_height
[1] 1.7

```

```

$height$sd_height
[1] 0.07905694

```

What is `{{}}` in R?

The `{{}}` syntax is known as curly-curly syntax, and it is part of the tidy evaluation framework in the `{r, lang}` package. It's used to capture variables directly and allows for easy manipulation within functions.

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
library(tidyr)
```

```
stat <- function(data, grp, var) {  
  # Group by specified grouping variables and summarize the statistics  
  summary_stats <- data |> group_by(across({{ grp }})) |> summarise(  
    n = as.character(n_distinct(USUBJID)),  
    mean = as.character(round(mean({{ var }}, na.rm = TRUE), 2)),  
    median = as.character(median({{ var }}, na.rm = TRUE)),  
    sd = as.character(round(sd({{ var }}, na.rm = TRUE), 3)),  
    min = as.character(min({{ var }}, na.rm = TRUE)),  
    max = as.character(max({{ var }}, na.rm = TRUE)),  
    .groups = 'drop' # Ungroup after summarising  
  )  
  
  # Pivot to wider format  
  transposed_data <- summary_stats |>  
  pivot_longer(  
    cols = c(n, mean, median, sd, min, max), # Columns to pivot  
    names_to = "stat", # Name for the new 'stat' column  
    values_to = "value" # Name for the new 'value' column  
  ) |> pivot_wider(  
    names_from = "stat",  
    values_from = "value"  
  )  
}
```

```

    names_from = {{ grp }},      # Treatment groups to become column names
    values_from = value          # Values for each statistic
  )

  return(transposed_data)
}

adsl <- haven::read_sas("./data/adam/adsl.sas7bdat")
age_stat <- stat(data=adsl,grp=c(TRT01A,TRT01AN),var=AGE)
wgt_stat <- stat(adsl,TRT01A,WEIGHTBL)

```

62 difference !,!!,!!!

In R, the symbols `!`, `!!`, and `!!!` are used in different contexts, primarily in logical operations and tidy evaluation. - `!` (Logical NOT): This is a logical operator used to negate a logical value. It takes a single logical value (TRUE or FALSE) and returns the opposite value. For example, `!TRUE` returns FALSE, and `!FALSE` returns TRUE. - `!!` (Unquote): In the context of tidy evaluation (used in packages like dplyr), `!!` is used to unquote a single variable or expression. It allows you to evaluate a variable or expression within a tidy evaluation context. For example, if you have a variable `var_name` that holds the name of a column, you can use `!!var_name` to refer to that column in a dplyr function. - `!!!` (Unquote-splice): This is also used in tidy evaluation and is used to unquote and splice a list of variables or expressions. It allows you to pass multiple arguments to a function by splicing a list. For example, if you have a list of column names, you can use `!!!` to pass them as individual arguments to a dplyr function.

```
stat_simple <- function(data, grp, var, id_var = "USUBJID") {
  # Convert strings to symbols
  grp_sym <- sym(grp)
  var_sym <- sym(var)
  id_var_sym <- sym(id_var)

  # Group by specified grouping variables and summarize the statistics
  summary_stats <- data |>
    group_by(!!grp_sym) |>
    summarise(
      n = as.character(n_distinct(!!id_var_sym)),
      mean = as.character(round(mean(!!var_sym, na.rm = TRUE), 2)),
      median = as.character(round(median(!!var_sym, na.rm = TRUE), 2)),
      sd = as.character(round(sd(!!var_sym, na.rm = TRUE), 3)),
      min = as.character(round(min(!!var_sym, na.rm = TRUE), 2)),
      max = as.character(round(max(!!var_sym, na.rm = TRUE), 2)),
      .groups = 'drop'
    )

  # Pivot to longer format then wider
  transposed_data <- summary_stats |>
    pivot_longer(
```

```

      cols = c(n, mean, median, sd, min, max),
      names_to = "stat",
      values_to = "value"
    ) |>
    pivot_wider(
      names_from = !!grp_sym,
      values_from = value
    ) |>
    arrange(factor(stat, levels = c("n", "mean", "median", "sd", "min", "max")))

  return(transposed_data)
}

adsl1 <- haven::read_sas("../data/adam/adsl.sas7bdat")
age_stat <- stat_simple(data=adsl1,grp="TRT01A",var="AGE")

```

```

#where function
adsl1 <- adsl |>
  dplyr::select(dplyr::where(is.character))

adsl2 <- adsl |>
  dplyr::select(dplyr::where(is.numeric))

# across functions structure and explanation

#across(.cols, .fns, ..., .names = NULL, .unpack = FALSE)

df <- data.frame(
  id = 1:6,
  age = c(21, 34, 28, NA, 45, 30),
  wt = c(70, 80, 65, 72, NA, 68),
  sex = c("M","F","F","M","F","M")
)

# 2) Summaries grouped by sex
df2 <- df |>
  group_by(sex) |>
  summarise(across(.cols=dplyr::where(is.numeric),
    .fns=list( mean = ~mean(., na.rm=TRUE),
               sd   = ~sd(., na.rm=TRUE)
             ),
    .names = "{.col}_{.fn}")

```

```

)

df_qc <- df |>
  group_by(sex) |>
  summarize(mm=mean(age,na.rm=TRUE),
            mn1=mean(wt,na.rm=TRUE))

df2 <- df |>
  group_by(sex) |>
  summarise(across(.cols=c(age, wt),
                    .fns=list(nrecs = ~dplyr::n(),
                              mean = ~mean(., na.rm=TRUE),
                              sd   = ~sd(., na.rm=TRUE)
                    ),
            .names = "{.col}_{.fn}")
)

df2 <- df |>
  group_by(sex) |>
  summarise(across(.cols=c(age, wt),
                    .fns= ~mean(., na.rm=TRUE))
)

a <- c(1, 2, 3, 4, 5, 9909)
b <- data.frame(
  var1 = c(1, 2, 3, 4, 5, 9909),
  var2 = c(5, 6, 7, 8, 9, 9909)
)
c <- list(a,b)

# Numeric: replace sentinel 9909 with 0
numericdata <- tibble::tribble(
  ~var1, ~var2, ~var3, ~var4, ~var5,
  9909, 1, 2, 3, 4,
  5, 9909, 6, 7, 8,
  9, 3, 9909, 2, 7
)

dm <- haven::read_sas("./data/sdtm/dm.sas7bdat") |>
  dplyr::mutate(dplyr::across(where(is.character),~ dplyr::na_if(.,"")))

numericdata01 <- numericdata |>

```

```
dplyr::mutate(dplyr::across(.cols=where(is.numeric), .fns=~ dplyr::if_else(.x == 9909, 0,

# multiply all numeric variables by 2
adsl3 <- adsl |>
  dplyr::mutate(
    dplyr::across(.cols = where(is.numeric),
                  .fns = ~ .x * 2)
  )
```


63 Error Handling

```
robust_divide <- function(a, b) {  
  tryCatch(a / b,  
    warning = function(w) NA_real_,  
    error   = function(e) NA_real_)  
}  
robust_divide(10, 0)
```

```
[1] Inf
```

64 Unit Testing with testthat

Install once: `install.packages(c("testthat","devtools","usethis","roxygen2"))`

```
usethis::use_testthat()
usethis::use_test("safe_mean")
```

Create `tests/testthat/test-safe_mean.R`:

```
testthat::test_that("safe_mean works", {
  x <- c(1,2,NA)
  testthat::expect_equal(safe_mean(x), 1.5)
  testthat::expect_error(safe_mean("oops"))
})
```

65 Document with roxygen2

```
#' Compute a safe mean
#' @param x Numeric vector
#' @param na.rm Logical; remove NAs
#' @return Numeric scalar
#' @examples
#' safe_mean(c(1,2,NA))
#' @export
safe_mean <- function(x, na.rm = TRUE) {
  stopifnot(is.numeric(x))
  mean(x, na.rm = na.rm)
}
```

Run:

```
devtools::document()
```

66 Exercises

1. Write `winsorize(x, probs=c(0.05,0.95))` and test it.
2. Create `validate_columns(df, required=c("USUBJID","AGE"))` and add tests.
3. Add roxygen docs and build help pages.

67 R Package Development

A practical, end-to-end cookbook for building robust **R packages** with **devtools**, **roxygen2**, and **testthat**. Written for day-to-day use, from scaffolding to checks and tests.

67.1 1. Prerequisites

- R (4.1 recommended so you have the base pipe |>)
- RTools (Windows) / Xcode command line tools (macOS) for compilation
- Recommended packages:
 - `install.packages(c("devtools", "roxygen2", "usethis", "testthat", "pkgdown"))`

```
# Load helpers  
library(devtools)
```

Loading required package: usethis

```
library(usethis)
```

67.2 2. Create the Package Skeleton

67.2.1 2.1 Create a new package directory

```
# Creates simutils with DESCRIPTION, NAMESPACE placeholder, R/ etc.  
#usethis::create_package("simutils")  
# Or from an existing project folder  
# usethis::create_package(getwd())
```

67.2.2 2.2 Initialize Git and a README

```
usethis::use_git()
usethis::use_readme_rmd() # README.Rmd + badge placeholders
```

67.2.3 2.3 The minimal structure (what you get)

```
simutils/
DESCRIPTION
NAMESPACE
R/
man/
```

Tip: Keep all your R code inside R/. Avoid subfolders there. Group related functions in sensible files (e.g., `na.R`, `sample.R`).

67.3 3. DESCRIPTION: Package Metadata

Run helpers and then edit as needed.

```
usethis::use_description(fields = list(
  Title = "Simulation Utilities",
  Description = "Convenience functions for sampling and NA counting.",
  `Authors@R` = 'person(given = "Your", family = "Name", role = c("aut","cre"), email = "you@your.org")',
  License = "MIT + file LICENSE"
))

usethis::use_mit_license("Your Name")
```

Key fields you usually set:

- Package, Title, Version, Description
- Authors@R (with aut, cre roles)
- License (e.g., MIT)
- Depends, Imports, Suggests (use helpers below)

67.4 4. NAMESPACE: Exports and Imports (auto-generated)

You generally **do not** edit **NAMESPACE** by hand. It's generated from your roxygen tags in code.

- `@export` marks a function for users.
- `@import`, `@importFrom` declare your dependencies at function level.

You'll generate it with `devtools::document()`.

67.5 5. Write Functions + roxygen2 Headers

Create your first R file:

```
# File: R/na.R
#' Count NAs in a vector
#'
#' Count number of missing values (NA) in a numeric/logical/character vector.
#'
#' @param x A vector.
#' @return Integer count of NAs in `x`.
#' @examples
#' sum_na(c(1, NA, 3))
#' sum_na(airquality$Ozone)
#' @export
sum_na <- function(x) {
  sum(is.na(x))
}
```

Add a second exported function that works on data frames/matrices:

```
# File: R/na_counter.R
#' Column-wise NA counts
#'
#' Returns the number of NAs per column for a data frame or matrix.
#'
#' @param x A data frame or matrix.
#' @return A named integer vector of NA counts per column.
#' @examples
#' na_counter(airquality)
#' @export
```

```
na_counter <- function(x) {
  if (!is.data.frame(x) && !is.matrix(x)) stop("x must be a data.frame or matrix")
  colSums(is.na(x))
}
```

A simple sampler utility with a documented argument:

```
# File: R/sample_from_data.R
#' Sample rows from a data frame
#'
#' Draw a row-wise sample with optional replacement.
#'
#' @param data A data frame.
#' @param size Number of rows to sample.
#' @param replace Logical; sample with replacement?
#' @return A data frame containing the sampled rows.
#' @examples
#' sample_from_data(airquality, 10)
#' sample_from_data(airquality, 10, replace = TRUE)
#' @export
sample_from_data <- function(data, size, replace = FALSE) {
  stopifnot(is.data.frame(data))
  idx <- sample.int(nrow(data), size, replace = replace)
  data[idx, , drop = FALSE]
}
```

67.6 6. Generate Documentation

```
# Parses roxygen headers -> creates/updates NAMESPACE and man/*.Rd
devtools::document()
```

This will populate `man/` with help files and refresh `NAMESPACE`.

67.7 7. Add Data, Vignettes, Tests (Optional but Recommended)

67.7.1 7.1 Package data

```
# Create example data
sim_dat <- data.frame(
  ID = 1:10,
  Value = sample(1:11, 10),
  Apples = sample(c(TRUE, FALSE), 10, replace = TRUE)
)

usethis::use_data(sim_dat, overwrite = TRUE)
```

67.7.2 7.2 Vignettes

```
usethis::use_vignette("getting-started")
# Edit vignettes/getting-started.Rmd, then:
devtools::build_vignettes()
```

67.7.3 7.3 Tests with testthat

```
usethis::use_testthat()
# Creates tests/testthat.R and tests/testthat/ directory

usethis::use_test("na_counter")
# -> tests/testthat/test-na_counter.R (edit and add tests)
```

Example tests:

```
# File: tests/testthat/test-na_counter.R

library(testthat)

test_that("na_counter counts NAs correctly for data.frame and matrix", {
  test_matrix <- matrix(c(NA, 1, 4, NA, 5, 6), nrow = 2)
  air_expected <- c(Ozone = 37, Solar.R = 7, Wind = 0, Temp = 0, Month = 0, Day = 0)
  mat_expected <- c(V1 = 1, V2 = 1, V3 = 0)
```

```

expect_equal(na_counter(airquality), air_expected)
expect_equal(na_counter(test_matrix), mat_expected)
})

# Non-exported functions can be called with ::: if truly needed
# expect_equal(simutils:::sum_na(airquality$Ozone), 37)

```

Common expectations:

- `expect_identical()`, `expect_equal(tolerance = ...)`, `expect_equivalent()`
- `expect_error()`, `expect_warning()`
- `expect_output()` / `expect_output_file()` for console text

Run the test suite:

```
devtools::test()
```

67.8 8. Manage Dependencies

Use helpers to declare dependencies in DESCRIPTION (rather than editing by hand):

```

usethis::use_package("dplyr")           # add to Imports
usethis::use_package("ggplot2", type = "Suggests")

```

Guidelines:

- **Imports:** packages needed at runtime for your functions.
- **Suggests:** optional packages used in examples/tests/vignettes.
- **Depends:** rarely needed; if used, it attaches packages for users (discouraged).

67.9 9. Check Your Package Thoroughly

```

# Full check like CRAN does; read 00check.log for details
rc <- devtools::check()
rc

```

Typical findings & quick fixes:

- **Missing fields** (e.g., `License`) → add via `use_mit_license()` or set in `DESCRIPTION`.
- **Undocumented args** → ensure every parameter appears in `@param` and usage matches.
- **Broken examples** → run your examples; fix typos (e.g., `airquality`, not `airuality`).
- **Missing system tools** for building PDF manual/vignettes → install LaTeX if you need PDFs.
- **Failing tests** → run `devtools::test()`; update your code or expectations.

67.10 10. Build & Install Locally

```
# Build source tarball
devtools::build()

# Install the package from source
devtools::install()

# Try it out
#library(simutils)
na_counter(airquality)
```

67.11 11. Continuous Integration (optional)

You can wire up CI so checks run automatically on each commit. (Any provider works; pick what your org supports.)

```
# Example: set up a CI service config (adjust per provider)
# usethis::use_github_action_check_standard()
# or legacy: usethis::use_travis()
```

67.12 12. Package Website (optional)

```
usethis::use_pkgdown()
pkgdown::build_site()
```

67.13 13. Release Checklist

- ☐ R CMD check passes locally with **0 errors, 0 warnings, 0 notes**
 - ☐ All exported functions documented with examples
 - ☐ Vignettes render without errors
 - ☐ Tests pass on your platforms (and CI)
 - ☐ NEWS.md updated; version bumped in DESCRIPTION
 - ☐ License added and included
-

67.13.1 Appendix A — Common Files (at a glance)

- DESCRIPTION: metadata & dependencies
- NAMESPACE: auto-generated exports/imports
- R/: function source files
- man/: help files (.Rd) generated by roxygen2
- tests/: unit tests (testthat)
- vignettes/: long-form docs
- data/ & R/sysdata.rda: package data

67.13.2 Appendix B — Troubleshooting Snippets

```
# 1) Missing License in check
usethis::use_mit_license("Your Name")

# 2) Undocumented argument in check
# -> Make sure @param includes it and re-run document()

devtools::document(); devtools::check()

# 3) Examples fail in check (typo)
# -> Fix the example code, rerun check

# 4) pdflatex not found during check
# -> Install a LaTeX distribution or run check without manual building

# 5) Dependency not available
# -> Ensure it's in Imports/Suggests and installed in your environment
```

Turn this file into your team's template. Duplicate it, search/replace `simutils`, and you're ready to scaffold a new package in minutes.

68 Git in RStudio (Setup & Auth)

68.1 One-Time Setup

- Install Git and ensure `git --version` works.
- In R:

```
usethis::use_git_config(user.name = "Your Name", user.email = "you@example.com")
```

68.2 Initialize Git for the Current Project

```
usethis::use_git()
```

68.3 Connect to GitHub

- Create a GitHub account.
- In R:

```
usethis::create_github_token()
gitcreds::gitcreds_set() # paste token when prompted
usethis::use_github(protocol = "https")
```

Or set up SSH keys via RStudio (Tools > Global Options > Git/SVN).

68.4 Typical Workflow

1. Stage changes (Git pane in RStudio).
2. Commit with a clear message.
3. Push to origin (GitHub).

68.5 Remove Git from a Project (macOS/RStudio)

- In Finder/Terminal, delete the hidden `.git` folder in the project root (careful!).
- Or from Terminal at project root:

```
rm -rf .git
```

- Reopen project in RStudio; Git pane will disappear.

Exercises - Create a new repo for this Quarto course and push it. - Branch, make a change, open a Pull Request on GitHub.

69 Creating ADaM: ADSL from SDTM-like Inputs

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(tidyr)  
library(lubridate)
```

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

We simulate **minimal** SDTM-like DM and EX to illustrate ADSL creation. If available, replace with your own data/sdtm/*.sas7bdat.


```
# DM
dm <- tibble::tibble(
  STUDYID = "XYZ123",
  USUBJID = sprintf("XYZ-%03d", 1:10),
  ARM = rep(c("Placebo","Active"), length.out=10),
  AGE = c(55, 62, 47, 50, 71, 66, 45, 59, 53, 68),
  SEX = rep(c("M","F"), length.out=10),
  RANDDT = as.Date("2025-01-15") + sample(0:20, 10, replace=TRUE)
)

# EX (first dose date)
ex <- tibble::tibble(
  USUBJID = dm$USUBJID,
  EXSTDTC = dm$RANDDT + sample(0:3, 10, replace=TRUE)
)
```

69.1 Build ADSL

```
adsl <- dm |>
  left_join(ex, by="USUBJID") |>
  transmute(
    STUDYID, USUBJID,
    TRT01P = ARM,
    TRT01PN = as.integer(factor(ARM, levels=c("Placebo","Active"))),
    AGE, SEX,
    RANDDT,
    TRTSDT = EXSTDTC,
    TRT01A = TRT01P,          # assume planned == actual for demo
    TRT01AN = TRT01PN
  )
adsl
```

```
# A tibble: 10 x 10
  STUDYID USUBJID TRT01P TRT01PN AGE SEX RANDDT TRTSDT TRT01A
  <chr>    <chr>   <chr>    <int> <dbl> <chr> <date>    <date>    <chr>
1 XYZ123 XYZ-001 Placebo      1  55 M  2025-01-24 2025-01-26 Placebo
2 XYZ123 XYZ-002 Active      2  62 F  2025-01-20 2025-01-22 Active
3 XYZ123 XYZ-003 Placebo      1  47 M  2025-01-31 2025-01-31 Placebo
4 XYZ123 XYZ-004 Active      2  50 F  2025-01-19 2025-01-21 Active
```

```

5 XYZ123 XYZ-005 Placebo      1    71 M    2025-01-20 2025-01-20 Placebo
6 XYZ123 XYZ-006 Active       2    66 F    2025-02-04 2025-02-06 Active
7 XYZ123 XYZ-007 Placebo      1    45 M    2025-01-21 2025-01-21 Placebo
8 XYZ123 XYZ-008 Active       2    59 F    2025-01-19 2025-01-22 Active
9 XYZ123 XYZ-009 Placebo      1    53 M    2025-01-18 2025-01-18 Placebo
10 XYZ123 XYZ-010 Active       2    68 F    2025-01-31 2025-01-31 Active
# i 1 more variable: TRT01AN <int>

```

Note: Real ADSL creation must follow **ADaM IG** (derive flags, dates, imputations, populations). This example is educational only.

Exercises 1. Add analysis populations (e.g., SAFFL, FASFL) based on simple rules. 2. Derive AGEGR1 as <65 / 65 and use ordered factor. 3. Add a treatment end date TRTEDT and compute treatment duration.

70 TLFs: Table, Figure, Listing

70.1 Create demographic table from ADSL

```
library(r2rtf)
library(dplyr)
library(tidyr)
library(haven)

adsl <- haven::read_sas("data/adam/adsl.sas7bdat")

bs_count <- function(data, grp, var,
                      var_label = var,
                      decimal = 1,
                      total = TRUE) {
  data <- data |>
    dplyr::rename(grp = !!grp, var = !!var)

  coding <- levels(factor(data$grp))

  data <- data |>
    dplyr::mutate(grp = as.numeric(factor(grp)))

  res <- with(data, table(var, grp)) |>
    as.data.frame() |>
    dplyr::mutate(grp = as.numeric(grp))

  if (total) {
    res_tot <- with(data, table(var)) |>
      as.data.frame() |>
      dplyr::mutate(grp = 9999)
    res <- dplyr::bind_rows(res, res_tot)
  }

  res <- res |>
```

```

    rename(n = Freq)

res <- res |>
  mutate(pct = formatC(n / sum(n) * 100, digits = decimal, format = "f", flag = "0"))

res$n <- as.character(res$n)

res <- res |>
  pivot_longer(cols = c(n, pct), names_to = "key", values_to = "value") |>
  unite(keys, grp, key) |>
  pivot_wider(names_from = keys, values_from = value) |>
  mutate(var_label = var_label) |>
  mutate(var = as.character(var))

names(res) <- gsub("_n", "", names(res), fixed = TRUE)
attr(res, "coding") <- coding

res
}

bs_continuous <- function(data, grp, var,
                          var_label = var,
                          decimal = 1,
                          total = TRUE,
                          blank_row = FALSE) {

  data <- data |>
    rename(grp = !!grp, var = !!var)
  coding <- levels(factor(data$grp))
  data <- data |> mutate(grp = as.numeric(factor(grp)))

  res <- data |>
    select(grp, var) |>
    na.omit() |>
    group_by(grp) |>
    summarise(
      n = n(),
      Mean = formatC(mean(var), digits = decimal, format = "f", flag = "0"),
      SD = formatC(sd(var), digits = decimal, format = "f", flag = "0"),
      Median = formatC(median(var), digits = decimal, format = "f", flag = "0"),
      Range = paste(range(var), collapse = " to ")
    )
}

```

```

if (total) {
  res_tot <- data |>
    select(grp, var) |>
    na.omit() |>
    summarise(
      n = n(),
      Mean = formatC(mean(var), digits = decimal, format = "f", flag = "0"),
      SD = formatC(sd(var), digits = decimal, format = "f", flag = "0"),
      Median = formatC(median(var), digits = decimal, format = "f", flag = "0"),
      Range = paste(range(var), collapse = " to ")
    ) |>
    mutate(grp = 9999)
  res <- bind_rows(res, res_tot)
}

res$"n" <- as.character(res$"n")

res <- res |>
  pivot_longer(cols = -grp, names_to = "key", values_to = "value") |>
  mutate(key = factor(key, levels = c("n", "Mean", "SD", "Median", "Range"))) |>
  pivot_wider(names_from = grp, values_from = value) %>%
  mutate(var_label = var_label) |>
  mutate(key = as.character(key)) |>
  rename(var = key)

if (blank_row) {
  res <- bind_rows(tibble(var_label = var_label), res)
}

res
}

# The code above define two utility function for baseline characteristic tables.

# Analysis Set

ana <- adsl |>
  subset(ITTFL == "Y")

ana <- ana |>
  mutate(
    RACE = factor(

```

```

    RACE,
    c("WHITE", "BLACK OR AFRICAN AMERICAN", "AMERICAN INDIAN OR ALASKA NATIVE"),
    c("White", "Black", "Other")
  ),
  SEX = factor(
    SEX,
    c("F", "M"),
    c("Female", "Male")
  ),
  AGEGR1 = factor(
    AGEGR1,
    levels = c("<65", "65-80", ">80")
  )
)

# Build Data for r2rtf
bs_tb <- bind_rows(
  bs_count(ana, "TRT01AN", "SEX", "Gender"),
  bs_count(ana, "TRT01AN", "AGEGR1", "Age (Years)"),
  bs_continuous(ana, "TRT01AN", "AGE", "Age (Years)", blank_row = TRUE),
  bs_count(ana, "TRT01AN", "RACE", "Race")
) |>
  mutate(
    var_label = factor(var_label, levels = c("Gender", "Age (Years)", "Race"))
  )

bs_tb[is.na(bs_tb)] <- "" # convert NA to blank string.

# reporting
bs_rtf <- bs_tb |>
  rtf_page(width = 9.5) |>
  rtf_title("Demographic and Anthropometric Characteristics", "ITT Subjects") |>
  rtf_colheader(" | Placebo | Drug Low Dose | Drug High Dose | Total ",
    col_rel_width = c(3, rep(2, 4))
  ) |>
  rtf_colheader(" | n | (%) | n | (%) | n | (%) | n | (%) ",
    col_rel_width = c(3, rep(c(1.2, 0.8), 4)),
    border_top = c("", rep("single", 8)),
    border_left = c("single", rep(c("single", ""), 4))
  ) |>
  rtf_body(
    page_by = "var_label",

```

```

col_rel_width = c(3, rep(c(1.2, 0.8), 4), 3),
text_justification = c("l", rep("c", 8), "l"),
text_format = c(rep("", 9), "b"),
border_left = c("single", rep(c("single", ""), 4), "single"),
border_top = c(rep("", 9), "single"),
border_bottom = c(rep("", 9), "single"),
) |>
rtf_footnote("This is a footnote") |>
rtf_source("Source: xxx")

# Output .rtf file
bs_rtf %>%
  rtf_encode() %>%
  write_rtf("rtf/bs_example.rtf")

```

Exercises 1. Format Table 1 to N (mean $\pm$ SD) for age. 2. Add risk table to the KM plot (use an extension like survminer outside of this minimal example). 3. Create a listing that includes population flags once you derive them.

71 Capstone: End-to-End Mini Workflow

This chapter ties everything together: **read data** → **derive ADSL** → **produce TLFs** → **render a report**.

71.1 Parameters

```
# You could parametrize paths via YAML; here we keep inline defaults.
dm_path <- "data/sdtm/dm.sas7bdat"
ex_path <- "data/sdtm/ex.sas7bdat"
```

71.2 1) Read (or Synthesize) SDTM

```
library(haven); library(dplyr); library(lubridate)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

```
if (file.exists(dm_path)) {
  dm <- read_sas(dm_path)
} else {
  dm <- tibble::tibble(
    STUDYID = "XYZ123",
    USUBJID = sprintf("XYZ-%03d", 1:60),
    ARM = sample(c("Placebo", "Active"), 60, replace=TRUE),
    AGE = round(rnorm(60, 60, 8)),
    SEX = sample(c("M", "F"), 60, replace=TRUE),
    RANDDT = as.Date("2025-01-15") + sample(0:40, 60, replace=TRUE)
  )
}
if (file.exists(ex_path)) {
  ex <- read_sas(ex_path)
} else {
  ex <- tibble::tibble(
    USUBJID = dm$USUBJID,
    EXSTDTC = dm$RANDDT + sample(0:3, nrow(dm), replace=TRUE)
  )
}
```

71.3 2) Derive ADSL (Minimal Demo)

```
adsl <- dm |>
  left_join(ex, by="USUBJID") |>
  mutate(
    TRT01P = ARM,
    TRT01PN = as.integer(factor(ARM, levels=c("Placebo", "Active"))),
    TRT01A = TRT01P,
    TRT01AN = TRT01PN,
    SAFFL = "Y",          # demo only; define rules in real life
    FASFL = "Y"
  ) |>
  dplyr::select(STUDYID.x, USUBJID, TRT01P, TRT01PN, TRT01A, TRT01AN, AGE, SEX, EXSTDTC, SAF)
```

[				
Table	1.	Baseline	by	Treatment
Table	1.	Baseline	by	Treatment
Description of Planned Arm	N	mean_age	sd_age	pct_female
Placebo	226	75.04867	8.503715	60.61947
Screen Failure	52	75.09615	9.699928	69.23077
Xanomeline High Dose	184	74.01087	7.939656	48.36957
Xanomeline Low Dose	181	75.29834	8.277778	60.77348

71.4 3) TLFs

```
library(gt); library(ggplot2); library(survival)
```

```
tbl1 <- adsl |>
  group_by(TRT01P) |>
  summarise(N=n(),
            mean_age = mean(AGE), sd_age = sd(AGE),
            pct_female = mean(SEX=="F")*100)
tbl1_gt <- gt(tbl1) |> tab_header(title="Table 1. Baseline by Treatment")
tbl1_gt
```

```
set.seed(123)
adsl$time <- rexp(nrow(adsl), rate=ifelse(adsl$TRT01P=="Active", 0.08, 0.1))
adsl$status <- rbinom(nrow(adsl), 1, 0.7)
fit <- survfit(Surv(time, status) ~ TRT01P, data=adsl)
# reuse plotting function from prior chapter
ggsurv <- function(fit) {
  ss <- summary(fit)
  dd <- data.frame(time=ss$time, surv=ss$surv, strata=rep(names(fit$strata), fit$strata))
  ggplot(dd, aes(x=time, y=surv, linetype=strata)) + geom_step() + theme_minimal() +
    labs(title="KM Curve (Toy)", x="Time", y="Survival", linetype="Treatment")
}
#ggsurv(fit)
```

71.5 4) Save Outputs

```
# Example: Save Table 1 as PNG  
#gtsave(tbl1_gt, "tlf-table1.png")
```

Challenge: Convert this chapter into a **parameterized report** (e.g., treatment subset or different cohort) and render multiple outputs.

72 Appendix: Tips, Profiles, .libPaths

72.1 Useful Profiles

Create `~/.Rprofile` to set options (be careful on shared systems):

```
options(  
  repos = c(CRAN = "https://cloud.r-project.org"),  
  scipen = 999  
)
```

72.2 Custom Library Paths

```
# In .Rprofile or project-level .Rprofile  
.libPaths(c("/path/to/Rlibs", .libPaths()))
```

72.3 Format vs formatC (quick recap)

```
x <- c(123.456, 0.00123456)  
format(x, digits = 4)
```

```
[1] "1.235e+02" "1.235e-03"
```

```
format(x, nsmall = 2)
```

```
[1] "1.23456e+02" "1.23456e-03"
```

```
formatC(x, digits = 3, format = "f")
```

```
[1] "123.456" "0.001"
```

72.4 POSIXct vs POSIXlt

- **POSIXct**: seconds since epoch (numeric), compact, fast.
- **POSIXlt**: list-like with components (year, mon, mday...), easier to extract parts.

72.5 Recommended Packages

- tidyverse, lubridate, janitor, gt, gtsummary, survival, broom, here.
- Pharma/CDISC: admiral, tlf/tern, pharmaverse meta-packages (explore as you grow).

72.6 Short Glossary

- **SDTM**: Study Data Tabulation Model (FDA submission standard for raw domains).
- **ADaM**: Analysis Data Model (derived analysis-ready datasets).
- **TLF**: Tables, Listings, Figures for reporting.